

**ACCESS - ADAPTIVE COGNITIVE AND EDUCATIONAL
SKILL SUPPORT**

MUHAMMAD ALIFF BIN AFFANDI

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

ACCESS - ADAPTIVE COGNITIVE AND EDUCATIONAL SKILL SUPPORT

MUHAMMAD ALIFF BIN AFFANDI

This report is submitted in partial fulfillment of the requirements for the
Bachelor of Computer Science (Software Development) with Honours.

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

2026

DECLARATION

I hereby declare that this project entitled
ACCESS - ADAPTIVE COGNITIVE AND EDUCATIONAL SKILL SUPPORT
is written by me and is my own effort and that no part has been plagiarized without
citations.

STUDENT : _____ Date : _____
(MUHAMMAD ALIFF BIN AFFANDI)

I hereby declare that I have read this project report and found this project report is
sufficient in term of the scope and quality for the award of Bachelor of Computer
Science (Software Development) with Honours.

SUPERVISOR : _____ Date : _____
(TS. DR. ZURAINI BINTI OTHMAN)

DEDICATION

To my beloved parents, whose endless sacrifices and unwavering faith have been my foundation through every challenge. To Ts. Dr. Zuraini Binti Othman, whose mentorship and invaluable guidance shaped not only this research but also my growth as a researcher and software developer. To all neurodivergent learners who navigate a world not yet built for them, may ACCESS serve as a small step toward a future where educational platforms adapt to every mind rather than demanding that every mind adapt to them.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, Ts. Dr. Zuraini Binti Othman, for her continuous guidance, patience, and insightful feedback throughout the development of this project. Her expertise in software engineering and dedication to accessibility has been instrumental in shaping the ACCESS platform into a functional and meaningful system.

My heartfelt thanks go to my family and friends, who offered unwavering emotional and moral support during the challenging moments of this journey. Their encouragement was a constant source of motivation during late nights and difficult debugging sessions.

ABSTRACT

Traditional e-learning platforms are commonly designed using a one-size-fits-all approach, creating accessibility challenges for neurodivergent learners, particularly those with Dyslexia, Attention Deficit Hyperactivity Disorder (ADHD), and Autism Spectrum Disorder (ASD). Existing learning management systems often provide limited support for these diverse cognitive needs, resulting in increased cognitive load and reduced learning effectiveness. This project presents the design and development of the Adaptive Cognitive and Educational Skill Support (ACESS) platform, a web-based learning management system that incorporates adaptive accessibility features to support learners with different cognitive profiles. The project adopts the Agile Scrum methodology, involving iterative planning, analysis, design, and development activities across six sprints, and is implemented using Next.js, Supabase, PostgreSQL, and Tailwind CSS. Key features include configurable accessibility presets for Dyslexia, ADHD and Autism, a distraction-free reading interface, an adaptive content recommendation engine, an authoring-time accessibility compliance checker, an AI-assisted lesson summary and question-answering assistant, embedded H5P interactive content, and educator analytics, achievements, leaderboard ranking, and certificate generation for monitoring learner progress. The platform targets WCAG 2.2 Level AA conformance, though this has not yet been independently verified through formal audit or assistive-technology testing. The developed system was evaluated through functional test cases executed against the running platform, a WCAG 2.2-based accessibility self-assessment carried out by the project author, and user acceptance testing in which participants worked through predefined user perspectives covering the default learner experience, each of the three accessibility presets, the educator and the administrator. The evaluation was not carried out by an independent accessibility specialist and did not involve neurodivergent participants. The outcomes of this project demonstrate the feasibility of integrating accessibility and adaptive learning mechanisms into a single platform, providing a foundation for a more inclusive digital learning environment for neurodivergent learners.

ABSTRAK

Platform e-pembelajaran tradisional sering direka menggunakan pendekatan satu-saiz-untuk-semua, yang menimbulkan cabaran kebolehcapaian bagi pelajar neurodivergen, khususnya mereka yang mengalami Disleksia, Attention Deficit Hyperactivity Disorder (ADHD), dan Autism Spectrum Disorder (ASD). Sistem pengurusan pembelajaran sedia ada sering memberikan sokongan terhadap keperluan kognitif yang pelbagai ini, sekali gus meningkatkan beban kognitif dan mengurangkan keberkesanan pembelajaran. Projek ini membentangkan reka bentuk dan pembangunan platform Adaptive Cognitive and Educational Skill Support (ACESS), sebuah sistem pengurusan pembelajaran berasaskan web yang menggabungkan ciri kebolehcapaian adaptif bagi menyokong pelajar dengan profil kognitif yang berbeza. Projek ini menggunakan metodologi Agile Scrum, melibatkan aktiviti perancangan, analisis, reka bentuk dan pembangunan secara berulang merentasi enam sprint, dan dibangunkan menggunakan Next.js, Supabase, PostgreSQL, dan Tailwind CSS. Ciri utama termasuk praset kebolehcapaian boleh konfigurasi bagi Disleksia, ADHD dan Autisme, antara muka bacaan bebas gangguan, enjin pengesyoran kandungan adaptif, pengaudit pematuhan kebolehcapaian semasa penulisan kandungan, pembantu ringkasan pelajaran dan soal jawab berasaskan AI, kandungan interaktif H5P terbenam, serta analitik pendidik, pencapaian, kedudukan papan pendahulu, dan penjanaan sijil bagi memantau kemajuan pelajar. Platform ini menyasarkan pematuhan WCAG 2.2 Peringkat AA, walaupun perkara ini belum disahkan secara bebas melalui audit formal atau pengujian teknologi bantuan. Sistem yang dibangunkan dinilai melalui kes ujian berfungsi yang dilaksanakan terhadap platform yang beroperasi, penilaian sendiri kebolehcapaian berasaskan WCAG 2.2 yang dijalankan oleh penulis projek, serta Ujian Penerimaan Pengguna yang melibatkan peserta yang menjalankan tugas mengikut perspektif pengguna yang ditetapkan, meliputi pengalaman pelajar lalai, ketiga-tiga praset kebolehcapaian, pendidik dan pentadbir. Penilaian ini tidak dijalankan oleh pakar kebolehcapaian bebas dan tidak melibatkan peserta neurodivergen. Hasil projek ini menunjukkan kebolehlaksanaan menggabungkan mekanisme kebolehcapaian dan pembelajaran adaptif dalam satu platform, seterusnya menyediakan asas bagi persekitaran pembelajaran digital yang lebih inklusif untuk pelajar neurodivergen.

TABLE OF CONTENTS

	PAGE
DECLARATION	ii
DEDICATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
ABSTRAK	vi
TABLE OF CONTENTS	vii
LIST OF TABLES	xii
LIST OF FIGURES	xv
LIST OF ABBREVIATIONS	xvii
CHAPTER 1: INTRODUCTION	1
1.1 Background of Study	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope	3
1.4.1 Target Users	3
1.4.2 Core System Modules	4
1.5 Significance of Project	5
1.6 Expected Output	6
1.7 Conclusion	7
CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY	8
2.1 Introduction	8
2.2 Theoretical Frameworks for Cognitive Accessibility	8
2.2.1 Universal Design for Learning	8
2.2.2 Web Content Accessibility Guidelines 2.2	9

2.3	Neurodivergent Profiles and Interface Interventions	10
2.3.1	Attention Deficit Hyperactivity Disorder	10
2.3.2	Dyslexia	11
2.3.3	Autism Spectrum Disorder	12
2.4	Comparative Analysis of Existing LMS Platforms	12
2.4.1	Moodle	13
2.4.2	Canvas LMS	13
2.4.3	Blackboard Learn	14
2.4.4	Google Classroom	14
2.5	Comparison Matrix	15
2.5.1	Alternative Approaches and Justification	16
2.6	Project Methodology	17
2.7	Project Requirements	18
2.7.1	Software Requirements	18
2.7.2	Hardware Requirements	20
2.7.3	Other Requirements	21
2.8	Project Schedule and Milestones	21
2.9	Summary	22
CHAPTER 3: SYSTEM ANALYSIS		23
3.1	Introduction	23
3.2	Problem Analysis	23
3.2.1	Current System Scenario	23
3.2.2	Detailed Problem Breakdown	24
3.3	Requirement Analysis	25
3.3.1	Data Requirement	25
3.3.2	Functional Requirement	26
3.3.3	Non-Functional Requirement	31
3.3.4	Other Requirements	34
3.4	System Architecture	35
3.4.1	Context Diagram	35
3.4.2	System Architecture Diagram	37
3.4.3	Deployment Diagram	39

3.5	Module Design	40
3.6	Summary	41
CHAPTER 4: SYSTEM DESIGN		42
4.1	Introduction	42
4.2	High-Level Design	42
4.2.1	Behavioural Design	42
4.2.2	Interaction Design	47
4.2.3	User Interface Design	52
4.2.3.1	Navigation Design	52
4.2.3.2	Input Design	53
4.2.3.3	Output Design	55
4.2.4	Conceptual and Logical Database Design	59
4.3	Accessibility Design	66
4.3.1	Design approach and standards baseline	66
4.3.2	Mapping of standards to implementation	67
4.3.3	The three learner profiles	68
4.3.4	Preset composition	69
4.3.5	Effect of the presets on the lesson interface	70
4.3.6	Authoring-time accessibility compliance checking	74
4.3.7	Known accessibility gaps	76
4.4	Data Dictionary and Normalisation	77
4.4.1	Normalisation	77
4.4.2	Core Identity and Configuration	79
4.4.3	Curriculum Structure	83
4.4.4	Lesson Content and Interaction	89
4.4.5	Assessment	92
4.4.6	Learner Progress, Recognition and Recommendation	94
4.5	Detailed Design	98
4.5.1	Software Design	98
4.5.2	Physical Database Design	103
4.5.3	Other Implementation Decisions	107
4.6	Limitations of Design	108

4.7	Summary	110
CHAPTER 5: IMPLEMENTATION		112
5.1	Introduction	112
5.2	Development Environment Setup	112
5.2.1	Hardware Environment	112
5.2.2	Software Environment	113
5.2.3	Local Backend Environment	114
5.3	Software Configuration Management	115
5.3.1	Configuration Environment Setup	115
5.3.2	Version Control Procedure	116
5.4	Implementation Status	118
5.4.1	Verification Performed During Implementation	119
5.5	Conclusion	120
CHAPTER 6: TESTING		122
6.1	Introduction	122
6.2	Test Plan	123
6.2.1	Test Organisation	123
6.2.2	Test Environment	124
6.2.3	Test Schedule	125
6.3	Test Strategy	126
6.3.1	Classes of Tests	126
6.4	Test Design	127
6.4.1	Test Data	127
6.4.2	Functional Test Cases	128
6.4.3	Load Testing	136
6.4.4	Manual Accessibility Evaluation	137
6.4.5	User Acceptance Testing	139
6.5	Test Results and Analysis	144
6.5.1	Functional Test Results	144
6.5.2	Defects Identified and Resolved	145
6.5.3	Load Test Results	147
6.5.4	Accessibility Evaluation Results	148

6.5.5	User Acceptance Testing Results	149
6.5.6	Limitations of the Testing Performed	149
6.6	Conclusion	150
CHAPTER 7: CONCLUSION		152
7.1	Introduction	152
7.2	Project Summarisation	152
7.3	Project Contribution	154
7.4	Project Limitation	155
7.5	Future Works	156
7.6	Conclusion	157
References		159

LIST OF TABLES

	PAGE
Table 2.1 LMS Accessibility Capability Comparison	15
Table 2.2 Project Schedule and Milestones	21
Table 3.1 Functional Requirements of the Learner Module	27
Table 3.2 Functional Requirements of the Educator Module	29
Table 3.3 Functional Requirements of the Administrator Module	30
Table 4.1 Input Validation Rules	54
Table 4.2 System Outputs	57
Table 4.3 Mapping of Accessibility Criteria to ACESS Features	67
Table 4.4 Learner Profiles and the Support Provided by ACESS	69
Table 4.5 Accessibility Preset Values	70
Table 4.6 Users Data Dictionary	79
Table 4.7 User Profiles Data Dictionary	80
Table 4.8 Referral Codes Data Dictionary	80
Table 4.9 Instructor Applications Data Dictionary	81
Table 4.10 Contact Messages Data Dictionary	81
Table 4.11 Notifications Data Dictionary	82
Table 4.12 Accessibility Templates Data Dictionary	82
Table 4.13 Adaptive Interactions Data Dictionary	82
Table 4.14 Courses Data Dictionary	83
Table 4.15 Course Chapters Data Dictionary	84
Table 4.16 Lessons Data Dictionary	85
Table 4.17 Course Accessibility Categories Data Dictionary	86
Table 4.18 Course Milestones Data Dictionary	87
Table 4.19 Course Achievements Data Dictionary	87

Table 4.20 Lesson Templates Data Dictionary	88
Table 4.21 Lesson Versions Data Dictionary	88
Table 4.22 Media Assets Data Dictionary	89
Table 4.23 Lesson Interactive Content Data Dictionary	89
Table 4.24 Video Questions Data Dictionary	90
Table 4.25 H5P Contents Data Dictionary	90
Table 4.26 Lesson Checkpoints Data Dictionary	91
Table 4.27 Lesson AI Summaries Data Dictionary	91
Table 4.28 Lesson Comments Data Dictionary	92
Table 4.29 Quizzes Data Dictionary	92
Table 4.30 Quiz Questions Data Dictionary	93
Table 4.31 Quiz Options Data Dictionary	93
Table 4.32 Quiz Attempts Data Dictionary	93
Table 4.33 Quiz Answers Data Dictionary	94
Table 4.34 Enrollments Data Dictionary	94
Table 4.35 Lesson Progress Data Dictionary	95
Table 4.36 Learner Checkpoints Data Dictionary	95
Table 4.37 H5P Responses Data Dictionary	96
Table 4.38 Recommendations Data Dictionary	96
Table 4.39 Course Favorites Data Dictionary	97
Table 4.40 User Achievements Data Dictionary	97
Table 4.41 Certificates Data Dictionary	97
Table 4.42 Tables Removed During Schema Consolidation	104
Table 5.1 Development Workstation Specification	113
Table 5.2 Software Environment and Versions	113
Table 5.3 Implementation Status of Each Module	118
Table 5.4 Executable Verification Checks	120
Table 6.1 Test Organisation	123
Table 6.2 Seeded Test Accounts	125
Table 6.3 Test Schedule	126
Table 6.4 Functional Test Cases and Results	129
Table 6.5 Load Test Results	137

Table 6.6	WCAG 2.2 Self-Assessment Findings	138
Table 6.7	User Acceptance Testing Perspectives	140
Table 6.8	User Acceptance Testing Task Briefs	141
Table 6.9	Common Agreement Statements Asked of Every Perspective	143
Table 6.10	Perspective-Specific Agreement Statements	143
Table 6.11	Summary of Functional Test Results	144
Table 6.12	Defects Identified During Testing and Their Resolution	145
Table 6.13	Summary of the WCAG 2.2 Self-Assessment	148

LIST OF FIGURES

	PAGE
Figure 2.1 Gantt Chart of the ACESS Development Schedule	22
Figure 3.1 Current System Problem Analysis Diagram	24
Figure 3.2 Use Case Diagram for the ACESS Platform	27
Figure 3.3 Context Diagram	36
Figure 3.4 System Architecture Diagram	37
Figure 3.5 Deployment Diagram	39
Figure 4.1 Learner Activity Diagram	43
Figure 4.2 Educator Activity Diagram	45
Figure 4.3 Administrator Activity Diagram	46
Figure 4.4 Course Enrolment Sequence Diagram	47
Figure 4.5 Lesson Learning Sequence Diagram	48
Figure 4.6 Quiz Attempt Sequence Diagram	49
Figure 4.7 Course Creation Sequence Diagram	51
Figure 4.8 Learner Dashboard in the Default State	53
Figure 4.9 Educator Course Workspace, Lessons Tab	54
Figure 4.10 Accessibility Settings Panel, Reading Group	56
Figure 4.11 Quiz Interface	56
Figure 4.12 Learner Progress Page	58
Figure 4.13 Administrator Analytics Dashboard	58
Figure 4.14 Accessibility Analytics	59
Figure 4.15 Identity, Access and Platform Entity Relationship Diagram	62
Figure 4.16 Curriculum and Content Entity Relationship Diagram	63
Figure 4.17 Assessment, Learning Record and Recognition Entity Relationship Diagram	64

Figure 4.18 Complete ACCESS Entity Relationship Diagram	65
Figure 4.19 Preset Preview Dialog	71
Figure 4.20 Lesson Interface: Default Compared with the Dyslexia Preset . . .	72
Figure 4.21 Lesson Interface Under the ADHD Preset	73
Figure 4.22 Lesson Interface Under the Autism Preset	73
Figure 4.23 Learner Dashboard Under the Dyslexia Preset	74
Figure 4.24 Lesson Accessibility Check in the Educator Editor	75

LIST OF ABBREVIATIONS

ACCESS	- Adaptive Cognitive and Educational Skill Support
ADHD	- Attention Deficit Hyperactivity Disorder
AI	- Artificial Intelligence
API	- Application Programming Interface
ASD	- Autism Spectrum Disorder
CSS	- Cascading Style Sheets
H5P	- HTML5 Package, an interactive content format
HTML	- Hypertext Markup Language
JSON	- JavaScript Object Notation
LMS	- Learning Management System
PDF	- Portable Document Format
QR	- Quick Response, as in a QR code
UAT	- User Acceptance Testing
UDL	- Universal Design for Learning
URL	- Uniform Resource Locator
UUID	- Universally Unique Identifier
W3C	- World Wide Web Consortium
WCAG	- Web Content Accessibility Guidelines

CHAPTER 1: INTRODUCTION

1.1 Background of Study

Teaching today is delivered through a mixture of face-to-face classes and online platforms (Prensky, 2001). Digital tools have become a routine part of instruction, yet the platforms that carry the content are usually built around a single, fixed way of presenting information. This creates difficulties for neurodivergent learners, particularly those with Dyslexia, Attention Deficit Hyperactivity Disorder (ADHD) and Autism Spectrum Disorder (ASD). Whether a lesson is delivered in a classroom or online, the tools used to present it rarely offer the flexibility that different cognitive profiles require.

Most Learning Management Systems (LMS) follow a one-size-fits-all design. They present dense blocks of text, deep navigation menus and rigid assessment formats. For a neurodivergent learner these standard interfaces become barriers rather than aids. Learners with Dyslexia may struggle with the default typography and line spacing (Franceschini et al., 2013), while learners with ADHD may find a cluttered dashboard and an unstructured learning path difficult to work through (Barkley, 1997).

There is therefore a need for a learning platform in which adaptation is built in rather than added on. The Adaptive Cognitive and Educational Skill Support (ACESS) platform is proposed to address this need. ACESS adjusts its interface and the way it presents content according to the accessibility preferences a learner selects. Through adjustable typography, distraction-reduced layouts and rule-based content recommendations, the platform aims to make learning material easier to work with for learners with different cognitive needs.

1.2 Problem Statement

Although a wide range of educational technology is available, cognitive accessibility barriers remain. The central problem addressed by this project is that current educational platforms are not designed to support learners with cognitive disabilities. The specific problems identified are as follows.

1. Most e-learning platforms are not designed for learners with disabilities. Their presentation formats place a heavy load on cognitive and sensory processing, which contributes to higher dropout. Reported figures indicate that only 46% of neurodivergent learners completed online courses compared with 74% of their neurotypical peers (Spencer and Podesta, 2022), and that up to 35% of learners with ADHD leave e-learning programmes within the first semester because of poor accessibility design (Burl and O'Brien, 2024).
2. Learners with disabilities lack platforms that adapt to their individual learning pace and present content in a format they can process comfortably.
3. Employers and support organisations have no reliable way to verify the skills gained by these learners, because progress tracking and accessible certification are not provided.
4. Educators have no single platform in which they can monitor learner progress, identify learners who need additional support, and manage accessible learning materials.

1.3 Objectives

This project aims to achieve the following objectives.

1. To analyse the accessibility requirements, learning challenges and content delivery needs of learners with Dyslexia, ADHD and Autism Spectrum Disorder through

a literature review and a comparative study of existing e-learning platforms and accessibility standards.

2. To develop a functional accessible web-based e-learning platform that provides rule-based adaptive content delivery, built-in text-to-speech support, adjustable accessibility settings, structured lesson and quiz management, and role-based user management for learners, educators and administrators.
3. To implement a certificate generation module and an educator analytics dashboard that provide verifiable records of course completion and support the monitoring of individual learner progress.
4. To evaluate the functional correctness, usability and accessibility of the developed platform through functional test cases, a WCAG 2.2-based accessibility self-assessment carried out by the project author, and User Acceptance Testing with participants working to predefined user perspectives.

1.4 Scope

The scope of the project defines the intended users of the platform and the functional modules that make up the system. ACCESS is intended as a supporting tool for both classroom-based and online learning.

1.4.1 Target Users

- **Learners.** The primary users of the learning content. The interface is designed for neurodivergent learners, although the same features remain useful to any learner.
- **Educators.** The authors of the courses, who require structured tools to write accessible content, manage media and monitor learner progress.
- **Administrators.** The operators of the platform, responsible for user and role management, course moderation and platform-level oversight.

1.4.2 Core System Modules

The platform is organised into the following modules.

- **Authentication and Profile Module.** Manages registration, login, email verification, password reset, profile management and role-based access control for the three user roles.
- **Accessibility and Personalisation Module.** Stores each learner's accessibility preferences and applies them across the whole platform. It provides control over typeface, text size, spacing, background tint, distraction-free reading, motion and text-to-speech, and applies the settings without requiring the learner to reconfigure them for each course.
- **Course Authoring Module.** Allows educators to build courses with chapters and milestones, write lessons in a rich-text editor, embed video with time-coded questions, create interactive activities, keep a version history of each lesson, and configure quizzes, certificates and achievements.
- **Accessibility Compliance Module.** Checks each lesson and course against a fixed set of rules drawn from WCAG 2.2, World Wide Web Consortium (W3C) cognitive accessibility guidance and the British Dyslexia Association Style Guide, selected according to the accessibility focus declared for the course. It reports each unmet rule, names the standard the rule comes from, and offers a direct route to the fix. The result advises the educator and does not prevent publication.
- **Adaptive Learning Module.** Manages course enrolment, lesson progression and the rule-based recommendation engine, which suggests revision, next-lesson or advanced content according to the learner's quiz results and progress.
- **Assessment Module.** Delivers timed and untimed quizzes, supports multiple attempts, grades answers automatically, allows answers to be reviewed, and evaluates the result against the pass threshold set by the educator.
- **Analytics Module.** Records engagement measures such as time spent on a lesson,

completion rate and quiz scores, and presents them to educators as per-course and per-learner views and to administrators as platform-wide views.

- **Certificate and Achievement Module.** Generates a personalised completion certificate in PDF form and awards badges when a learner meets the criteria defined for a course.
- **Interactive Content Module.** Supports five activity types (flashcards, drag-and-drop, fill-in-the-blanks, memory game and timeline), time-coded video questions, embedded H5P content, and an optional assistant that produces a plain-language lesson summary and answers questions about the lesson currently open.
- **Communication and Notification Module.** Manages lesson discussion threads, a contact form, and notifications raised automatically on enrolment, lesson progress, quiz attempts, new lessons and course publication.
- **Localisation Module.** Presents the interface in English or Bahasa Melayu, switchable by the learner and saved to the learner's profile.

1.5 Significance of Project

The significance of ACESS lies in moving the responsibility for accessibility from the learner to the system. Instead of expecting a learner to install browser extensions or work around an unsuitable interface, the platform applies the learner's stated preferences automatically, so that attention can be spent on the learning material itself.

For learners, the platform provides a predictable and adjustable environment that reduces reading effort and visual distraction. For educators, it provides authoring tools that make accessibility problems visible while a lesson is being written, together with analytics on learner engagement. For the faculty and for future work in this area, the project demonstrates how accessibility features can be built into the architecture of a learning platform rather than added to it afterwards.

1.6 Expected Output

The following outputs are expected on completion of the project.

- **Adaptive web application.** A working web-based learning management system that adjusts its presentation according to the accessibility settings of the signed-in learner.
- **Role-based dashboards.** Three separate interfaces for learners, educators and administrators, each with its own navigation and permissions.
- **Accessibility settings and presets.** Three accessibility presets (Dyslexia, ADHD and Autism) together with individual controls for typeface, text size, line and word spacing, background tint, reading width, layout, motion, read-aloud and executive-function supports. A preset shows what it will change before it is applied, and every value it sets remains adjustable afterwards.
- **Adaptive recommendation engine.** A rule-based engine that reads quiz results and progress and produces lesson recommendations with a stated reason for each.
- **Course authoring tools.** A rich-text lesson editor with media upload, five interactive activity builders, in-video question authoring and version snapshots for recovery.
- **Accessibility compliance checker.** A checker that scores each lesson and course against a fixed rule set, reports the measured value behind each unmet rule, and points to where it can be corrected.
- **Quiz and assessment system.** Timed and untimed quizzes with automatic grading, attempt management and answer review.
- **Analytics dashboards.** Engagement and progress visualisations for educators at course and learner level, and for administrators at platform level.
- **Certificate and achievement system.** Automatic PDF certificate generation on course completion, with public verification, and a badge system for learner milestones.

- **Notification system.** Notifications raised automatically for enrolment, quiz results, new lessons and course publication.

1.7 Conclusion

This chapter has established the context of the ACCESS project. It identified the accessibility barriers present in current educational platforms, stated the problems the project addresses, and set out the objectives, scope, significance and expected outputs of the system. The next chapter reviews the relevant literature and standards, compares four established learning management systems against the needs of neurodivergent learners, and describes the methodology, requirements and schedule adopted for the project.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

This chapter presents the academic and technical basis of the ACCESS platform. It reviews current research on cognitive accessibility in digital learning, examines the frameworks that guide accessible design, and compares four widely used learning management systems against the needs of neurodivergent learners. The chapter then describes the project methodology, the software and hardware requirements, and the project schedule.

2.2 Theoretical Frameworks for Cognitive Accessibility

The design of ACCESS rests on two established frameworks: the Universal Design for Learning (UDL) guidelines (Meyer et al., 2014) and the Web Content Accessibility Guidelines (WCAG) 2.2 (W3C Web Accessibility Initiative, 2023).

2.2.1 Universal Design for Learning

Universal Design for Learning is a framework developed by CAST (Meyer et al., 2014) for designing instruction that works for the widest possible range of learners. It rests on three principles, each of which is reflected in a corresponding feature of ACCESS.

- **Multiple means of engagement.** Learners differ in what motivates them, and an unstructured environment is particularly difficult for learners with ADHD (Barkley,

1997). ACESS addresses this through achievement badges and through lessons that are presented in sections rather than as a single long page.

- **Multiple means of representation.** Learners differ in how they take in information. A learner with Dyslexia may find spoken material easier than printed material. ACESS addresses this through built-in text-to-speech and through learner control over typeface, text size, spacing and background tint, including the Atkinson Hyperlegible typeface (Braille Institute of America, 2021).
- **Multiple means of action and expression.** Learners differ in how they demonstrate understanding. ACESS addresses this through several question formats and through quizzes that are not terminated automatically by a timer, which reduces time pressure during assessment.

2.2.2 Web Content Accessibility Guidelines 2.2

While UDL provides the pedagogical framework, WCAG 2.2 (W3C Web Accessibility Initiative, 2023) provides measurable technical criteria. ACESS targets Level AA, together with four Level AAA criteria that are particularly relevant to reading, attention and sensory differences: 1.4.8 Visual Presentation, 2.2.6 Timeouts, 2.3.3 Animation from Interactions, and 3.2.5 Change on Request. This is stated as a design target and not as a conformance claim, because no independent audit has been carried out against the platform. The accessibility statement published on the platform records this limitation, and Chapter 4 maps each targeted criterion to the ACESS feature that addresses it.

Because the barriers this project addresses are cognitive rather than purely perceptual, WCAG alone is not sufficient. ACESS therefore also draws on the W3C guidance *Making Content Usable for People with Cognitive and Learning Disabilities* (W3C Web Accessibility Initiative, 2021), which sets out design objectives such as breaking content into manageable sections, reducing memory load and stating expectations before a task begins, and on the British Dyslexia Association Style Guide (British Dyslexia Association, 2023), which supplies the typographic and readability

thresholds used by the platform's accessibility compliance checker. The criteria addressed most directly by ACESS are the following.

- **1.4.8 Visual Presentation (AAA).** Line length is limited by a reading width expressed in characters rather than pixels, so that it stays within a comfortable range as the learner changes text size.
- **1.4.12 Text Spacing (AA).** Line height, word spacing and text size are set by the learner and take effect across lesson content immediately, without reloading the page.
- **2.2.1 Timing Adjustable (A) and 2.2.6 Timeouts (AAA).** A quiz time limit set by an educator is displayed to the learner, and the session timeout is announced by a warning thirty seconds before it takes effect.
- **2.3.3 Animation from Interactions (AAA).** A single motion setting reduces or removes transitions and animation across the interface.
- **3.2.5 Change on Request (AAA).** Applying an accessibility preset first shows the learner which settings will change and what each will change to. Text-to-speech never starts on its own.

2.3 Neurodivergent Profiles and Interface Interventions

Understanding the specific difficulties faced by the target learners is necessary before interface decisions can be justified. This section summarises the three profiles the platform is designed around and the response ACESS makes to each.

2.3.1 Attention Deficit Hyperactivity Disorder

ADHD is associated with difficulties in executive function (Zelazo et al., 2003), affecting working memory, task initiation and sustained attention (Barkley, 1997). In a

digital environment, a busy screen forces the eye to move constantly between competing elements, which accelerates fatigue (Munoz et al., 2003).

The response in ACCESS is a distraction-free reading mode that hides the navigation sidebar and notification area while a lesson is open, combined with a limited reading width. The width is expressed in character units rather than pixels, so it remains comfortable as the learner changes text size: 72 characters by default and 66 characters under the ADHD preset. The reading width is retained while the navigation is hidden rather than released by it. The ADHD preset also displays a persistent bar carrying the current task, an auto-save indicator and a task checklist, so that a learner who is interrupted can return to the lesson without having to work out where they were. These features are described in Chapter 4.

2.3.2 Dyslexia

Dyslexia involves difficulty with accurate or fluent word recognition and with spelling and decoding (Rello and Baeza-Yates, 2013). Research indicates that typographic adjustment can improve reading speed and comprehension (Franceschini et al., 2013).

The response in ACCESS is to give the learner direct control over presentation rather than relying on browser zoom. Selecting the Dyslexia preset sets the typeface to Atkinson Hyperlegible (Braille Institute of America, 2021) (OpenDyslexic (Gonzalez, Abelardo, 2024) is also available), raises the base text size to 19 px, sets line spacing to 1.7, sets word spacing to 0.16 em, which is the value required by WCAG 1.4.12, narrows the reading width to 62 characters in line with the 50 to 70 character range recommended by the British Dyslexia Association (British Dyslexia Association, 2023), applies a cream background to reduce glare, and enables both text-to-speech and a reading spotlight that dims the paragraphs the learner is not currently reading (Batanero et al., 2014). The full set of values is given in Chapter 4.

2.3.3 Autism Spectrum Disorder

Learners on the autism spectrum generally work best in structured and predictable environments, and unexpected visual or auditory stimuli can cause significant distress (Lawson, 2011).

ACCESS responds to this from two directions. For the learner, the Autism preset keeps the reading column at the same width in every course so that pages do not change shape between lessons, removes motion, reduces colour saturation, and displays a visual schedule that lists the phases of a lesson before the learner begins, together with step-by-step guidance that always explains why a control is unavailable rather than simply disabling it. Media never plays automatically and text-to-speech never starts on its own. For the educator, the accessibility compliance checker described in Section 4.3.6 examines the draft lesson for the barriers this profile is most sensitive to: figurative language, missing learning objectives, a missing plain-language summary, and structural inconsistency with the other lessons in the same course.

2.4 Comparative Analysis of Existing LMS Platforms

To establish the need for ACCESS, four established platforms were reviewed: Moodle (MoodleHQ, 2023b), Canvas LMS (Instructure, Inc., 2023), Blackboard Learn (Anthology Inc., 2023) and Google Classroom (Google LLC, 2023).

One distinction must be made before the comparison, because it determines whether the comparison is fair. A capability may be present in three different ways: **native**, meaning it is part of the product's own interface and available to every learner; **plugin**, meaning it is available only after an institution's administrator installs and configures an additional component; or **external**, meaning it is not provided by the platform and must be obtained from the operating system, the browser or separate assistive software. The comparison in Table 2.1 is expressed in these terms rather than as a simple yes or no, because a plugin that a learner cannot install for themselves is not equivalent to a setting the learner controls. All four platforms are mature products with

established accessibility programmes; the gap identified here is specifically in learner-controlled cognitive adaptation, not in accessibility in general.

2.4.1 Moodle

Moodle is an open-source learning platform with wide adoption and a modular architecture (MoodleHQ, 2023a). It maintains an accessibility programme and publishes its conformance work (MoodleHQ, 2024b), and its editor includes an accessibility checker that flags problems such as missing image alternative text and low contrast while an educator writes (MoodleHQ, 2024a). Institution-wide auditing is available through the Brickfield Accessibility Toolkit (Brickfield Education Labs, 2024).

The limitation relative to this project is that the accessibility effort is directed at the correctness of authored content and at screen-reader and keyboard operability rather than at cognitive load. A learner cannot select a profile that changes typography, reading width, pacing and task scaffolding together; the nearest equivalents are administrator-selected themes and third-party blocks. Adaptive remediation depends on an educator having written conditional activity rules in advance, and no rule-based engine proposes revision content from assessment results on its own.

2.4.2 Canvas LMS

Canvas presents a card-based interface and publishes accessibility conformance documentation, including a Voluntary Product Accessibility Template (Instructure, Inc., 2024a; Instructure, Inc., 2023). Its editor includes an accessibility checker that an educator can run before publishing (Instructure, Inc., 2024c), and Microsoft Immersive Reader, which provides read-aloud, line focus and adjustable text spacing, can be integrated into Canvas pages, although it must be enabled by the institution and is invoked one page at a time rather than applied as a persistent learner setting (Instructure, Inc., 2024b).

The limitation relative to this project is that the cognitive supports are delivered

through an embedded third-party reader rather than as platform settings, so they do not persist as a learner preference across the product and do not extend to quizzes or interactive activities. Course progression remains linear unless an educator builds mastery paths manually.

2.4.3 Blackboard Learn

Blackboard Learn is an enterprise platform whose accessibility strategy centres on Anthology Ally, which scores uploaded course materials, gives educators remediation guidance, and generates alternative formats such as audio, ePub and tagged PDF for learners to download (Anthology Inc., 2024a). Blackboard also provides a native Achievements tool for badges and certificates (Anthology Inc., 2024b).

The limitation relative to this project is that Ally operates on files rather than on the running interface. It can convert a handout into an audio file, but it does not change the typography, spacing, reading width or motion behaviour of the platform the learner is working in, and the alternative-format model moves the learner out of the course and onto a downloaded document.

2.4.4 Google Classroom

Google Classroom is a lightweight coordination layer over Google Workspace and inherits that ecosystem's accessibility features, including screen-reader support, keyboard navigation, and the read-aloud and dictation tools built into Google Docs and Chrome (Google LLC, 2024; Google LLC, 2023).

The limitation relative to this project is that these supports belong to the surrounding Workspace tools rather than to Classroom. Classroom itself provides no accessibility profiles, no lesson-level engagement data and no recommendation logic; it is a system for distributing and collecting assignments rather than for supporting the learning process.

2.5 Comparison Matrix

Table 2.1 summarises the four platforms against ACESS using the native, plugin and external distinction described above. Each entry states how a capability is delivered rather than claiming that a platform is incapable of it.

Table 2.1: LMS Accessibility Capability Comparison

Capability	Moodle	Canvas	Blackboard	Google Classroom	ACESS
Read-aloud of lesson text	Plugin	Plugin: per page (Instructure, Inc., 2024b)	Plugin: audio alt-format (Anthology Inc., 2024a)	External: Workspace (Google LLC, 2024)	Native , learner setting
Learner-selectable dyslexia typeface and spacing	Plugin / theme (MoodleHQ, 2024b)	Plugin: per page (Instructure, Inc., 2024b)	External	External	Native , persistent
One preset changing typography, layout and pacing together	Not provided	Not provided	Not provided	Not provided	Native : three presets
Distraction-reduced view with bounded line length	Theme-dependent	Not provided	Not provided	Minimal interface by design	Native , width-bounded
Automatic remedial recommendation from assessment	Educator-authored rules	Educator-authored paths	Educator-authored rules	Not provided	Native , rule-based
Authoring-time accessibility checker	Native, editor (MoodleHQ, 2024a)	Native, editor (Instructure, Inc., 2024c)	Native, file-level (Anthology Inc., 2024a)	Not provided	Native , profile-aware
Achievements and badges	Native	Native / add-on	Native (Anthology Inc., 2024b)	Not provided	Native
Lesson-level engagement data	Native	Native	Native	Limited	Native
Reduced-motion control	External: OS	External: OS	External: OS	External: OS	Native setting

continued on the next page

Table 2.1 continued from the previous page

Capability	Moodle	Canvas	Blackboard	Google Classroom	ACCESS
Published WCAG position	Documented (MoodleHQ, 2024b)	VPAT published (Instructure, Inc., 2024a)	Documented (Anthology Inc., 2024a)	Documented (Google LLC, 2024)	Targeted, not audited

Three points should be made about Table 2.1. First, the ACCESS column is not a claim of superiority in accessibility overall. Moodle, Canvas and Blackboard all publish conformance documentation and maintain accessibility teams, whereas ACCESS has not been independently audited, which is why its entry in the final row reads “targeted, not audited”. On the available evidence those three platforms hold a stronger verified accessibility position than ACCESS.

Second, “not provided” means the capability was not found in the vendor’s current documentation for the base product. It does not mean that no third party offers it; the plugin ecosystems are large, and an institution with sufficient resources can close several of these gaps.

Third, the row that carries the argument of this project is the third one. Every platform reviewed can enlarge text, and three of the four can read text aloud. What none of them offers is a single learner-controlled preset that changes typography, reading width, content structure, pacing and task scaffolding together, without administrator involvement. That is the gap ACCESS is designed to fill, and Section 4.3 documents both what the platform does in that space and what it does not yet do.

2.5.1 Alternative Approaches and Justification

Several alternative technical approaches were considered before the final architecture was selected.

- **Monolithic architecture against a decoupled client and server.** A single traditional server-rendered application would have been simpler to deploy. A

decoupled front end (Vercel Inc., 2023a) with a managed backend service (Supabase, 2023) was chosen instead because it allows the accessibility settings to be applied on the client without reloading the page, which is necessary for the immediate feedback described in Chapter 3.

- **Runtime styling libraries against a utility-first framework.** Styling libraries that compute styles at runtime were considered but rejected because of their processing cost on lower-specification devices. Tailwind CSS (Tailwind Labs, 2023) was selected because it produces a small stylesheet and supports instant theme switching through Cascading Style Sheets (CSS) custom properties.
- **Document database against a relational database.** A document database would have offered a more flexible schema. A relational database was chosen because enrolments, progress records and quiz attempts depend on referential integrity, and because the recommendation engine reads across several related entities (Supabase, 2024).
- **Central state management against the framework’s own context mechanism.** A dedicated state management library was considered but judged unnecessary for a single set of accessibility settings. The framework’s built-in context mechanism (Meta Platforms, Inc., 2023) was used instead, which keeps the settings logic small and directly connected to the styling layer.

2.6 Project Methodology

The project was developed using the Agile Scrum methodology (Schwaber, 1997), an iterative framework based on short development cycles, regular review and adaptive planning. Scrum was preferred to a sequential model such as Waterfall because designing for cognitive accessibility is an empirical activity: assumptions about whether a particular interface change actually helps must be checked by using the interface, and cannot safely be deferred to the end of development.

Development was organised into six sprints of two weeks each.

- **Sprint 0: requirements and setup.** Definition of the product backlog from WCAG 2.2 and UDL, creation of the project structure, and initial database schema design.
- **Sprints 1 and 2: core architecture.** User authentication, database schema and security policies, and role-based route protection.
- **Sprints 3 and 4: feature development.** The educator authoring interface (lesson editor, course creation, quiz builder) and the learner experience (enrolment, lesson viewing, progress tracking).
- **Sprints 5 and 6: adaptive features and refinement.** The rule-based recommendation engine, the accessibility personalisation engine, analytics dashboards, certificate generation and notifications.

Each sprint ended with a review and a retrospective. The activities carried out in each sprint are reflected in the analysis and design work reported in Chapters 3 and 4.

2.7 Project Requirements

2.7.1 Software Requirements

The following software was used to develop and deploy the ACCESS platform.

- **Visual Studio Code.** Primary code editor.
- **Node.js.** Runtime for the development server and the build toolchain.
- **Git and GitHub.** Source control, with the repository connected to the deployment platform.
- **Supabase CLI (Supabase Inc., 2025b).** Runs the complete backend stack locally, applies database changes and rebuilds the database from scratch for development and verification.

- **Docker Desktop (Docker, Inc., 2025).** Container runtime hosting the local backend stack, so that the database, authentication and storage layers can be rebuilt on the development machine without affecting the hosted project.
- **Vercel (Vercel Inc., 2023b).** Hosting platform for the deployed application, with automatic deployment from the GitHub repository.
- **Next.js (Vercel Inc., 2023a).** React framework providing server-side rendering, routing and request-level route protection (Vercel Inc., 2024).
- **TypeScript (Microsoft, 2023).** Used throughout the codebase for compile-time type checking.
- **Tailwind CSS (Tailwind Labs, 2023).** Styling framework, used to apply the accessibility settings through CSS custom properties.
- **Supabase and PostgreSQL (Supabase, 2023; PostgreSQL Global Development Group, 2025).** Managed relational database with row-level security, token-based authentication (Supabase Inc., 2025a), object storage and an automatically generated data interface (PostgREST Contributors, 2025).
- **Framer Motion (Framer, 2024).** Animation library, used for interface transitions that respect the learner's motion setting.
- **Tiptap (Brainhub, 2024).** Rich-text editor used in the educator authoring workflow.
- **Recharts (Recharts Team, 2024).** Charting library used for the analytics dashboards.
- **jsPDF (Parallax Agency Ltd., 2025).** PDF generation library used for completion certificates and administrator reports, paired with a QR-code generator that encodes each certificate's public verification address.
- **Google Gemini API (Google DeepMind, 2025).** Generative model service used for two optional and clearly labelled learner features: a plain-language lesson summary and a lesson-scoped question-answering assistant. No assessment is graded by a model.

- **Web Speech API (Mozilla Developer Network, 2025).** The browser's own speech synthesis interface, used for the read-aloud control. Using the browser rather than a network service keeps lesson text on the learner's device and avoids a per-request cost.
- **DOMPurify (Heiderich, Mario and Cure53, 2025).** Sanitiser applied to authored lesson content before it is displayed, which prevents harmful markup from being stored and rendered.
- **dnd kit (Bernier, Claudéric, 2025).** Drag-and-drop toolkit used by the interactive activities, and the component that makes drag-and-drop activities operable from the keyboard.
- **Nodemailer and Resend.** Mail services used for password recovery and notification messages.
- **Figma (Figma, 2023).** Design tool used to prototype interface layouts before implementation.

2.7.2 Hardware Requirements

- **Development workstation.** A personal computer with at least 8 GB of memory, a multi-core processor and a solid-state drive, sufficient to run the development server and the local database stack together.
- **Display.** A monitor with a resolution of at least 1920 by 1080 pixels, needed to check responsive layouts and the effect of each accessibility preset.
- **Network connectivity.** A stable broadband connection for access to the hosted database, the deployment platform and the source repository.
- **Server infrastructure.** Cloud infrastructure provided by the hosting and database services, which removes the need for dedicated server hardware.

2.7.3 Other Requirements

- **Hosted database account.** A cloud-hosted project for the production database, authentication and file storage. A separate local stack is used for schema development and verification, so that operations such as a complete database rebuild are never carried out against the hosted project.
- **Generative model API key.** Required only for the optional lesson summary and lesson assistant features. The platform continues to work without it.
- **GitHub account.** Repository hosting and the source of automated deployment.
- **Deployment platform account.** Linked to the repository for automated builds and hosting.
- **Mail service.** Used for password reset and account verification messages.
- **Web browser.** A current version of Chrome, Firefox or Edge for development and testing.

2.8 Project Schedule and Milestones

The project was carried out over six sprints of two weeks each. Table 2.2 lists the focus and deliverables of each sprint, and Figure 2.1 presents the overall schedule across fourteen weeks.

Table 2.2: Project Schedule and Milestones

Sprint	Focus Area	Key Deliverables
0	Requirements and setup	Product backlog, project structure, database schema design
1	Core authentication	Registration and login, role-based route protection, email verification
2	Database and security	Row-level security policies, storage configuration, data dictionary
3	Educator authoring	Course creation, lesson editor, quiz builder, media upload

continued on the next page

Table 2.2 continued from the previous page

Sprint	Focus Area	Key Deliverables
4	Learner experience	Enrolment, lesson viewer, progress tracking, distraction-free mode
5	Adaptive features and analytics	Recommendation engine, accessibility presets, analytics dashboards, certificates, notifications

Phase	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Requirements Gathering														
System Design														
Core Architecture														
Feature Development														
Advanced Features														
Testing & Refinement														
Documentation & Deployment														

Figure 2.1: Gantt Chart of the ACCESS Development Schedule

2.9 Summary

This chapter has shown that existing learning management systems provide capable administrative and content-hosting features but do not address the cognitive requirements of neurodivergent learners. Where cognitive supports exist, they are typically delivered through plugins, embedded third-party readers or downloadable alternative formats rather than as settings the learner controls, and none of the four platforms reviewed offers a single preset that changes typography, layout and pacing together. The frameworks reviewed here, UDL and WCAG 2.2 supplemented by cognitive accessibility guidance and dyslexia-specific typographic recommendations, provide the basis for the design that follows. The methodology, requirements and schedule set out in this chapter define how that design was carried out. The next chapter analyses the problem in more detail and states the data, functional and non-functional requirements of the system.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Introduction

This chapter translates the findings of the literature review into software engineering specifications. It begins with an analysis of the problems observed in existing educational platforms, then states the data, functional, non-functional and other requirements of the system, and finally presents the system architecture and the division of the platform into modules.

3.2 Problem Analysis

Chapter 2 established the general limitations of Moodle, Canvas, Blackboard Learn and Google Classroom. This section examines the operational problems that follow from those limitations and that directly shaped the design of ACESS.

3.2.1 Current System Scenario

Existing learning management systems present the same interface to every learner regardless of cognitive profile. A neurodivergent learner who needs a different presentation must obtain it elsewhere, through a browser extension for read-aloud, a separate tool for changing typography, or by manually closing interface elements that are in the way. The responsibility for accessibility therefore rests with the learner rather than with the system.

The flow of activity in these platforms is also linear: a learner enrolls, receives content, is assessed, and completes the course. There is no feedback path by which a poor assessment result changes what the learner is offered next. If a learner performs badly on a quiz, the next topic is presented as though nothing had happened.

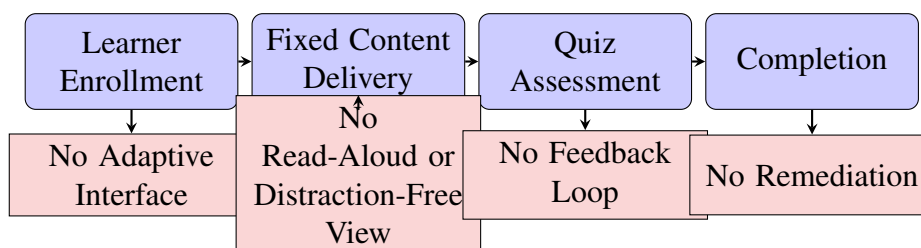


Figure 3.1: Current System Problem Analysis Diagram

3.2.2 Detailed Problem Breakdown

Figure 3.1 shows this linear flow together with the point at which each barrier arises. The problems stated in Chapter 1 are analysed below in more detail.

1. **Fixed typography and layout.** Standard platforms use a fixed presentation that does not respond to individual needs. Learners with Dyslexia benefit from wider letter spacing, line spacing of roughly 1.5 to 1.8, and a typeface designed to reduce confusion between similar letter shapes such as ‘b’ and ‘d’ (Braille Institute of America, 2021; Batanero et al., 2014). Without these adjustments, reading is slower and fatigue accumulates, which contributes to lessons being abandoned.
2. **Cognitive overload from navigation.** Established platforms present deeply nested menus together with dashboards showing calendars, announcements, grades and messages at the same time. For learners with ADHD (Barkley, 1997) this competition for attention makes it difficult to stay with a single learning objective, and without a distraction-free mode the learner must remove the surrounding elements manually.
3. **No adaptive remediation.** Existing assessment features record a score but do not act on it. When a learner fails a foundational topic, no revision content is

recommended and no adjustment is made to the learning path. This assumes that all learners are equally ready to progress, which is particularly unsuitable for learners who need repeated exposure to a topic.

3.3 Requirement Analysis

3.3.1 Data Requirement

ACCESS requires a relational data model covering users, courses, lessons, assessments, progress and accessibility preferences. The categories of data the system must store and process are as follows.

- **User data.** Account credentials, profile information such as name, avatar and role, and notification preferences. Access to user data is restricted by role.
- **Accessibility data.** Each learner's accessibility settings: the active preset, typeface, text size, line and word spacing, background tint, theme, layout and structure mode, motion level, read-aloud and caption settings, and the executive-function supports such as the task checklist, visual schedule, step-by-step guidance, progress timeline and auto-save indicator. These settings must persist between sessions and apply throughout the platform. They are held as a single structured document attached to the learner's profile rather than as separate columns, because the list of settings changes far more often than the rest of the schema and because every part of the system reads the settings as a whole. The trade-off this represents is discussed in Section 4.4.
- **Curriculum data.** The course, chapter and lesson hierarchy, with lesson content, video references, transcripts, and metadata such as difficulty level, estimated duration and prerequisite relationships.
- **Assessment data.** Quiz configuration such as time limit, pass threshold and maximum attempts; questions and answer options with the correct option marked; and each learner's attempts, answers and scores.

- **Progress data.** Whether a lesson has been viewed and completed, the time spent on it, the state of each enrolment, and the history of quiz attempts and their outcomes.
- **Content and achievement data.** Interactive activity definitions, certificate records with a unique reference code, and achievement definitions with each learner's progress towards them.
- **Notification and communication data.** Notifications raised on enrolment, quiz attempts, new lessons and course publication, together with lesson discussion threads and contact form submissions.

The complete database schema and data dictionary are presented in Section 4.4.

3.3.2 Functional Requirement

Functional requirements define what the system must do. They are grouped below by user role. Figure 3.2 shows the interaction between the three actors and the functions of the system, and Tables 3.1 to 3.3 state each requirement together with its inputs and outputs.

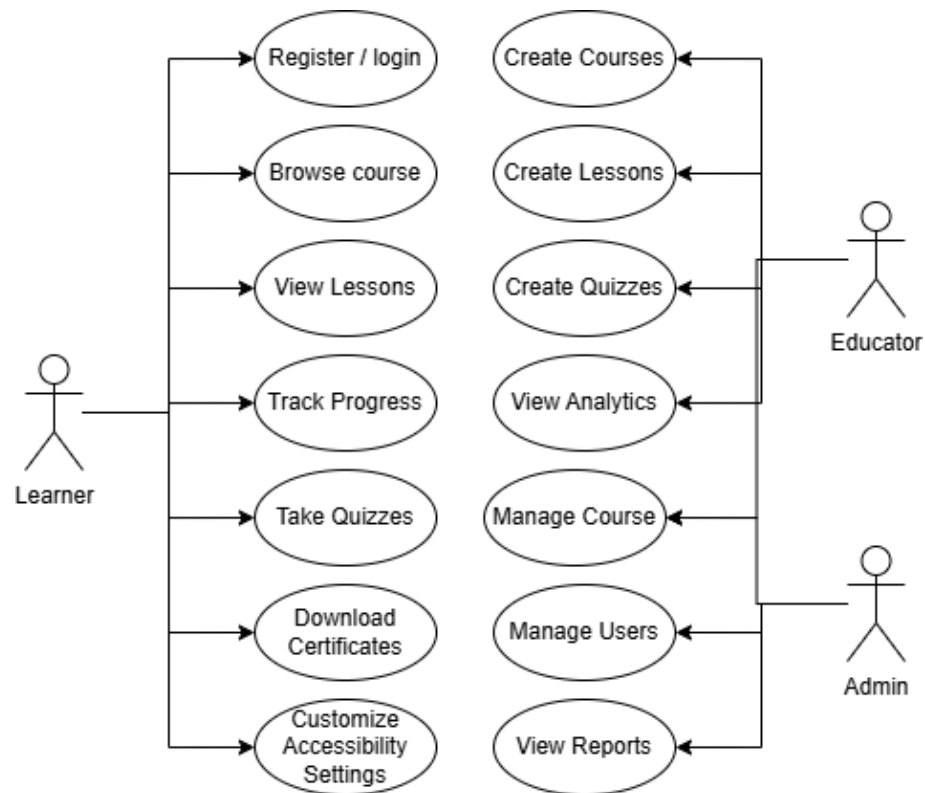


Figure 3.2: Use Case Diagram for the ACCESS Platform

Learner functions

Table 3.1: Functional Requirements of the Learner Module

Req ID	Description	Actor	Inputs	Outputs
FR-STU-01	Registration and secure login.	Learner	Email, password	An authenticated session, and a new account and profile created automatically with the role fixed to learner.
FR-STU-02	Selection of an accessibility preset or an individual setting.	Learner	Preset choice or setting value	A preview of the changes, followed by the saved settings and an immediate change to the interface without reloading the page.
FR-STU-03	Course discovery and enrolment.	Learner	Course selection	A new enrolment record, and a notification to the educator who owns the course.

continued on the next page

Table 3.1 continued from the previous page

Req ID	Description	Actor	Inputs	Outputs
FR-STU-04	Lesson viewing with adaptive presentation.	Learner	Navigation to a lesson	The lesson rendered at the learner's reading width, layout and typography, with navigation hidden if distraction-free mode is active.
FR-STU-05	In-video questions.	Learner	Video playback reaching a stored timestamp	Playback is paused and the question is displayed over the video.
FR-STU-06	Quiz attempt.	Learner	Selected answer options	The attempt is graded on the server and recorded. The correct answers are not sent to the browser before submission.
FR-STU-07	Progress recording.	Learner	Time spent on the lesson page	The time spent, view count and viewed and completed status are updated.
FR-STU-08	Achievement earning.	Learner	Course progress reaching a threshold	The system derives completion and records the earned achievements. A learner cannot award an achievement directly.
FR-STU-09	Certificate claim and download.	Learner	Course completion	Eligibility is checked, a reference code and public verification address are issued, and the certificate is produced as a PDF.
FR-STU-10	Read-aloud playback.	Learner	Selection of the Listen control	The lesson text is spoken at the learner's chosen rate. Playback never starts automatically.
FR-STU-11	Adaptive recommendations.	Learner	Quiz results, viewing history, saved courses	Revision, next-lesson, near-completion and cross-course recommendations, each with the reason it was given.
FR-STU-12	Saved courses and lesson discussion.	Learner	Save action, comment text	The course is added to the learner's saved list, and the comment is added to the lesson discussion thread.
FR-STU-13	Lesson assistant.	Learner	A question about the lesson currently open	An answer limited to the content of that lesson, together with a stored plain-language summary of the lesson.

continued on the next page

Table 3.1 continued from the previous page

Req ID	Description	Actor	Inputs	Outputs
FR-STU-14	Interface language switch.	Learner	English or Bahasa Melayu	The interface text changes immediately and the choice is saved to the learner's profile.

Educator functions Requirement FR-EDU-05 has no equivalent in the platforms surveyed in Chapter 2, and its design is described in Section 4.3.6.

Table 3.2: Functional Requirements of the Educator Module

Req ID	Description	Actor	Inputs	Outputs
FR-EDU-01	Course creation and editing.	Educator	Title, metadata, difficulty, accessibility focus	A new or updated course, restricted to courses the educator owns.
FR-EDU-02	Lesson authoring.	Educator	Rich text, headings, images, tables, links	Sanitised lesson content saved to the lesson, with a snapshot added to the version history.
FR-EDU-03	Quiz and activity creation.	Educator	Question text, options, correct option, activity settings	Stored questions and options, and stored interactive activity definitions.
FR-EDU-04	Asset management.	Educator	Image, document and media files	Files uploaded to course storage and recorded against the course. Instructional video is embedded from an external host rather than uploaded.
FR-EDU-05	Accessibility compliance check.	Educator	The lesson draft currently open and the course accessibility focus	A score out of 100, a list of the standards met and unmet, the source of each rule, the editor tab that corrects it, and a one-click fix where one is available. The check is re-run as the lesson is written.
FR-EDU-06	Learner monitoring and analytics.	Educator	Course or learner selection	Charts of progress and quiz performance, with flagging of learners at risk.
FR-EDU-07	Achievement and milestone configuration.	Educator	Requirement type and threshold	Stored achievement and milestone definitions for the course.

continued on the next page

Table 3.2 continued from the previous page

Req ID	Description	Actor	Inputs	Outputs
FR-EDU-08	Certificate issue and revocation.	Educator	Learner selection, revocation reason	A new or updated certificate record. A revoked certificate keeps its reference code so that verification reports the revocation rather than reporting the certificate as unknown.
FR-EDU-09	Preview as a learner.	Educator	Course or lesson selection	The learner view is rendered without creating an enrolment or a progress record.
FR-EDU-10	Lesson version history.	Educator	Version selection	A previous version of the lesson content is restored.

Administrator functions

Table 3.3: Functional Requirements of the Administrator Module

Req ID	Description	Actor	Inputs	Outputs
FR-ADM-01	User role management.	Administrator	Target user, new role	The role is updated in both the user record and the authentication account, so that routing and database policies agree. A database guard prevents the same change being made from an ordinary session.
FR-ADM-02	Course moderation.	Administrator	Approve or reject	The course status moves from pending review to published, or back to draft, and the educator is notified.
FR-ADM-03	Educator application review.	Administrator	Decision and reviewer notes	The application is updated, and an approved applicant is promoted to the educator role in both the user record and the authentication account.
FR-ADM-04	Official course creation.	Administrator	Course metadata	A course marked as administered by the platform, which the learner catalogue presents as an official course.

continued on the next page

Table 3.3 continued from the previous page

Req ID	Description	Actor	Inputs	Outputs
FR-ADM-05	Platform analytics.	Adminis- trator	Date range	Aggregated registrations, enrolments, completion, learning time, and figures on accessibility preset adoption and feature use.
FR-ADM-06	Report generation.	Adminis- trator	Report type and date range	A PDF report covering platform, course, learner or content coverage.
FR-ADM-07	Certificate oversight.	Adminis- trator	Search or filter	Read access to all issued certificates, including revocation status.
FR-ADM-08	Enquiry handling.	Adminis- trator	Message status change	The status of a contact form message is updated between unread, read and replied.

3.3.3 Non-Functional Requirement

Non-functional requirements state the quality attributes the platform must satisfy.

Performance

- **Page response time.** Movement between pages within the application should complete within 300 ms, so that navigation does not itself interrupt concentration.
- **Course loading.** Course data should be retrieved within 800 ms on a broadband connection.
- **Accessibility setting response.** A change to any accessibility setting must be visible without reloading the page. This is the strictest performance requirement in the system, because a learner adjusting spacing or text size is judging the result as it changes and cannot do so if the change is delayed.
- **Video loading.** Instructional video is embedded from an external host rather than served by the platform, so playback start depends on that host and no target is stated for it here.

- **Concurrency.** The database must sustain a realistic worst-case burst of simultaneous activity for a deployment of this size without a sharp increase in response time. Chapter 6 reports load tests against this requirement.

Security

- **Authentication.** All sessions are secured by signed tokens held in cookies that the browser's scripts cannot read (Supabase Inc., 2025a), with a fifteen-minute inactivity timeout announced thirty seconds before it takes effect.
- **Authorisation.** Every non-public page is checked against the role held in the session before the page is rendered. This layer controls which interface a user reaches and is deliberately not treated as the security boundary; the authoritative check is repeated inside the database by row-level security policies and inside privileged server routes. Section 4.5.2 describes this two-layer arrangement.
- **Database protection.** Row-level security must be enabled on every table in the application schema, so that data isolation holds at the database itself regardless of which client issues a query. The platform currently has all 36 tables protected by 88 policies, confirmed by an executable check rather than by inspection.
- **Privilege guards.** Three database guards prevent a learner from forging an outcome even with a valid session: one rejects any attempt by a client to change a privileged account field, one rejects any attempt to mark a course complete outside the normal completion process, and one rejects any attempt to write a quiz score directly instead of through the grading function.
- **Input handling.** Authored lesson content is sanitised before it is displayed (Heiderich, Mario and Cure53, 2025), which is the platform's principal defence against harmful scripts stored in lesson content. All database access is parameterised, so query text is never assembled from user input. Field validation is performed in the interface and enforced again by constraints in the database.

Accessibility

- **WCAG target.** The platform targets WCAG 2.2 Level AA (W3C Web Accessibility Initiative, 2023) together with four Level AAA criteria (1.4.8, 2.2.6, 2.3.3 and 3.2.5). Body text contrast must meet or exceed 4.5:1. This is a design target rather than an audited result, as discussed in Section 4.3.
- **Dyslexia support.** The platform must provide the Atkinson Hyperlegible (Braille Institute of America, 2021) and OpenDyslexic (Gonzalez, Abelardo, 2024) typefaces, line spacing adjustable between 1.0 and 3.0, and word spacing reaching at least the 0.16 em value required by WCAG 1.4.12.
- **Reading width.** Lesson text must be limited by a width expressed in character units, so that line length stays within a comfortable range as text size changes: 72 characters by default, 62 under the Dyslexia preset and 66 under the ADHD preset. The limit must be retained when navigation is hidden.
- **ADHD and Autism support.** A single motion setting must reduce or remove animation across the interface. No media may play automatically, and read-aloud must never begin without an explicit action by the learner.
- **Screen reader compatibility.** Interface components must expose the correct accessible names and states, navigation must be built from real links rather than click handlers, and every interactive activity must be operable from the keyboard. It should be noted that no assistive technology testing has been carried out against this build, so this requirement is currently satisfied by construction and code review only.

Usability

- **Consistent navigation.** The navigation structure must remain the same across all pages of a role, so that the interface stays predictable.
- **Error prevention.** Destructive actions, such as an educator deleting a course, must require an explicit confirmation step.

Reliability

- **Data integrity.** The schema must enforce referential integrity so that records cannot be left orphaned, for example a progress record without a valid enrolment.
- **Backup and reproducibility.** The hosted database relies on the provider's managed backup. Independently of that, the whole schema must be reconstructible from version-controlled definitions and the demonstration dataset from a repeatable script, so that a complete working database can be rebuilt from the project repository alone.

Maintainability and scalability

- **Modular architecture.** The codebase must separate data retrieval from interactive presentation, so that each part can be changed independently.
- **Normalisation.** The schema must follow Third Normal Form, other than the deliberate exceptions documented in Section 4.4, so that data is not duplicated as the number of courses and learners grows.

3.3.4 Other Requirements

The software and hardware listed in Section 2.7 were selected for the following reasons.

- **Next.js and React (Vercel Inc., 2023a).** Rendering pages on the server reduces the amount of code the browser must process before a page becomes usable, which supports the page-load requirement above. The framework also provides a single place to check a user's role before a protected page is rendered (Vercel Inc., 2024).
- **Supabase and PostgreSQL (Supabase, 2023).** Row-level security (Supabase, 2024) enforces data isolation inside the database rather than only in application

code, so progress and assessment data remain separated per learner regardless of how a query reaches the database.

- **Tailwind CSS (Tailwind Labs, 2023).** The framework applies the accessibility settings through CSS custom properties, which allows typography and colour to change immediately without re-rendering the page.
- **TypeScript (Microsoft, 2023).** Type checking across the codebase reduces the risk of errors in critical paths such as quiz scoring and the application of accessibility settings.
- **Vercel (Vercel Inc., 2023b).** Deployment close to the user keeps page transitions fast, which supports the performance requirements stated above.

3.4 System Architecture

ACCESS uses a decoupled client and server architecture in which the browser communicates with a managed database service through a generated data interface, with a small number of server routes reserved for privileged operations.

3.4.1 Context Diagram

The context diagram gives the highest-level view of the system, showing its boundary and the external parties and services it interacts with.

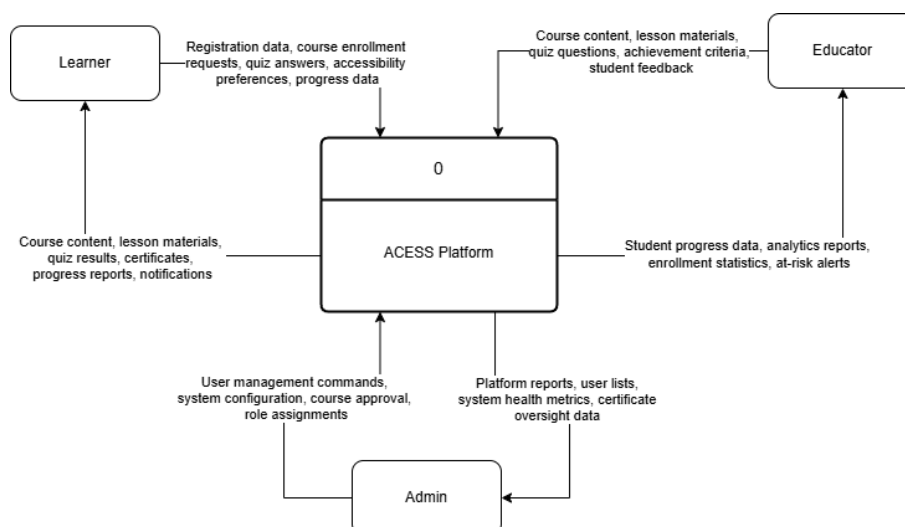


Figure 3.3: Context Diagram

As shown in Figure 3.3, ACCESS is the central processing system. The three human actors are the learner, who consumes content and generates progress data; the educator, who authors content and configures assessments; and the administrator, who oversees the platform. Four external services sit on the system boundary: a mail provider for password recovery and notification messages; an external video host from which instructional video is embedded rather than uploaded; a generative model service used only for the optional lesson summary and lesson assistant; and the browser’s own speech synthesis engine (Mozilla Developer Network, 2025), which performs read-aloud on the learner’s own device. The last of these is a deliberate choice: it avoids a per-request cost and means that lesson text does not have to leave the device in order to be read aloud.

3.4.2 System Architecture Diagram

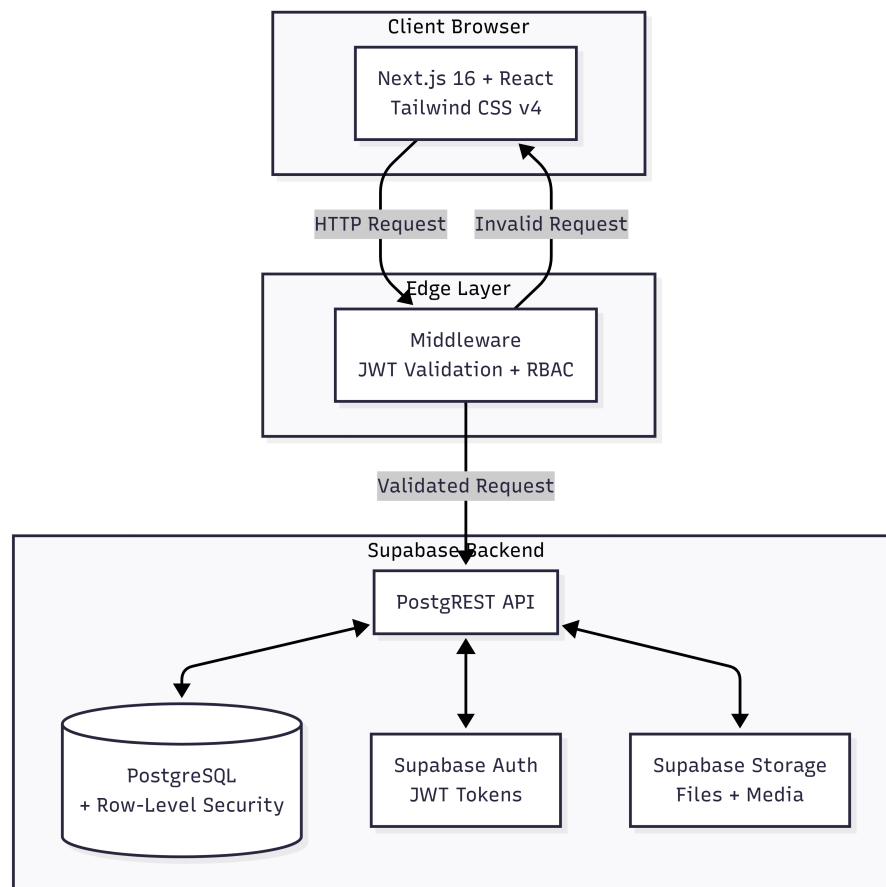


Figure 3.4: System Architecture Diagram

Figure 3.4 shows the three tiers of the architecture.

- **Presentation tier.** The interface is built with the Next.js App Router. Data retrieval is performed on the server so that less code has to be sent to the browser, and interactive behaviour such as the lesson editor and the quiz interface runs on the client. Styling is applied through CSS custom properties, which is what allows the accessibility settings to take effect immediately.
- **Route protection layer.** A request check runs before every non-public page. It validates the session, refuses suspended or deleted accounts, and admits the learner, educator or administrator area according to the role held in the session. It reads the role from the session rather than from the database in order to avoid a database query on every request. The authoritative role check is repeated in the database and

in privileged server routes, so this layer decides which interface a user sees rather than which data they may reach.

- **Data tier.** A managed PostgreSQL database is exposed over a generated data interface (PostgREST Contributors, 2025). Security is written into the schema itself: 88 row-level security policies across all 36 tables, three guards that prevent privilege and outcome forgery, and two operations that must not be decided by the client, quiz grading and course completion, implemented as trusted database functions rather than as browser logic.
- **Storage tier.** Three storage areas hold uploaded files: course assets such as lesson media and thumbnails, educator-supplied certificate designs, and profile pictures. All three are readable by address because the application renders those addresses directly in the page, while write access is restricted by policy, and a learner may write only within a folder belonging to their own account. Instructional video is not stored here, and completion certificates are produced in the browser at the moment of download rather than stored as files.

3.4.3 Deployment Diagram

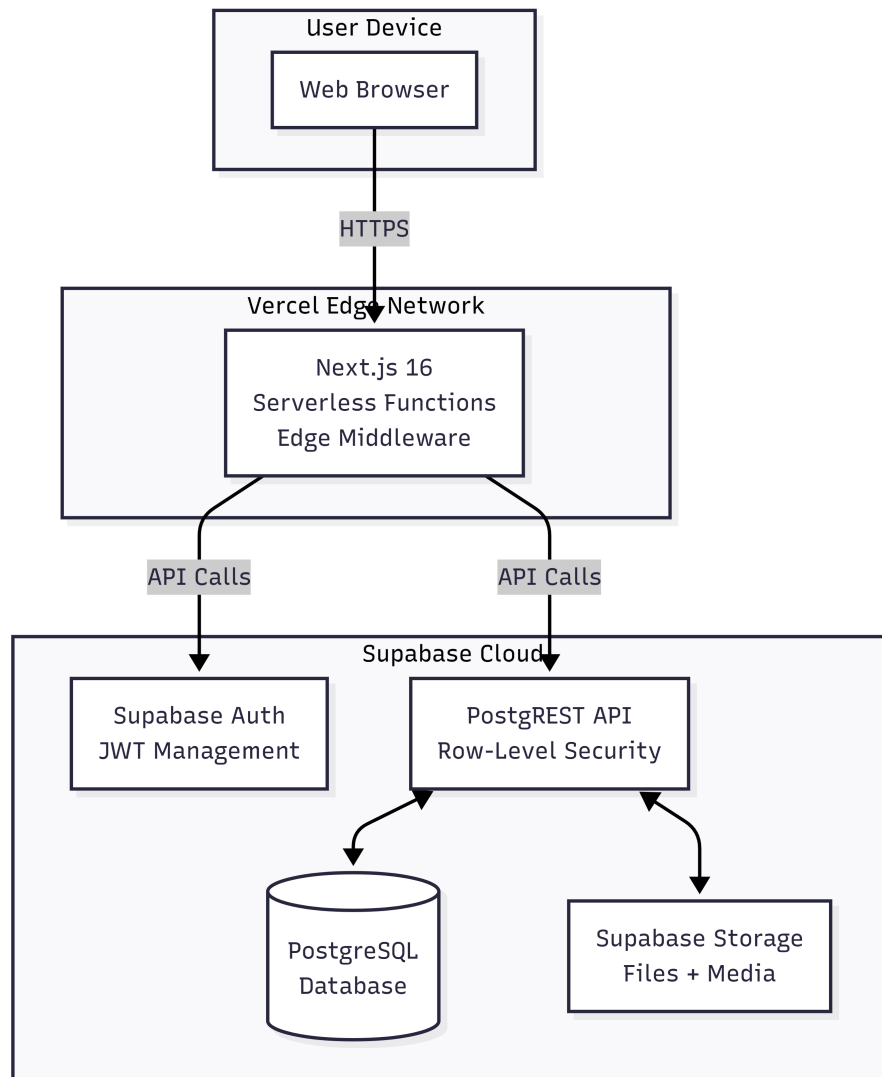


Figure 3.5: Deployment Diagram

Figure 3.5 shows how the components are distributed. The application is deployed to a serverless hosting platform (Vercel Inc., 2023b), which rebuilds it from the source repository on each release. The backend runs on managed cloud infrastructure and exposes the schema through a generated data interface (PostgREST Contributors, 2025) used both by the browser, under row-level security, and by the server routes. Four operations, namely account provisioning, role changes, recommendation generation and certificate issue, require privileges the learner does not have and are therefore implemented as server routes rather than as calls from the browser.

A second, complete copy of the same backend runs locally in containers for development, comprising the database, the authentication service, the data interface, storage and the administration console. Because the schema is defined entirely by version-controlled definitions, this local environment can be rebuilt from empty on demand, which is what makes the database evidence presented in Section 4.5.2 reproducible rather than a description of a single mutable instance.

3.5 Module Design

The system is divided into the following modules.

- **Learner module.** Course discovery, enrolment, lesson viewing, quiz attempts and progress.
- **Educator module.** Course and lesson authoring, the lesson editor, quiz and activity creation, and media upload.
- **Administrator module.** User and role management, course moderation, educator application review, platform analytics and reporting.
- **Accessibility module.** Loads the learner's stored settings, resolves any conflicts between them, and applies the result to the whole interface, from which typography, colour, reading width and motion behaviour are derived.
- **Accessibility compliance module.** A checking engine shared by two views: the lesson editor, which re-runs it on the draft as it is written, and the course-level report, which runs the same rules against saved lessons. Because the engine has no side effects and produces the same result for the same input, the two views always agree.
- **Interactive activity module.** Five activity types together with in-video questions and embedded H5P content. Quiz assessment is handled separately and is graded in the database rather than in this module.

- **Analytics module.** Aggregates lesson progress, quiz attempts and accessibility usage into charts, giving educators a course-level view and administrators a platform-level view, including which accessibility features are actually being used.

3.6 Summary

This chapter has analysed the specifications required to build the ACCESS platform. It examined the problems in existing educational systems in detail, stated the data, functional, non-functional and other requirements, presented the system architecture through context, architecture and deployment diagrams, and described the division of the platform into modules. The next chapter translates these requirements into a system design, covering the behaviour of each role, the user interface, the database model, the accessibility design and the physical implementation of the database.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

This chapter translates the requirements established in Chapter 3 into a system design. It follows the order set out in the writing guide. The high-level design is presented first, covering the behaviour of each role, the interactions between the system components, the user interface, and the conceptual and logical database model. The accessibility design, which is the substantive contribution of this project, follows. The data dictionary is then presented, and the chapter closes with the detailed design of the software components and the physical database.

Two features of this chapter should be noted. First, the figures are evidence rather than illustration: the interface figures are screenshots of the running system taken against a database that can be rebuilt from the project repository, and the entity relationship diagrams are generated from that database's own catalogue rather than drawn by hand. Second, Section 4.3 states both what the accessibility implementation does and what it does not do, because an accessibility design that reports only its successes would be of limited value to a reader trying to judge it.

4.2 High-Level Design

4.2.1 Behavioural Design

Activity diagrams model the workflow of each role.

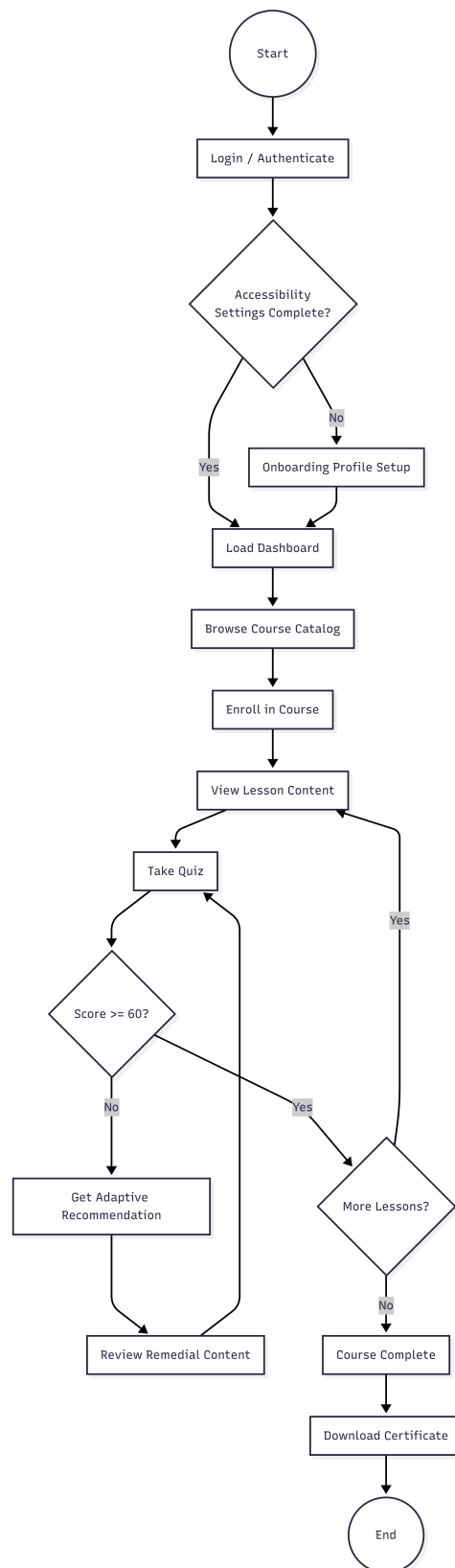


Figure 4.1: Learner Activity Diagram

Learner workflow Figure 4.1 shows the learner workflow. After the learner signs in, the accessibility module loads the stored settings and applies them before the first page is drawn, so that the learner does not briefly see an unstyled page followed by a correction. A learner with no stored settings is offered a short onboarding step but is not prevented from continuing without it, because an accessibility profile is an aid rather than a requirement.

The main loop consists of browsing the catalogue, enrolling in a course and working through its lessons in order, with progress updated as each lesson is opened and completed. The branch of interest occurs after a quiz. The attempt is graded by the database rather than by the browser. If the score falls below the pass threshold, the recommendation engine records a revision recommendation naming the lesson and the score that produced it. The recommendation is shown prominently but does not lock the learner out of the next lesson, because forcing remediation would turn a support into a penalty for a learner who is already struggling.

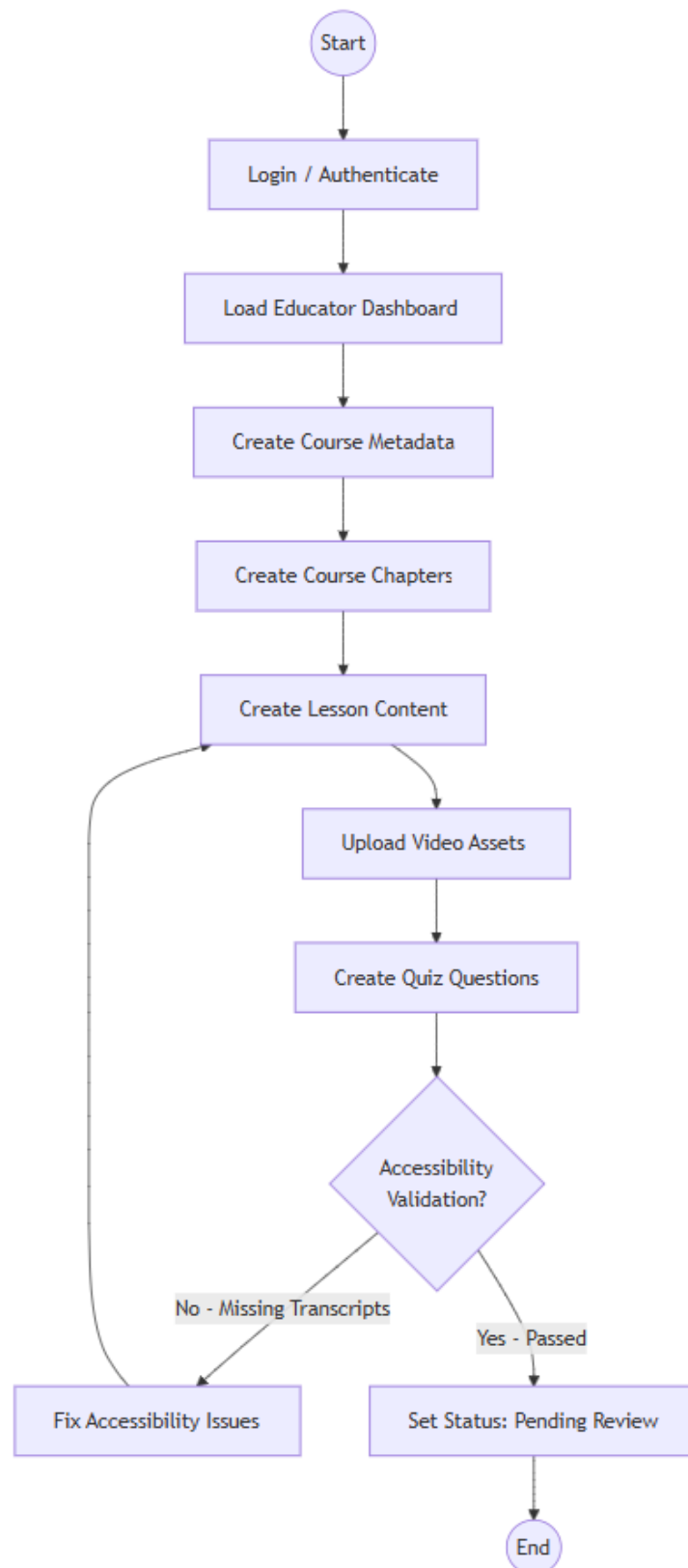


Figure 4.2: Educator Activity Diagram

Educator workflow Figure 4.2 shows the educator’s authoring workflow. It begins with course metadata and, optionally, the chapter structure, and includes one decision that affects everything after it: the primary accessibility focus of the course. That choice determines which rule set the accessibility compliance checker applies to every lesson in the course, so that an educator writing for learners with Dyslexia is checked on sentence length and readability while one writing for autistic learners is checked on figurative language and stated learning objectives.

The educator then enters a content loop of drafting lesson text, embedding media, building activities and writing quiz questions. The compliance checker re-evaluates the draft as it is written, so the accessibility score shown in the editor changes while the lesson is being composed rather than appearing only at the end. When the educator submits the course, its status moves to pending review for administrator moderation. The accessibility check is not a gate in that path: a lesson with unmet standards can still be saved and published, and the panel states this explicitly. The reasoning is given in Section 4.3.6.

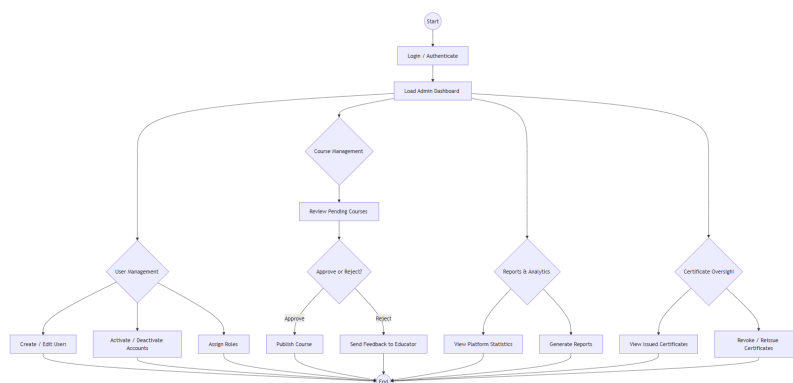


Figure 4.3: Administrator Activity Diagram

Administrator workflow The administrator workflow in Figure 4.3 is mainly reactive. The administrator monitors two queues: courses awaiting review and applications from users wishing to become educators. When a course enters pending review, the administrator evaluates it, including its course-level accessibility report, and approval publishes the course and notifies the educator. Approving an educator application promotes the applicant, updating both the user record and the authentication account so that the two remain consistent. The remaining administrator activities are oversight

rather than moderation: reviewing platform analytics, generating reports, managing user status and handling enquiries submitted through the contact form.

4.2.2 Interaction Design

Sequence diagrams show how the components interact over time for four important use cases. One convention runs through all four: an operation drawn as reaching the database directly is one the browser issues under the learner's own session and therefore under row-level security, while an operation drawn as passing through a server route is one that requires privileges the learner does not have. Which of the two is used is a security decision rather than an implementation detail, so the diagrams distinguish them.

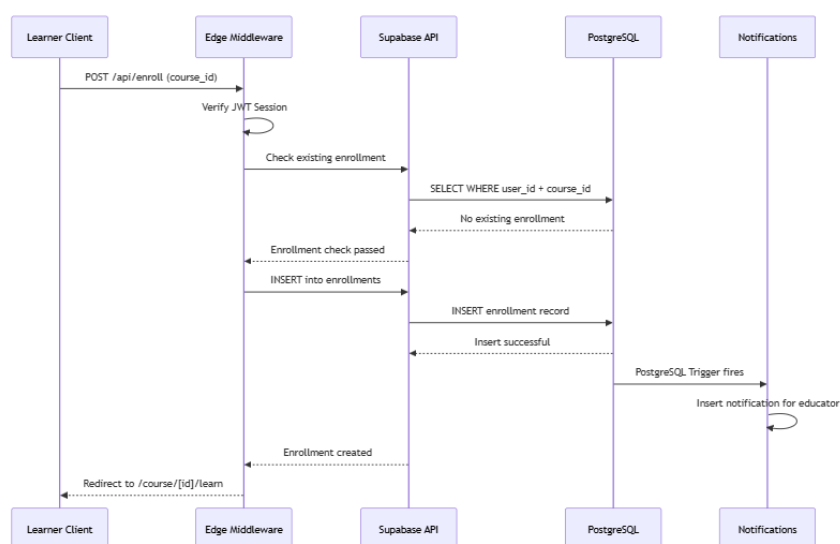


Figure 4.4: Course Enrolment Sequence Diagram

Course enrolment Figure 4.4 shows the enrolment transaction and illustrates how much of the correctness in this architecture is carried by the database rather than by application code.

1. The learner selects Enrol on a course page. The request is issued directly to the database under the learner's own session and is therefore evaluated by the database's own access policies.

2. The insert policy on the enrolment table admits the record only if the learner identified on it is the caller, so a learner cannot enrol anybody else. No application check is required, and none can be bypassed.
3. A uniqueness rule on the learner and course together prevents a duplicate enrolment. Enrolling again reactivates the existing record instead, which preserves the learner's earlier progress.
4. A database trigger creates a notification addressed to the educator who owns the course.
5. The browser returns to the course page, which now shows the lesson roadmap rather than the enrolment prompt.

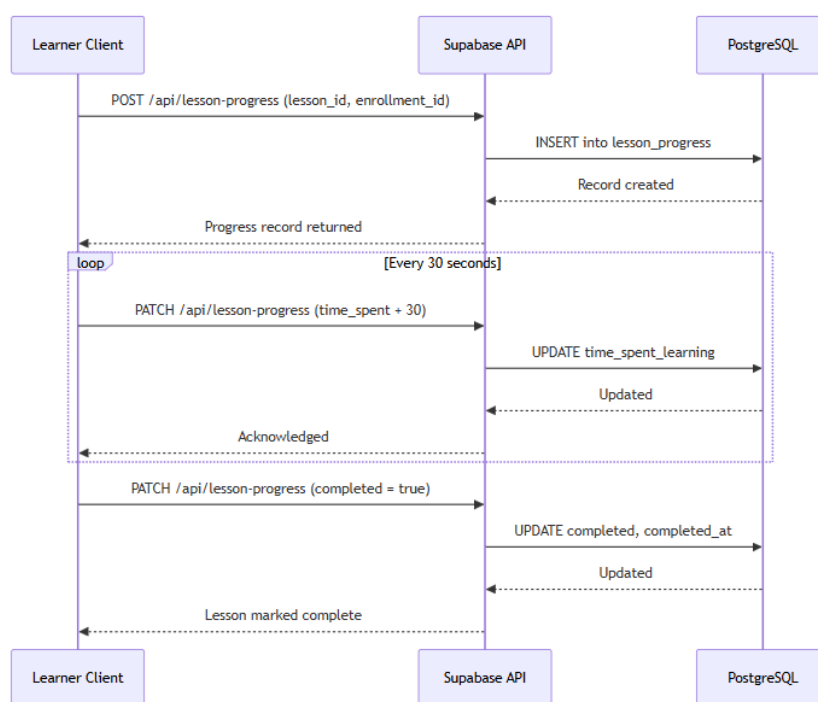


Figure 4.5: Lesson Learning Sequence Diagram

Lesson viewing and progress Figure 4.5 shows how lesson progress is recorded. Opening a lesson creates or updates the learner's progress record, marking the lesson viewed, increasing the view count and recording when it was first opened. While the lesson stays open, the elapsed time is added to that record at intervals rather than only

when the page is closed, so that a browser closed abruptly still leaves an accurate record up to the last interval.

Completion is recorded separately from viewing, and the two are kept consistent by a database rule that rejects any progress record marked complete but not viewed. The distinction matters to the recommendation engine, which decides what a learner should do next from completion rather than from viewing; using viewing instead would skip every lesson the learner had merely glanced at.

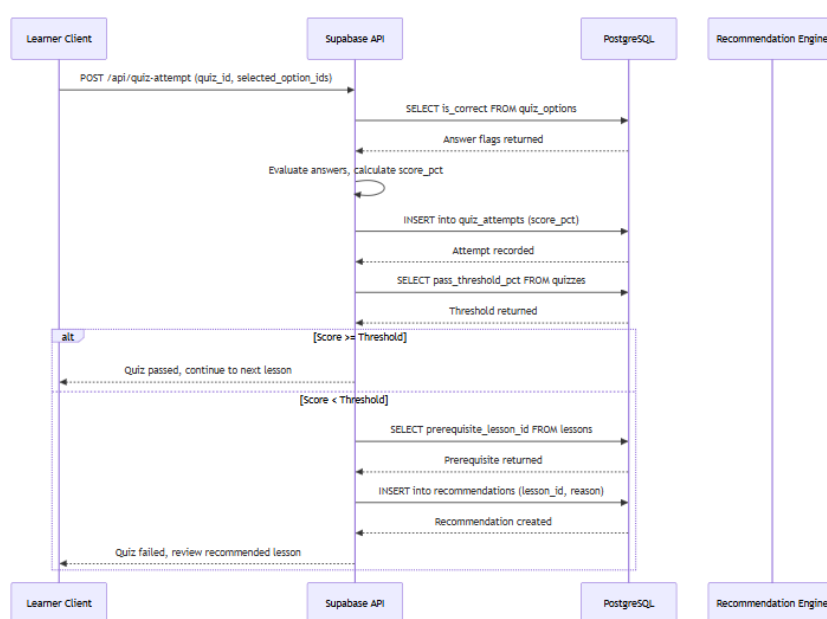


Figure 4.6: Quiz Attempt Sequence Diagram

Quiz attempt and adaptive recommendation Figure 4.6 shows the most security-sensitive interaction in the system.

1. While the learner is answering, the browser reads the answer options through a restricted view that withholds which option is correct for any quiz the learner has not yet submitted. The answer key is therefore not present in the browser at all rather than merely hidden from view.
2. On submission the browser calls a trusted database function with the quiz identifier and the selected options. It does not calculate a score and does not write the attempt

record itself; a database guard would reject such a write.

3. The function checks that the caller is authenticated and enrolled in the course, and refuses the attempt if the quiz's attempt limit has been reached.
4. It grades each answer against the stored correct option, records the attempt and its answers, and evaluates the result against the pass threshold. It returns the score, the pass decision and the threshold used, so that the interface can explain the result rather than only report it.
5. A database trigger raises a notification for the educator.
6. The answer key becomes readable to that learner for that quiz, which is what allows the review screen to show which answers were wrong.
7. The recommendation engine records a revision recommendation for the lesson, carrying the score that produced it as its stated reason.

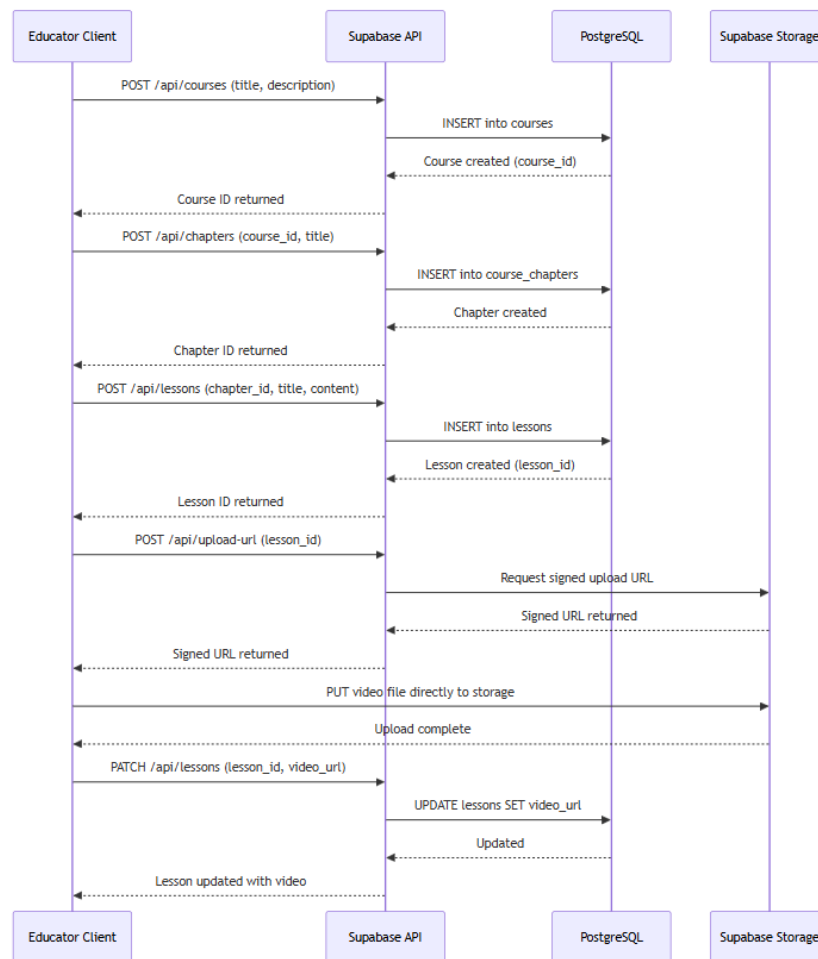


Figure 4.7: Course Creation Sequence Diagram

Course creation and media upload Figure 4.7 shows the authoring pipeline. Lesson text is composed in the rich-text editor, sanitised and saved to the lesson, with a snapshot added to the version history so that an edit can be undone later. Images and documents are uploaded from the browser directly into cloud storage, so that a large file does not have to pass through the application server, and the resulting address is recorded against the course.

Instructional video is handled differently. It is embedded from an external host by address rather than uploaded, so the lesson holds only a reference to a resource the platform does not store or serve. Throughout this pipeline the accessibility compliance checker re-runs on every change, so the accessibility consequence of an edit, such as adding a video without a transcript, is visible at the moment the edit is made rather than at publication.

4.2.3 User Interface Design

The interface is organised into three role-specific shells, each with its own navigation, input surfaces and outputs. Every figure in this section is a screenshot of the running system taken against the reproducible demonstration database described in Section 4.5.2.

4.2.3.1 Navigation Design

Navigation follows a role-based shell pattern. The area of the site a user reaches determines the shell, and a server-side check refuses a shell the signed-in role is not entitled to before the page is rendered.

- **Public navigation.** Visitors who are not signed in can reach the landing page, login, sign-up, password recovery, the contact form, the educator application form and the published accessibility statement. Certificate verification is also public, so that a third party can check a certificate without an account.
- **Learner shell.** A sidebar carries Dashboard, My Courses, Progress, Achievements and Certificates, and Accessibility. A top bar holds the notification indicator and the profile menu. Two properties of this sidebar are deliberate. Every item is a real hyperlink rather than a click handler, so that a screen reader announces navigation as navigation and the browser's own behaviour such as opening a link in a new tab continues to work. The sidebar also responds to the accessibility preset: the Dyslexia preset widens it and enlarges its text, and the distraction-free presets remove it entirely while a lesson is open.
- **Educator shell.** A sidebar carries Dashboard, Courses, Educator Ranking, Analytics, Learner Progress and Certificates. Inside a course, a tab bar provides Overview, Lessons, Assets, Learners, Certificates, Achievements and Settings.
- **Administrator shell.** A sidebar carries Dashboard, User Management, Course Management, Educator Applications, Feedback, Certificates, Analytics and Reports.

Figure 4.8 shows the learner dashboard with no accessibility preset applied, and Figure 4.9 shows the corresponding educator course workspace with the tab bar, the ordered lesson list and the per-lesson authoring controls.

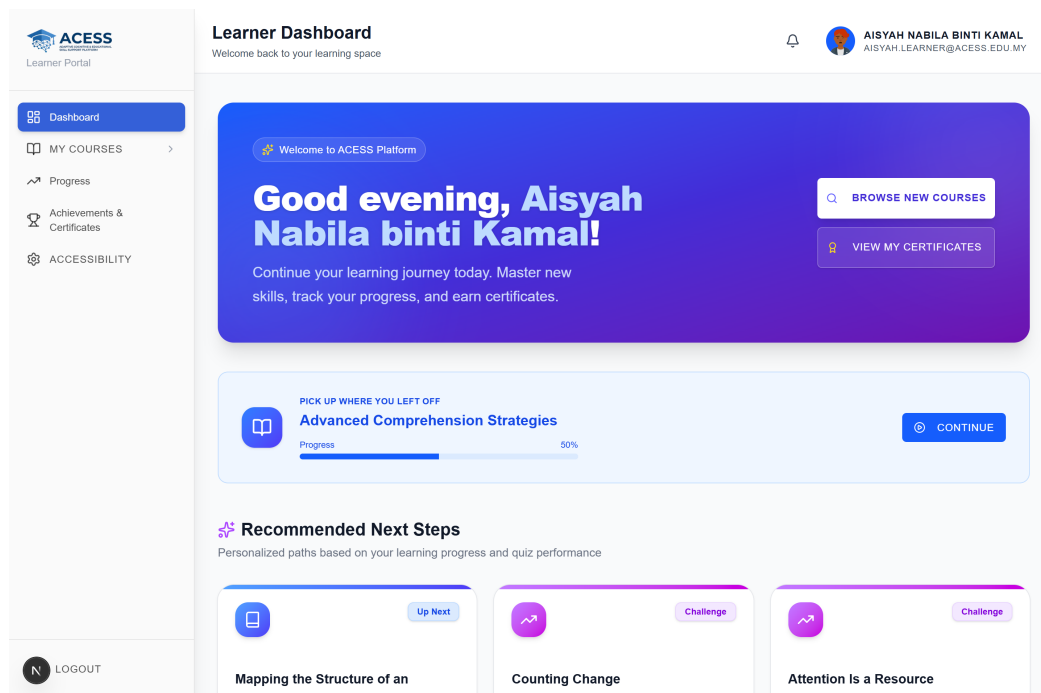


Figure 4.8: Learner Dashboard in the Default State

4.2.3.2 Input Design

The input surfaces of the system are grouped below by the role that uses them.

- **Public input.** A visitor supplies an email address and password at sign-up, and a category, subject and message on the contact form. A requested role is never accepted from the sign-up request itself: self-registration is always fixed to the learner role by the database.
- **Learner input.** A learner selects a course to enrol in, adjusts accessibility settings such as text size and line spacing from the accessibility panel shown in Figure 4.10, answers quiz questions as shown in Figure 4.11, and uploads a profile picture.
- **Educator input.** An educator supplies course metadata such as title, address, difficulty and status, writes lesson content in the rich-text editor, sets the order and layout of each lesson, and creates quiz questions and their answer options.

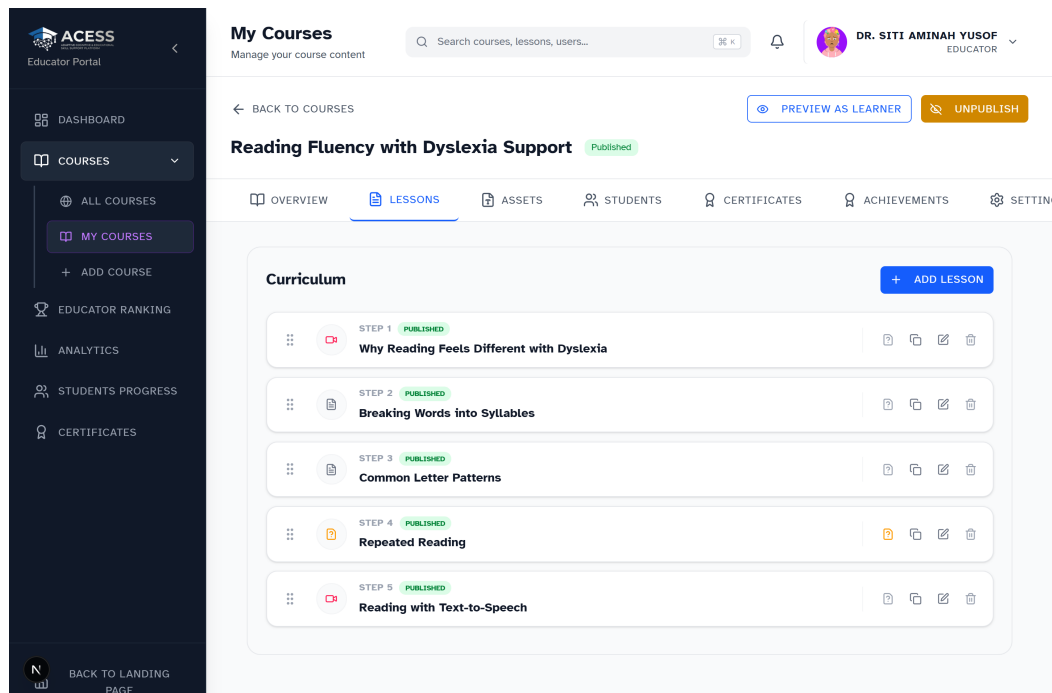


Figure 4.9: Educator Course Workspace, Lessons Tab

- **Administrator input.** An administrator sets a user's role, decides whether a submitted course is approved or rejected, and selects the date range for a platform report.

Table 4.1 states the validation rule applied to each significant field and the layer at which it is enforced. The distinction matters, because a rule enforced only in the browser is a convenience, whereas a rule enforced by a database constraint or an access policy cannot be bypassed by a crafted request.

Table 4.1: Input Validation Rules

Screen	Field	Validation rule	Enforced at
Sign up	Email	Well-formed address, must be unique	Browser, then authentication service
Sign up	Password	Minimum length and complexity	Browser, then authentication service
Sign up	Role	Always learner, not accepted from the request	Database trigger
Course form	Title	Required, must not be blank	Browser and database
Course form	Address	Unique across all courses	Database constraint
Course form	Difficulty	One of beginner, intermediate or advanced	Database constraint

continued on the next page

Table 4.1 continued from the previous page

Screen	Field	Validation rule	Enforced at
Course form	Status	One of draft, pending review, published or archived	Database constraint
Lesson editor	Rich text	Sanitised before display, disallowed markup removed	Browser sanitiser
Lesson editor	Sequence order	Integer greater than zero	Database constraint
Lesson editor	Lesson layout	One of the five permitted layouts	Database constraint
Quiz builder	Pass threshold	Integer between 0 and 100	Database constraint
Quiz builder	Answer options	At least two options, exactly one correct	Browser
Quiz attempt	Selected options	Graded on the server, answer key not sent early	Database function and view
Accessibility panel	Text size	Between 12 and 24 px	Browser
Accessibility panel	Line spacing	Between 1.0 and 3.0	Browser
Enrolment	Course reference	A learner may only enrol themselves	Access policy
Enrolment	Status	One of active, completed or dropped; completion cannot be set by the browser	Database constraint and guard
Contact form	Category	One of five permitted categories	Database constraint
Profile picture	File location	Must sit inside the uploader's own folder	Storage policy

The most important input surface in the system is the accessibility panel shown in Figure 4.10. It presents four preset buttons above five grouped tabs (Reading, Focus, Sensory, Supports and Profile), and every control is saved to the learner's settings and takes effect without reloading the page.

The second significant input surface is the quiz shown in Figure 4.11. Its design carries three accessibility decisions: questions are presented one per screen regardless of the lesson's own layout setting, a time limit set by the educator is displayed to the learner, and the conditions of the attempt are stated before it begins rather than discovered during it.

4.2.3.3 Output Design

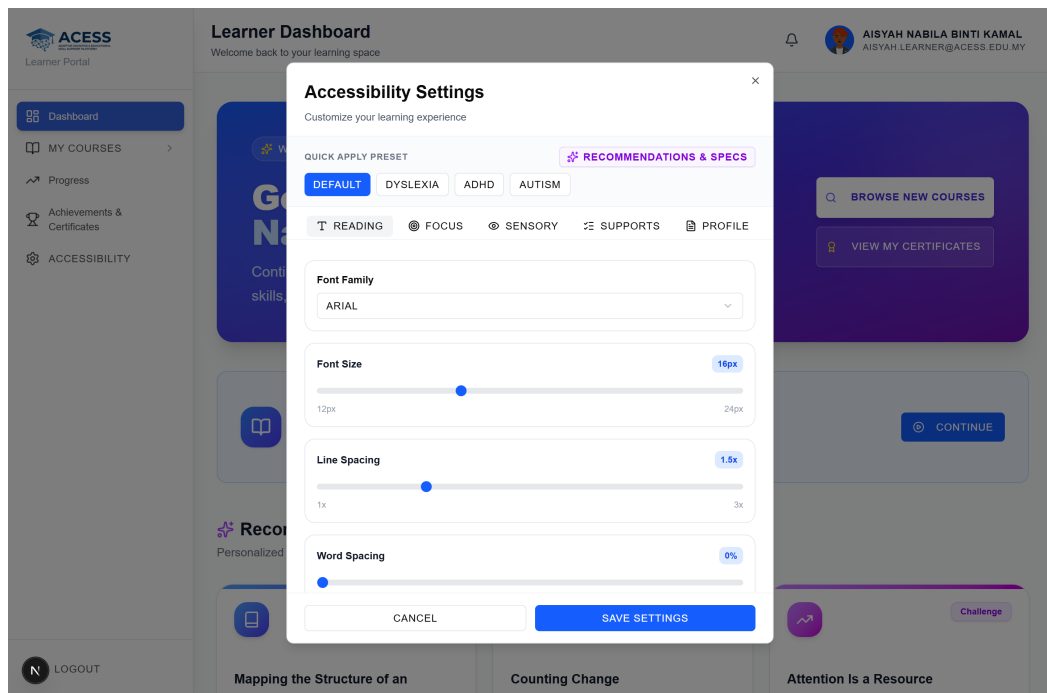


Figure 4.10: Accessibility Settings Panel, Reading Group

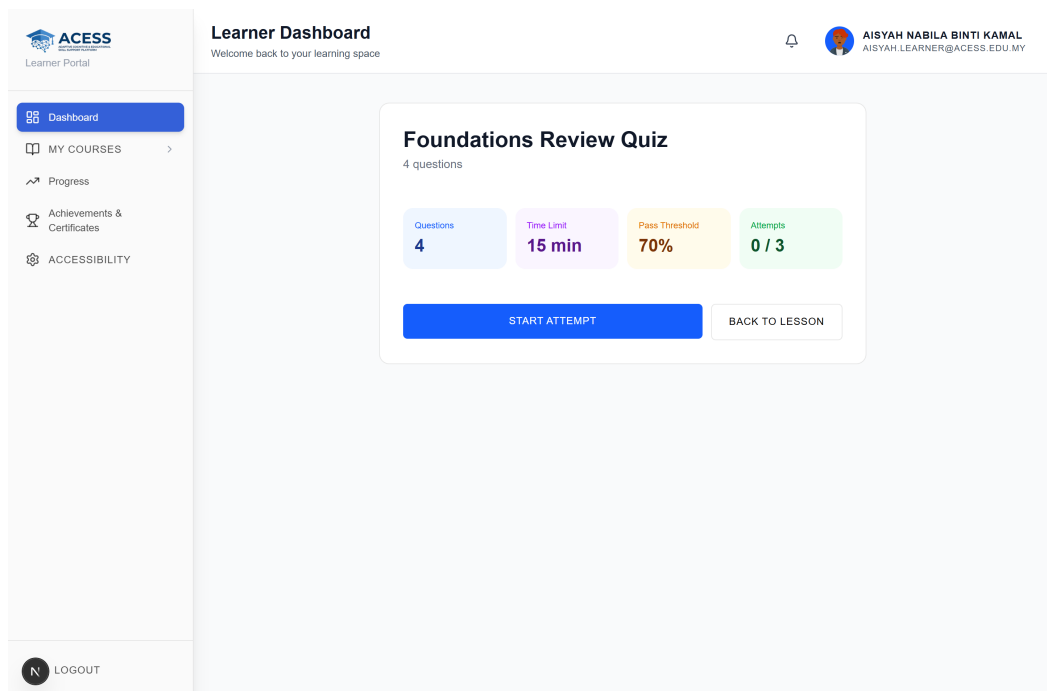


Figure 4.11: Quiz Interface

Outputs fall into three classes: screen output produced on demand, document output produced on demand, and output raised automatically by an event. Table 4.2 classifies every output by its basis, its audience and its content.

Table 4.2: System Outputs

Output	Basis	Audience	Content
Learner progress page	On demand	Learner	Course completion, lessons completed, study time, quiz history, visit history
Course analytics	On demand	Educator	Enrolment and completion trends, per-lesson completion, quiz score distribution, at-risk flags
Platform analytics	On demand	Administrator	Registrations, enrolments, learning time, accessibility feature use and preset adoption
Accessibility compliance report	Continuous	Educator	Per-lesson and per-course score, unmet rules, the source of each rule
Administrator report (PDF)	On demand	Administrator	Platform, course, learner or content-coverage report over a chosen date range
Completion certificate (PDF)	Event	Learner	Learner name, course, completion date, reference code, verification QR code
Certificate verification page	On demand	Public	Validity, issue date and revocation status for a reference code
Notifications	Event	All roles	Enrolment, lesson progress, quiz attempt, lesson added, course published
Achievement award	Event	Learner	Badge unlocked when a course achievement requirement is met

Figure 4.12 shows the learner progress page, with completion, study time and quiz history for the signed-in learner, and Figure 4.13 shows the administrator equivalent with platform-level summary tiles and the registration trend. Both use the same charting library, so that a chart carries the same visual conventions wherever it appears.

One output deserves separate mention because it is unusual in a learning platform. The administrator analytics view includes an Accessibility tab, shown in Figure 4.14, which reports which accessibility features are actually being used and which presets learners actually select. Accessibility features are often built and then never measured; this view exists so that the question of whether they are used has an answer.

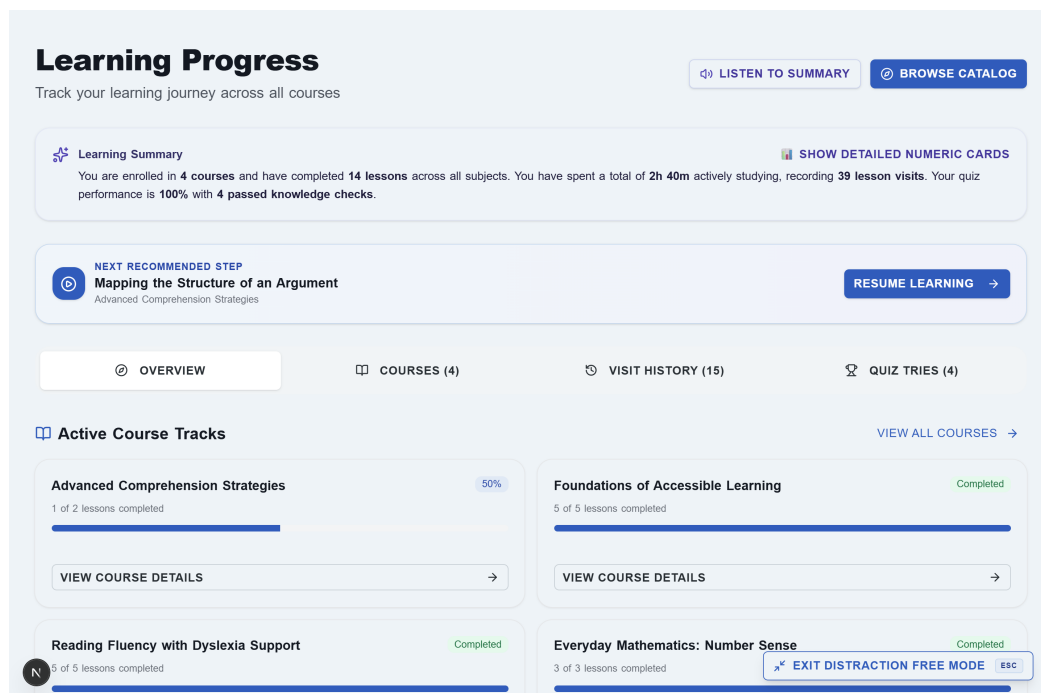


Figure 4.12: Learner Progress Page

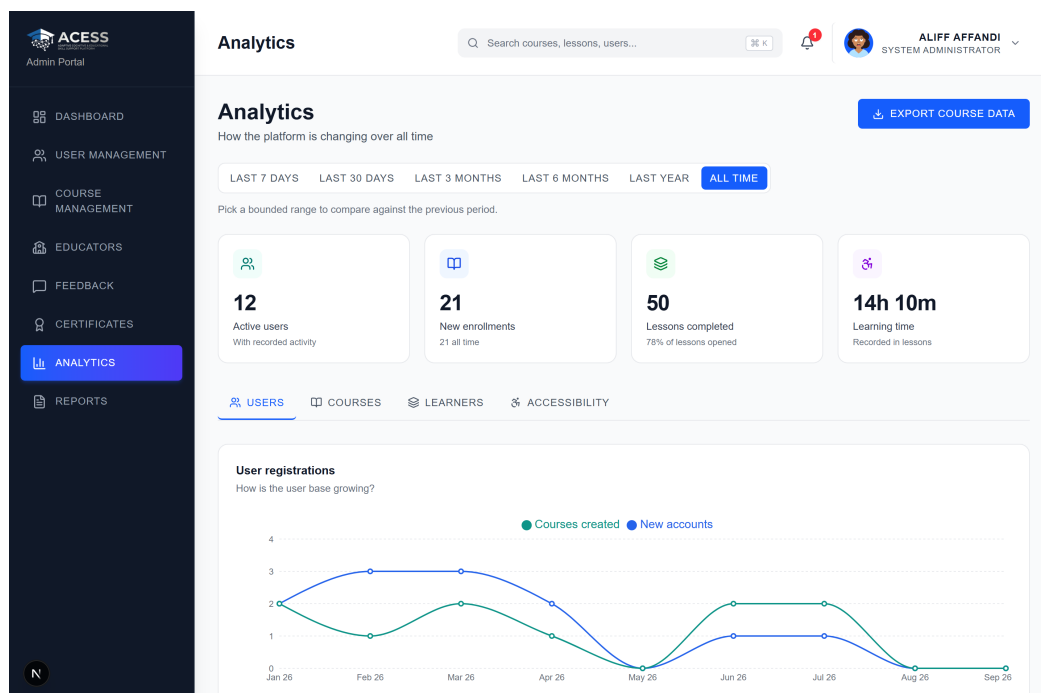


Figure 4.13: Administrator Analytics Dashboard

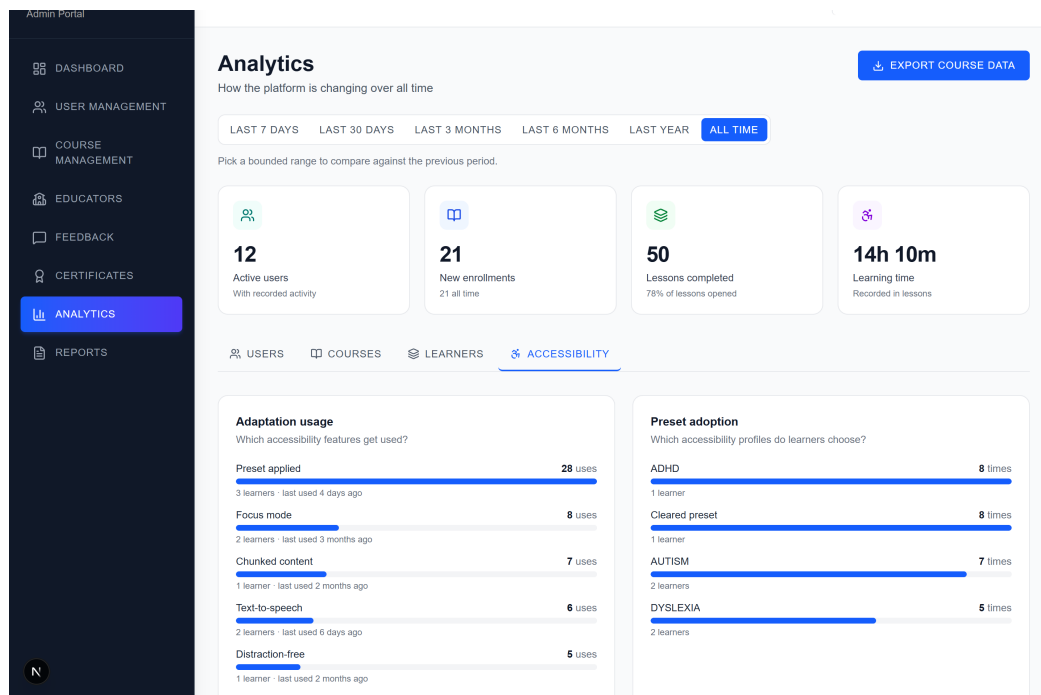


Figure 4.14: Accessibility Analytics

4.2.4 Conceptual and Logical Database Design

The ACCESS data model is a normalised relational schema implemented on PostgreSQL. It comprises 36 tables, one view, 370 columns, 57 foreign keys, 33 value constraints, 20 uniqueness constraints, 115 indexes, 20 functions, 15 triggers and 88 access policies. These figures were obtained by querying the system catalogue of a database rebuilt from the project's own schema definitions rather than estimated, and the same query can be re-run to confirm them.

The entity relationship diagrams in this section were generated from that live schema by a script in the project repository rather than drawn by hand, so that a diagram in this report cannot fall out of step with the database it describes. Because thirty-six entities cannot be shown legibly on a single portrait page, the model is presented as three domain diagrams followed by the complete schema in landscape orientation.

Business rules governing the relationships The following rules were derived from the operational requirements of the platform, and they are what the foreign keys, uniqueness rules and deletion rules in the schema encode.

1. **One account has one profile.** Every account has exactly one extended profile, which is removed automatically when the account is removed.
2. **A course must have an owner, and that owner cannot be removed.** Deleting the educator who owns a course is refused. This is the only such restriction in the schema, and it exists so that removing an account cannot silently destroy courses other people are enrolled in.
3. **A learner enrolls in a course at most once.** A uniqueness rule over the learner and course pair enforces this. Enrolling again reactivates the existing enrolment rather than creating a second one, which keeps the learner's progress history continuous.
4. **All learning activity belongs to the enrolment.** Progress, quiz attempts, checkpoints, recommendations and certificates all reference the enrolment rather than the learner directly, so that the enrolment is the single unit of participation and withdrawing a learner from one course affects nothing else.
5. **A lesson belongs to exactly one course and optionally to one chapter.** The course link is required; the chapter link is optional, because chapter organisation is a feature an educator may or may not use.
6. **A lesson may declare one prerequisite lesson.** If that prerequisite is later removed, the requirement is cleared rather than leaving the dependent lesson unusable.
7. **Assessment is strictly hierarchical.** A lesson has at most one quiz, a quiz has many questions, and a question has many answer options of which exactly one is correct. An attempt records one answer per question, and an option referenced by a submitted attempt cannot be deleted.
8. **One certificate per enrolment.** A revoked certificate keeps its record and its reference code, with its status changed to revoked, and a constraint requires a revoked record to carry a revocation date while an issued one does not. This is what allows public verification to report that a certificate was revoked rather than that it does not exist.
9. **An achievement is earned once per learner per course.** The record links the learner, the achievement definition and the course, and records are created only by

the platform's own derivation logic.

10. **Comments form a thread.** A comment may reference another comment as its parent, which gives one level of reply nesting.
11. **Accessibility settings belong to the learner, not to an enrolment.** A learner's presentation preferences follow them across every course, so they are stored on the learner's profile.

Domain 1: identity, access and platform Figure 4.15 shows the identity domain. It holds the account record, the extended profile carrying accessibility and notification preferences, the educator application workflow, referral codes, notifications, reusable accessibility content templates, and the records of which accessibility features are actually used. Every entity in this domain references the account record directly, which is why its access policies are the simplest in the schema.

Domain 2: curriculum and content Figure 4.16 shows the curriculum domain. It holds the course, chapter and lesson hierarchy, the artefacts attached to a lesson (interactive activities, in-video questions, embedded content, checkpoints, stored lesson summaries and discussion threads), and the supporting infrastructure of templates, version snapshots and uploaded media. Everything in this domain is authored by an educator.

Domain 3: assessment, learning record and recognition Figure 4.17 shows the assessment and learning record domain, which holds everything generated by a learner: the enrolment itself, per-lesson progress, quiz attempts and answers, checkpoint and activity responses, generated recommendations, saved courses, and the recognition records of achievements, milestones and certificates. The convergence of these entities on the enrolment is what business rule 4 above describes.

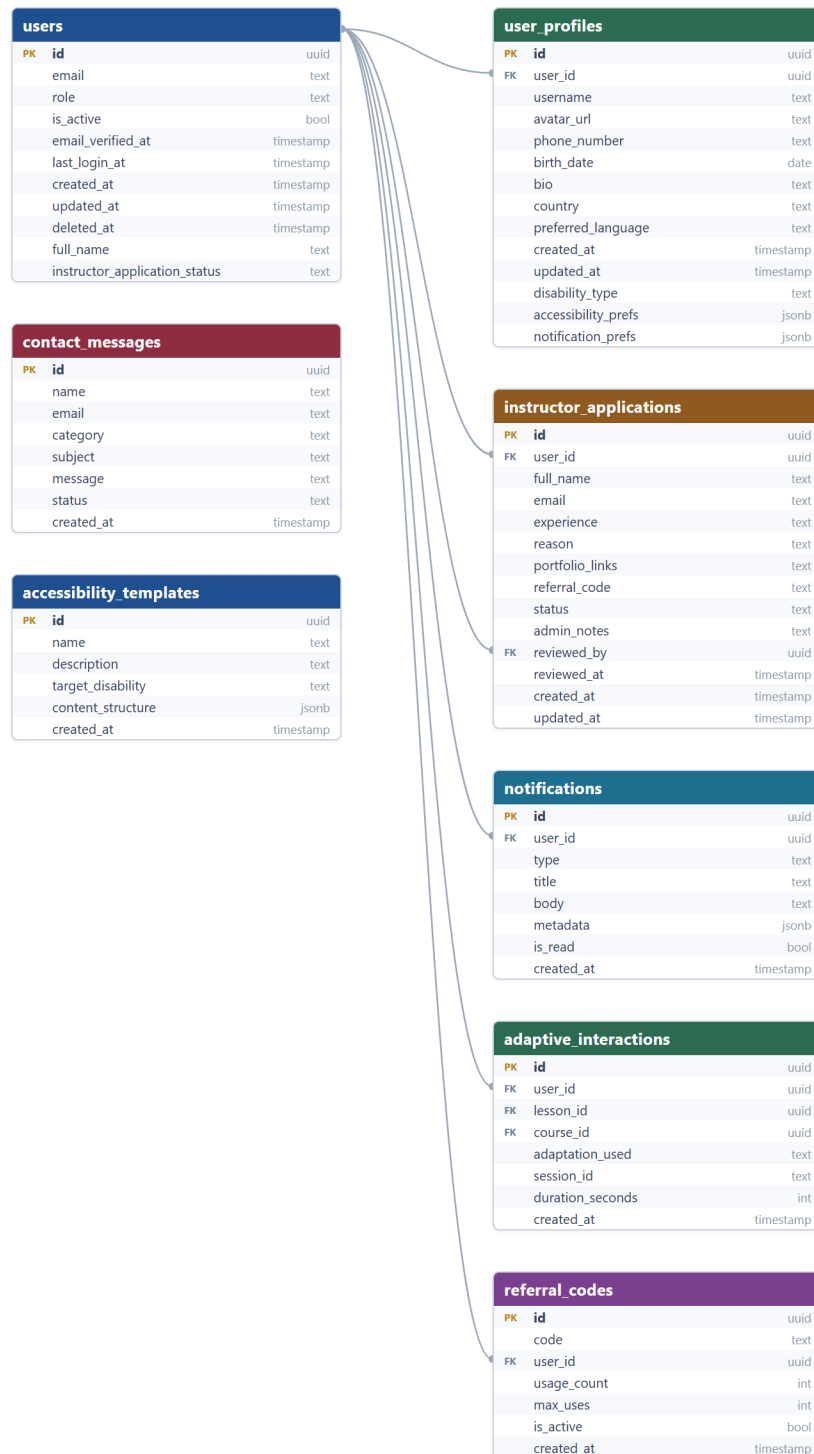


Figure 4.15: Identity, Access and Platform Entity Relationship Diagram

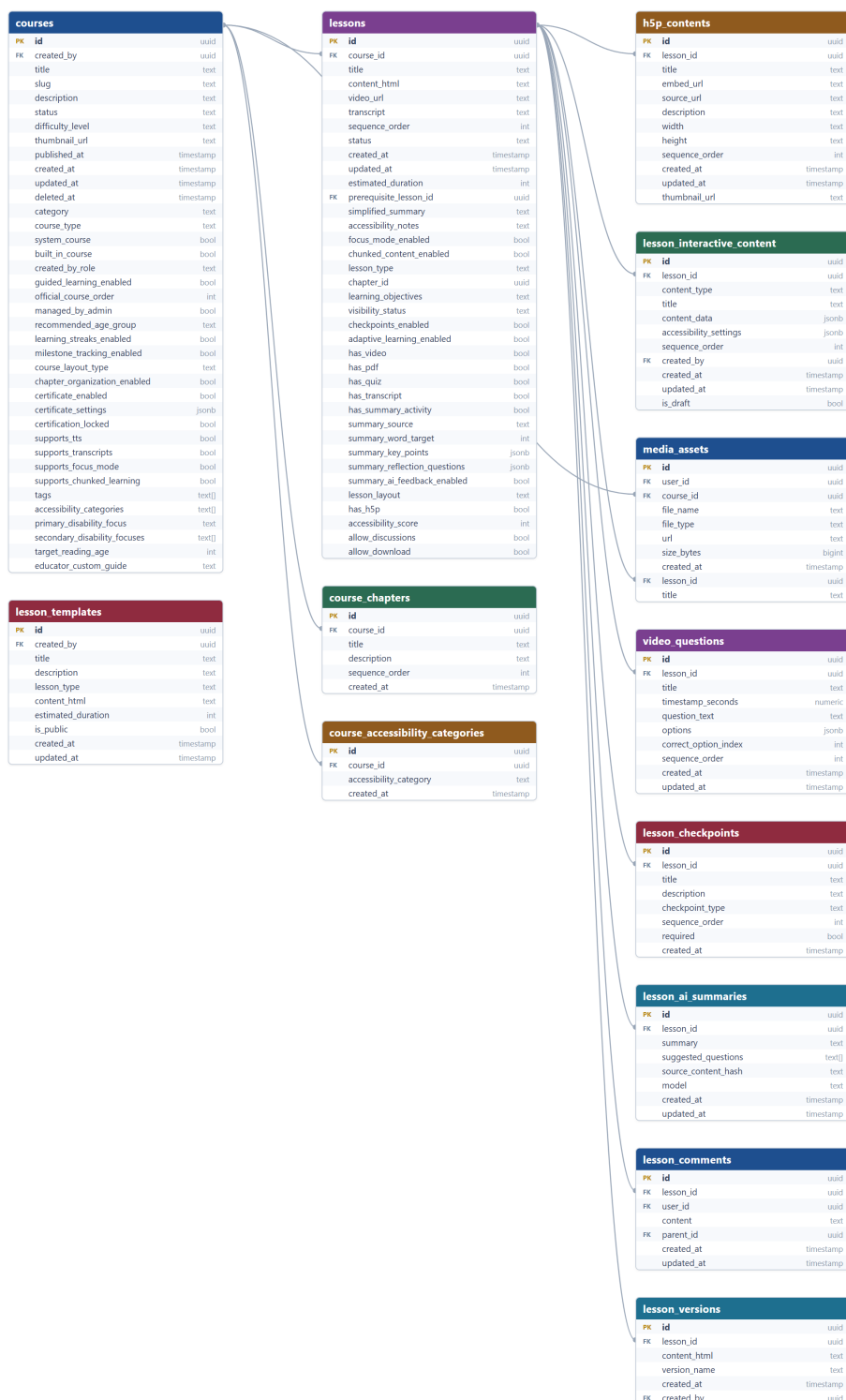


Figure 4.16: Curriculum and Content Entity Relationship Diagram

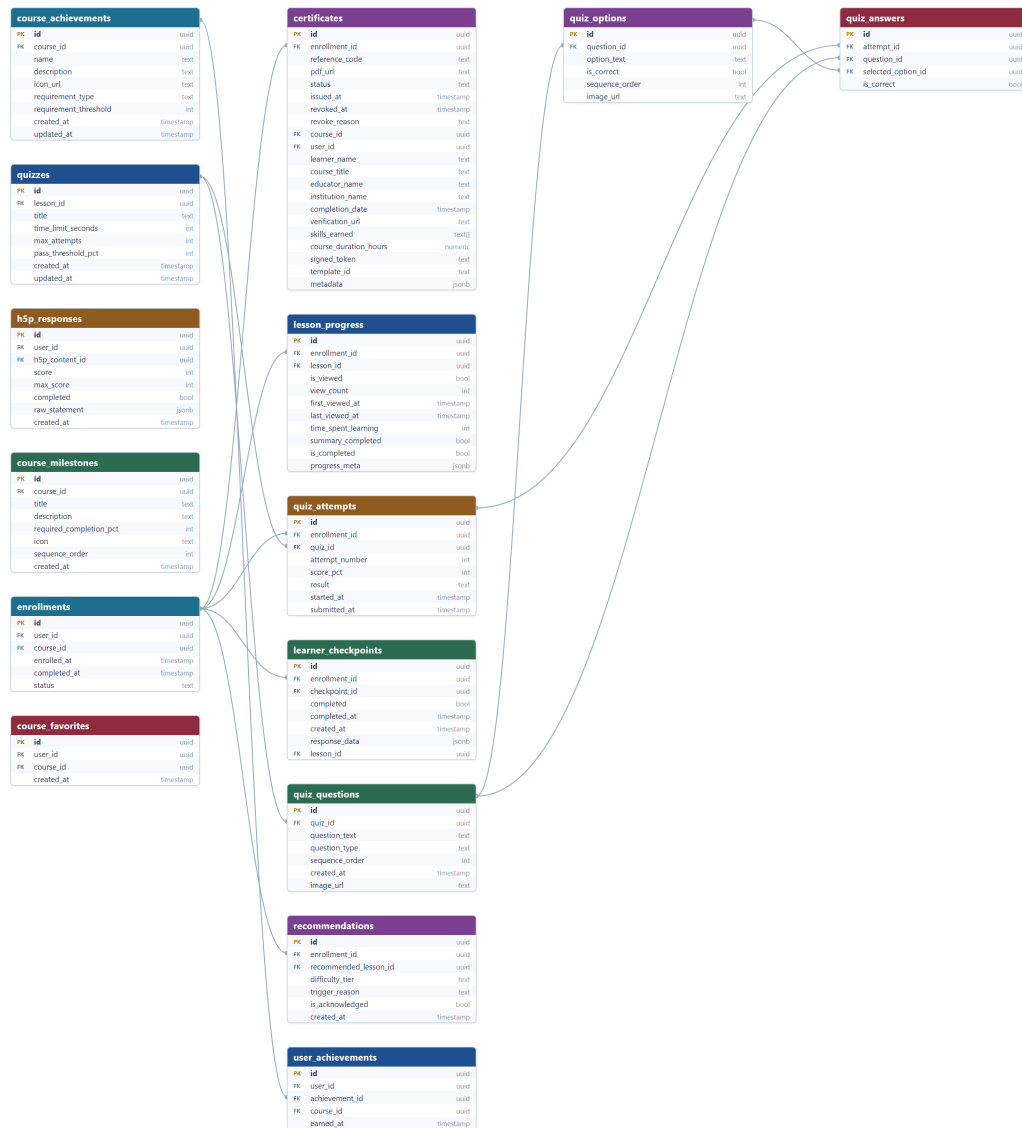


Figure 4.17: Assessment, Learning Record and Recognition Entity Relationship Diagram

Figure 4.18: Complete ACCESS Entity Relationship Diagram

Figure 4.18 presents the full model of all 36 tables and 57 foreign keys. The account record acts as the identity hub and the enrolment as the participation hub: nearly every record generated by a learner reaches the account record through an enrolment rather than directly, which is what allows most access policies to be expressed as a single relationship check.

4.3 Accessibility Design

4.3.1 Design approach and standards baseline

Accessibility in ACESS is treated as a design constraint that the rest of the system is arranged around rather than as a settings panel added to a finished product. Three commitments follow from that position, and they explain most of the design decisions in this section.

The first is that a control must do what its label says. During development several settings were found to have no effect: a word-spacing control that a stylesheet rule silently overrode, a line-spacing control that changed the settings preview but not the lesson itself, and a simplified reading mode that could not be reached from anywhere in the interface. Each of these appeared to work and did not. They were corrected, and the platform now maintains an explicit catalogue of every learner-facing setting, checked automatically, under the rule that a setting which is not in the catalogue does not exist. Two settings that still do not fully match their labels are recorded as known gaps in Section 4.3.7 rather than presented as working.

The second is that the standards baseline has to be wider than WCAG. WCAG 2.2 is necessary but was not written primarily for cognitive difference: it can establish that contrast is insufficient, but not that a paragraph is too long for a learner to re-enter after being interrupted. ACESS therefore works from three sources together: WCAG 2.2 (W3C Web Accessibility Initiative, 2023) for testable technical criteria, the W3C guidance on making content usable for people with cognitive and learning disabilities (W3C Web Accessibility Initiative, 2021) for cognitive design objectives, and the British

Dyslexia Association Style Guide (British Dyslexia Association, 2023) for typographic and readability thresholds.

The third is that the claim must not exceed the evidence. ACCESS targets WCAG 2.2 Level AA together with four Level AAA criteria. It has not been independently audited, has not been tested with a screen reader or other assistive technology, and has not been tested with neurodivergent learners. That statement is published on the platform's own accessibility statement page, where a prospective learner can read it, and it is repeated in Section 4.3.7.

4.3.2 Mapping of standards to implementation

Table 4.3 maps each addressed criterion to the ACCESS feature that addresses it, the learner profile it primarily serves, and the part of the system in which it is implemented. The final column names the figure in this report that shows it, where one exists.

Table 4.3: Mapping of Accessibility Criteria to ACCESS Features

Criterion	ACCESS feature addressing it	Profile served	Implemented in	Evidence
1.1.1 Non-text Content (A)	The compliance checker requires alternative text on every image and provides a field to supply it	All	Compliance checker	Fig. 4.24
1.2.1 Video-only (A)	A lesson containing video but no transcript is reported as a required failure	All	Compliance checker, lesson record	Fig. 4.24
1.2.2 Captions (A)	Captions are shown on video by default when the learner enables the setting	Hearing, slower processing	Accessibility module	Fig. 4.10
1.3.1 Info and Relationships (A)	The checker rejects empty paragraphs used as spacing and requires real headings	All	Compliance checker	Fig. 4.24
1.4.4 Resize Text (AA)	Text size adjustable between 12 and 24 px, applied without reloading the page	Dyslexia	Accessibility module	Fig. 4.10
1.4.5 Images of Text (AA)	The checker requires body content to be real text so that read-aloud can reach it	Dyslexia	Compliance checker	Fig. 4.24
1.4.8 Visual Presentation (AAA)	Reading width bounded in character units (72 default, 62 Dyslexia, 66 ADHD) and retained when navigation is hidden	Dyslexia, ADHD	Stylesheet	Fig. 4.20
1.4.12 Text Spacing (AA)	Line spacing between 1.0 and 3.0 and word spacing up to 0.2 em, both learner-set, with the Dyslexia preset at the required 0.16 em	Dyslexia	Accessibility module	Fig. 4.20
2.2.1 Timing Adjustable (A)	A quiz time limit set by an educator is displayed to the learner	ADHD, Autism	Quiz interface	Fig. 4.11

continued on the next page

Table 4.3 continued from the previous page

Criterion	ACCESS feature addressing it	Profile served	Implemented in	Evidence
2.2.4 Interruptions (AAA)	Distraction-free mode removes navigation and notification elements during a lesson	ADHD	Stylesheet, lesson view	Fig. 4.21
2.2.6 Timeouts (AAA)	A fifteen-minute inactivity timeout announced by a warning thirty seconds beforehand	All	Session timeout component	N/A
2.3.3 Animation (AAA)	One motion control (normal, low or none) reduces or removes animation across the site	Autism, ADHD	Accessibility module	Fig. 4.10
2.4.4 Link Purpose (A)	The checker flags link text that does not describe its destination	All	Compliance checker	Fig. 4.24
2.4.6 Headings and Labels (AA)	The checker requires headings once a lesson passes 200 words	All	Compliance checker	Fig. 4.24
3.1.5 Reading Level (AAA)	The checker computes a Flesch–Kincaid grade level and compares it with the course’s target reading age (Kincaid et al., 1975)	Dyslexia	Compliance checker	Fig. 4.24
3.2.3 Consistent Navigation (AA)	Identical shell and navigation order on every page of a role, and the checker verifies structural consistency between lessons of one course	Autism	Navigation, compliance checker	Fig. 4.8
3.2.5 Change on Request (AAA)	A preset shows exactly what it will change before applying it, read-aloud never starts by itself, and no media plays automatically	All	Preset preview dialog	Fig. 4.19
4.1.2 Name, Role, Value (A)	Navigation built from real links carrying current-page state, and icon-only controls given accessible names and states	All	Navigation and lesson header	Fig. 4.8
W3C cognitive guidance, objectives 3–5	Content presented in sections, per-section time estimates, visual schedule and step-by-step guidance	ADHD, Autism	Visual schedule and guidance components	Fig. 4.22

Table 4.3 makes one structural point worth drawing out. The criteria divide into two groups according to who can satisfy them. Presentation criteria such as 1.4.4, 1.4.8, 1.4.12, 2.3.3, 2.2.6 and 3.2.5 are satisfied by the platform once, for every learner. Content criteria such as 1.1.1, 1.2.1, 1.3.1, 2.4.4, 2.4.6 and 3.1.5 cannot be satisfied by the platform at all, because they are properties of what an educator writes. A platform can only make the requirement visible at the moment of writing, which is what the accessibility compliance checker described in Section 4.3.6 exists to do.

4.3.3 The three learner profiles

Table 4.4 states, for each of the three profiles the platform is designed around, the accessibility needs it addresses and the support ACCESS provides. Every entry in the third column corresponds to a feature that is implemented.

Table 4.4: Learner Profiles and the Support Provided by ACESS

Profile	Main accessibility needs	ACESS support
Dyslexia	• Reduced decoding effort • Reduced visual crowding between letters, words and lines • A reliable way to keep one's place in a long passage • An alternative to reading • Reduced glare over long sessions	• Atkinson Hyperlegible or OpenDyslexic typeface, 19 px base size, 1.7 line spacing • Word spacing at the 0.16 em value required by WCAG • Reading width narrowed to 62 characters, within the 50 to 70 range recommended by the British Dyslexia Association • Cream background, and a reading spotlight that dims all but the current paragraph • Read-aloud with adjustable rate and a click-to-read-from-here control • A reading toolbar carrying size, spacing, tint and spotlight controls without leaving the lesson • Italic text rendered as bold, and text never justified • Content presented one section at a time, on a single-column dashboard • The compliance checker verifies sentence length and readability against the course's target reading age
ADHD	• Sustained attention is costly • Starting a task is difficult • An interruption destroys one's place • Passive reading is poorly retained • Competing on-screen elements all draw attention equally	• Distraction-free mode hiding the sidebar and notification area • Reading width held at 66 characters, including while navigation is hidden • A persistent bar carrying the current task and its progress • A task checklist and a progress timeline • An auto-save indicator, so that leaving the page carries no risk • Reading spotlight • Content divided into sections with a reading-time estimate for each • The compliance checker caps a lesson at 20 minutes and an unbroken block at 150 words, expects a section break roughly every 400 words, expects video under six minutes or in-video questions, and requires at least one activity in every lesson
Autism	• Uncertainty about what is coming is itself a barrier • Sensory input can overload • Figurative language must be decoded before the content can be • Inconsistent structure has to be relearned each time	• A visual schedule listing every phase of the lesson, with progress, before the learner starts • Step-by-step guidance that states why a control is unavailable rather than only disabling it • Checklist structure mode, which states expectations before each task • Motion removed, colour saturation reduced, pale blue background • Reading width fixed at the same value for every course, so pages do not change shape between lessons • No automatic playback anywhere • The compliance checker looks for figurative language, requires at least two stated learning objectives and a plain-language summary, and verifies structural consistency with the other lessons of the same course

4.3.4 Preset composition

An accessibility preset is not simply a theme. It is a single decision by the learner that sets nineteen individual values at once, covering typography, colour, layout, pacing and executive-function support. Table 4.5 gives the exact values, taken from the

platform's own preset definitions. Every one of these settings remains individually adjustable afterwards, so choosing a preset is a starting point rather than a mode the learner is locked into.

Table 4.5: Accessibility Preset Values

Setting	Default	Dyslexia	ADHD	Autism
Typeface	Arial	Atkinson Hyperlegible	Arial	Arial
Base text size	16 px	19 px	18 px	18 px
Line spacing	1.5	1.7	1.6	1.6
Word spacing	0 % (0 em)	40 % (0.16 em)	20 % (0.08 em)	20 % (0.08 em)
Reading width	72 characters	62 characters	66 characters	72 characters
Background tint	White	Cream	Grey	Pale blue
Reading spotlight	Off	On	On	Off
Distraction-free mode	Off	Off	On	On
Layout mode	Slide	One section at a time	One section at a time	Scroll
Structure mode	Full schedule	Full schedule	Minimal	Checklist
Motion level	Normal	Low	Low	None
Muted colours	Off	Off	Off	On
Read-aloud	Off	On	Off	Off
Task checklist	Off	Off	On	Off
Visual schedule	Off	Off	Off	On
Step-by-step guidance	Off	Off	Off	On
Progress timeline	Off	Off	On	On
Auto-save indicator	Off	On	On	On
Lesson divided into sections	Off	On	On	On

Applying a preset is itself an accessibility decision and is handled as one. Figure 4.19 shows the dialog that appears before a preset takes effect. It lists every value that will change and what it will change to, offers an explanation for each, and states that any of the values can be adjusted afterwards. This is the platform's implementation of WCAG 3.2.5, and it exists because an interface that rearranges itself without warning is precisely the kind of unpredictability the Autism preset is intended to remove.

4.3.5 Effect of the presets on the lesson interface

Figures 4.20 to 4.22 show the same lesson, viewed by the same learner account at the same window size, with only the accessibility preset changed between captures. The preset values in each case were applied by the platform itself and match Table 4.5, so the comparison shows what a learner selecting that preset actually receives.

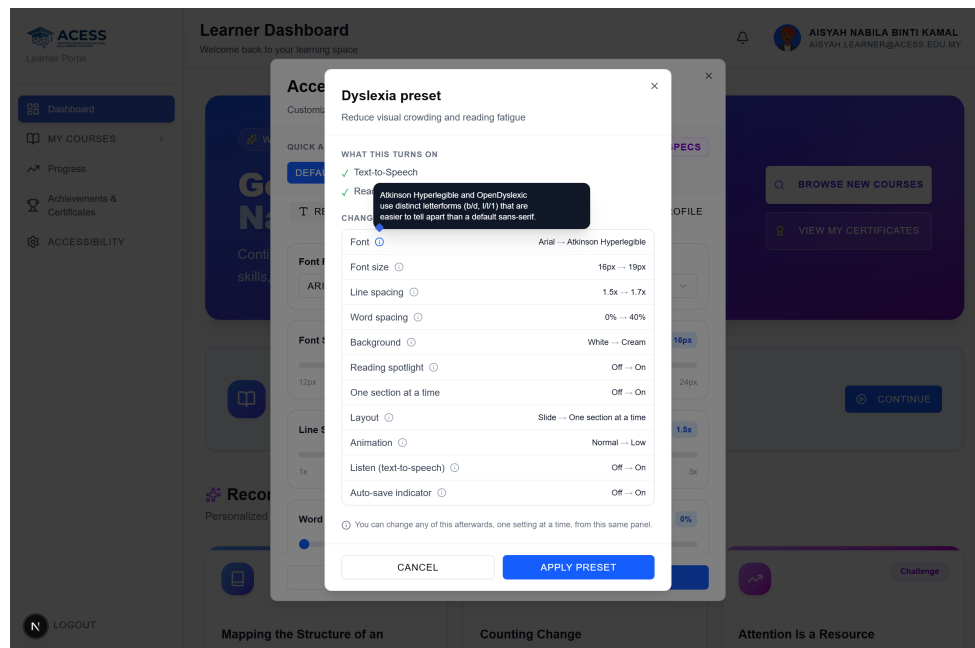
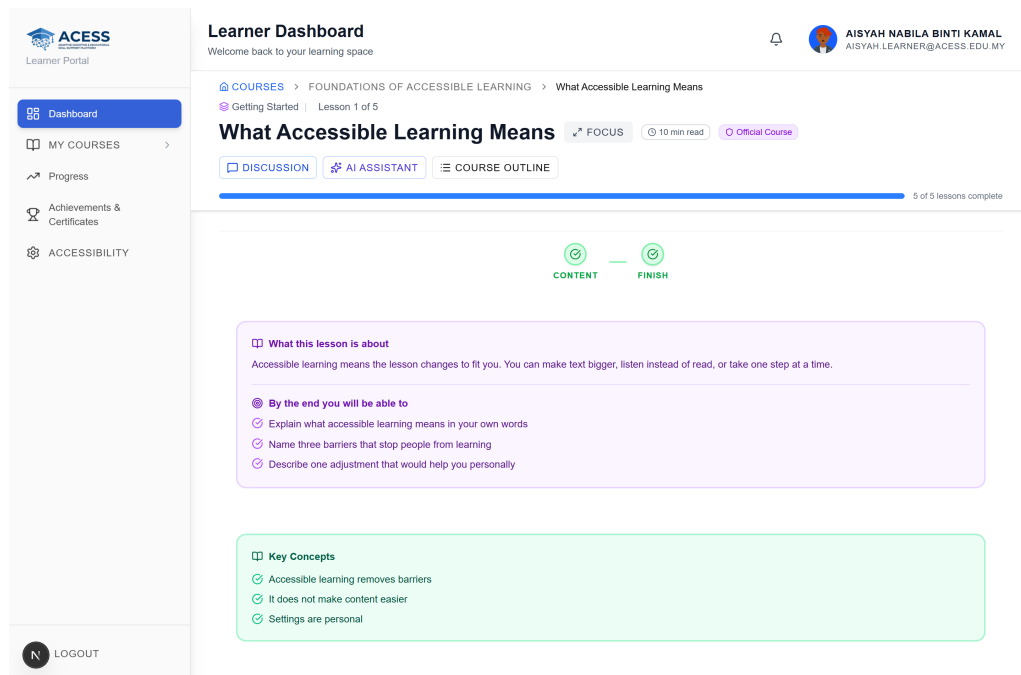


Figure 4.19: Preset Preview Dialog

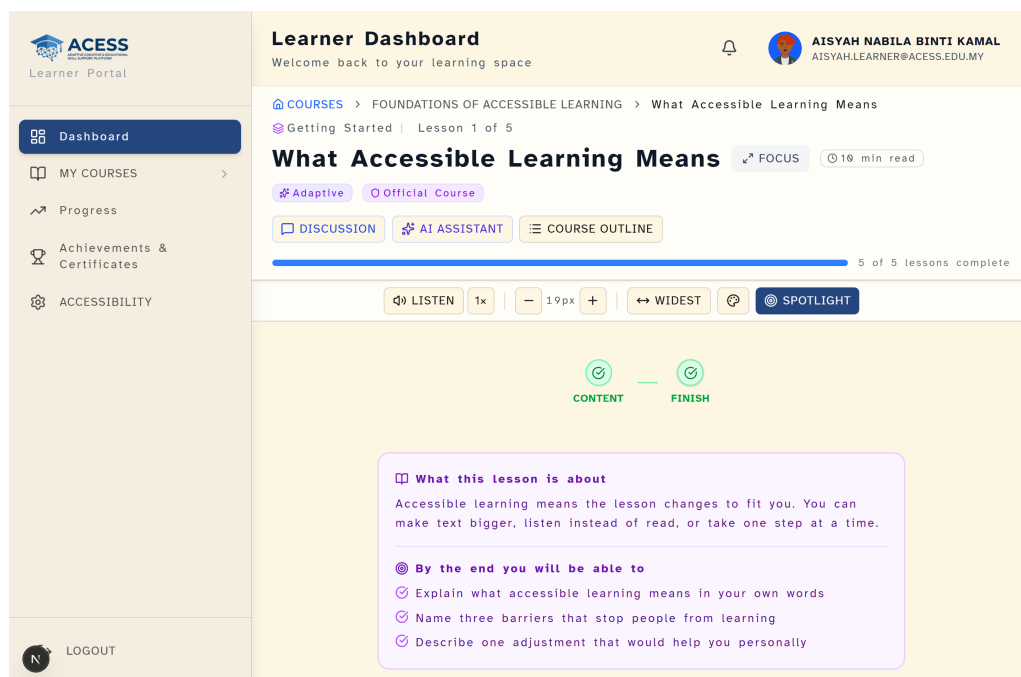
Figure 4.20 shows the changes most readers would expect from an accessibility feature, namely typography and colour, together with two that are easy to miss. The reading column has narrowed rather than widened, because the limit is expressed in characters and the characters have become larger. A toolbar has also appeared below the lesson header carrying Listen, size, spacing, tint and spotlight controls, so that the learner can adjust the presentation without leaving the lesson to find a settings panel. Requiring a learner to navigate away in order to make reading easier would itself be a barrier.

Figure 4.21 illustrates the difference between hiding elements and supporting attention. The sidebar and notification area are removed, a persistent bar carries the current task together with an auto-save indicator, and the reading spotlight dims every paragraph except the one in focus. Everything competing for attention has gone, but the reading column has not expanded to fill the space and remains bounded at 66 characters. An exit control in the lower right, and the keyboard shortcut it advertises, ensure that the mode is never a trap.

Figure 4.22 shows the least visually dramatic and most structurally significant of the three, with its pale blue background, reduced colour saturation and motion removed. The substantive change is that the phases of the lesson are shown as an explicit sequence above the content, which tells the learner what the lesson consists of and how far through



(a) Default



(b) Dyslexia preset

Figure 4.20: Lesson Interface: Default Compared with the Dyslexia Preset

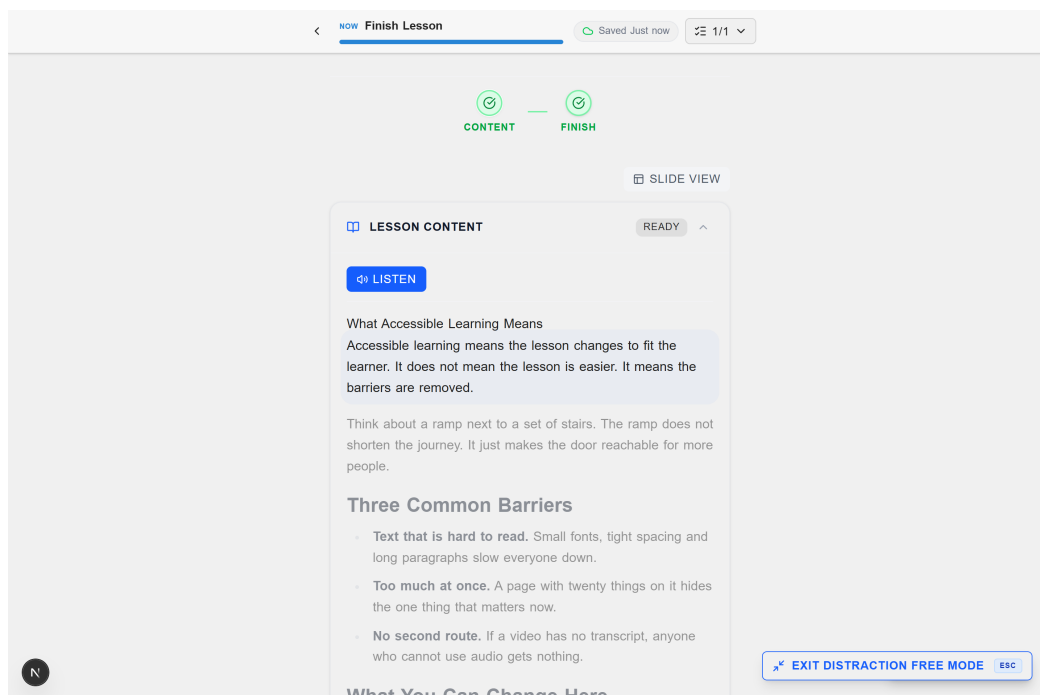


Figure 4.21: Lesson Interface Under the ADHD Preset

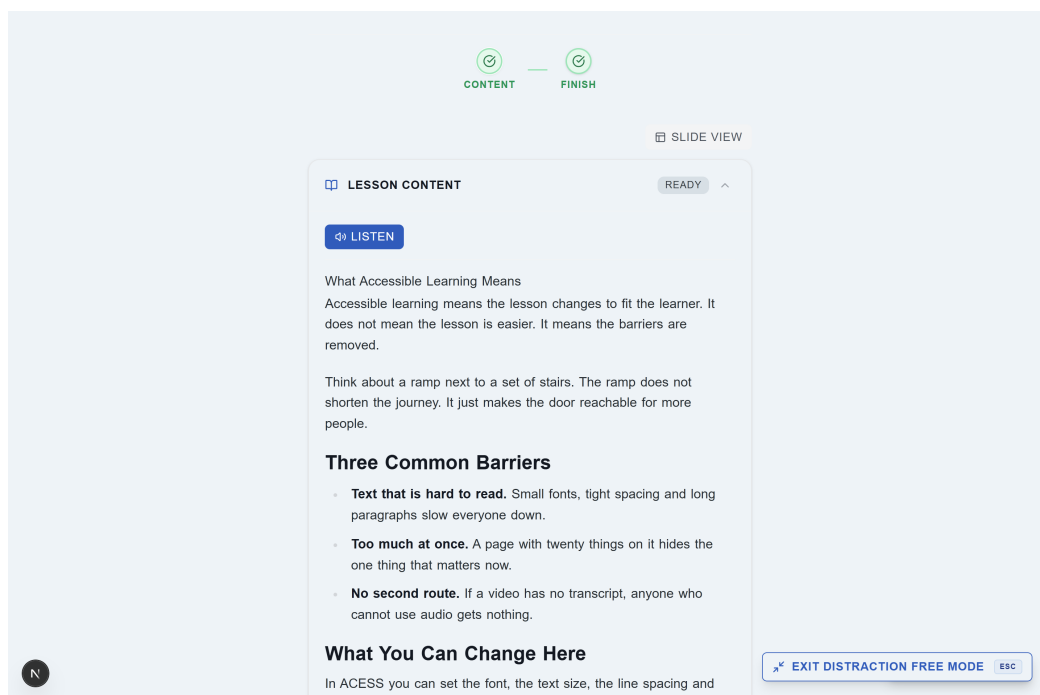


Figure 4.22: Lesson Interface Under the Autism Preset

it they are before they begin. The reading width is deliberately not narrowed here: unlike Dyslexia, where a narrow measure assists decoding, the need addressed by this profile is predictability, and keeping the default width means the page does not change shape when the learner moves between courses.

Figure 4.23 shows the effect at dashboard rather than lesson level, where the Dyslexia preset replaces the multi-column card grid of Figure 4.8 with a single-column list and enlarges the navigation.

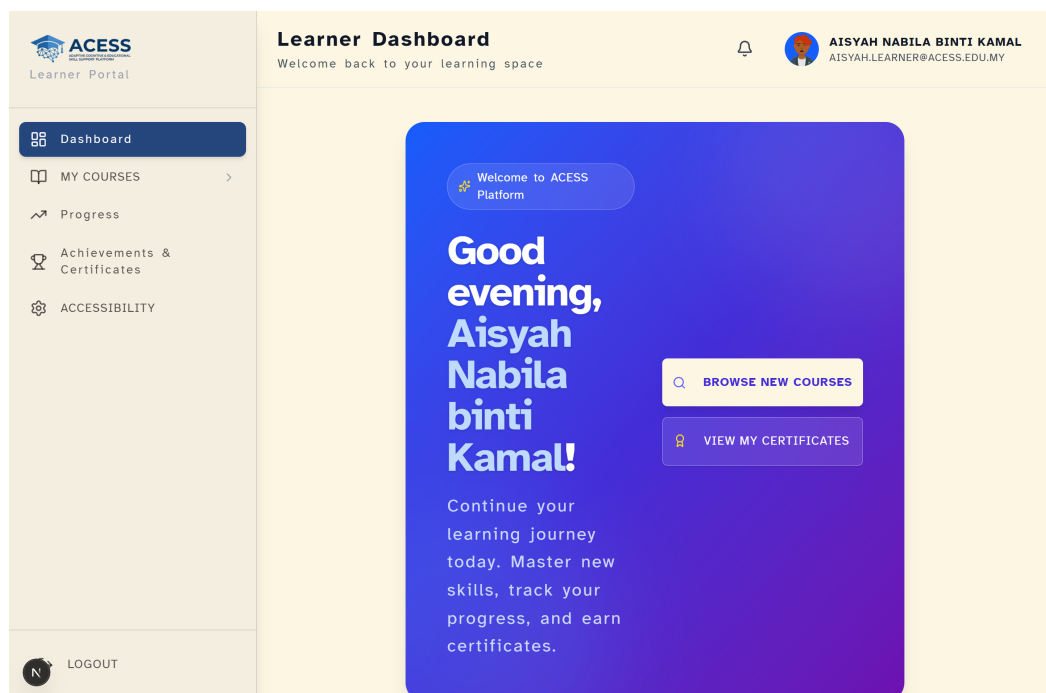


Figure 4.23: Learner Dashboard Under the Dyslexia Preset

4.3.6 Authoring-time accessibility compliance checking

The content criteria identified in Section 4.3 cannot be met by the platform on the educator's behalf. ACESS therefore provides a compliance checker that evaluates a lesson while it is being written.

The checker has three properties that make its output defensible. It is deterministic: every rule is a fixed threshold with a published source, so the same lesson always produces the same score. It has no side effects and runs immediately, which is why the same engine can be used both against the unsaved draft in the editor and against saved lessons for

the course-level report, with the two necessarily agreeing. And it is profile-aware: a baseline of seven rules applies to every lesson, with a further set added according to the accessibility focus declared for the course (six for ADHD, five for Autism and seven for Dyslexia), so an educator writing for learners with Dyslexia is checked on sentence length and readability while one writing for autistic learners is checked on figurative language and stated objectives.

Rules that cannot apply are marked as not applicable rather than failed and are excluded from the score, so a lesson with no video is not penalised for lacking a transcript. The score is the weighted proportion of applicable rules passed, reported as critical below 50, warning below 80, and good at 80 or above.

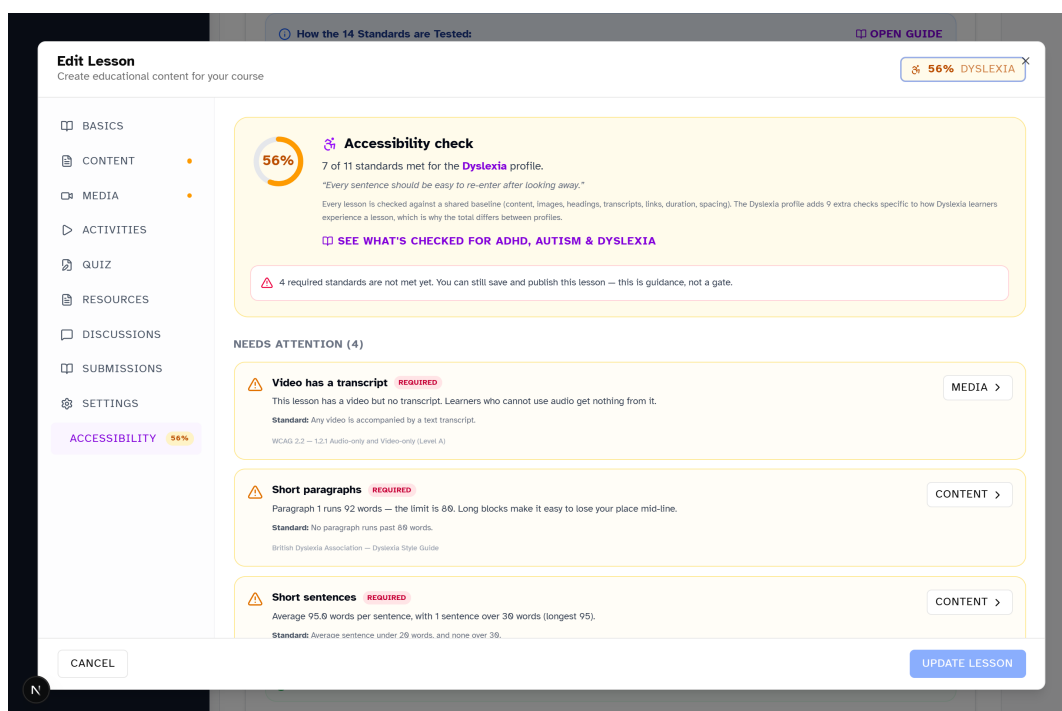


Figure 4.24: Lesson Accessibility Check in the Educator Editor

Figure 4.24 shows the checker reporting on a lesson from the demonstration dataset. Three properties of that output distinguish it from a generic accessibility linter. Each finding states the measured value rather than only that a rule failed, so it reads, for example, that a paragraph runs 92 words against a limit of 80. Each finding names its source, so an educator who disagrees can consult the standard rather than the tool. And each finding provides a route to the correction, opening the editor tab that owns the problem.

The panel also states, in the same view, that unmet standards remain and that the lesson can still be saved and published. This is a deliberate policy choice. A publication gate would encourage educators to satisfy the rule rather than the reader, for example by padding a summary to clear a word count, and would position accessibility as an obstacle to teaching. The checker advises and the educator decides; what the platform guarantees is that the decision is an informed one.

The same engine runs at course level, aggregating the fourteen standards that apply across a whole course and identifying which lessons are responsible for each unmet one, so that an educator can distinguish a single weak lesson from a course-wide problem.

4.3.7 Known accessibility gaps

The following are real limitations of the current build, and each is disclosed on the platform's own accessibility statement as well as here.

1. **No assistive technology has been tested against this build.** No screen reader or other assistive technology has been used with ACCESS. Every claim in Table 4.3 is verified by inspecting the code that implements it and by operating the interface in a browser, not by using it as a screen-reader user would. This is the most significant limitation in this chapter.
2. **No neurodivergent learner has tested the platform.** No learner with lived experience of Dyslexia, ADHD or Autism took part in building or evaluating it. The design draws on published guidance rather than on the people it is intended for, which is a methodological weakness discussed further in Section 4.6.
3. **The muted colours setting is too broad.** It is applied as a single saturation reduction across the whole interface, so it also reduces the distinctness of error, warning and success colours, which is the opposite of the intent. Correcting it requires reworking the colour definitions throughout the stylesheet, which was judged too large a change to attempt without visual verification. The setting's own description states its current behaviour rather than the intended one.

4. **The keyboard navigation setting does not do what its label promises.** It sets a value that nothing currently reads, and keyboard operability should not be optional in any case. The entry is retained in the settings catalogue so that the gap remains visible rather than silently absent.
5. **Only one preset can be active at a time.** A learner who would benefit from Dyslexia typography together with ADHD pacing support must currently choose one and adjust the remainder manually. Combinable presets are the most frequently identified extension to this design.
6. **One drag-and-drop mode has no alternative to dragging.** All other activity types provide a keyboard path and, where relevant, an alternative to dragging. The mode that places labels at arbitrary points on an image does not.
7. **Reading from a chosen paragraph requires a pointer.** Selecting a paragraph during read-aloud restarts speech from that paragraph, but this is not reachable from the keyboard. The Listen control remains the keyboard baseline.
8. **No quantitative accessibility baseline was captured.** No measurement of learner outcomes was taken before the accessibility work began, so no before-and-after comparison is possible. The comparisons in this chapter are demonstrations that the interface adapts as designed, not evidence that the adaptation improves learning outcomes.

4.4 Data Dictionary and Normalisation

4.4.1 Normalisation

The schema is normalised to Third Normal Form with two documented and deliberate departures. Every table has a single-column primary key, so first normal form holds and no partial dependency on part of a composite key is possible, which gives second normal form. Third normal form holds throughout the operational tables: a lesson stores the course it belongs to but not the course's title, a quiz attempt stores the

enrolment but not the learner's name, and a progress record holds no copy of the lesson's duration.

The two departures are as follows.

1. **The certificate record is deliberately denormalised.** It stores the learner name, course title, educator name, institution name and course duration as literal values even though all of them could be obtained by joining to other tables. This is the correct design for this entity, because a certificate is a statement about a moment in time. If the course were later renamed, or the learner changed their display name, a certificate that read those values from elsewhere would silently change and would no longer match the document the learner already holds. Freezing the values at the time of issue is what makes the certificate verifiable years later.
2. **Accessibility and notification preferences are stored as structured documents** on the learner's profile rather than as separate columns. Strictly, this is a repeating group and departs from first normal form. It was chosen because the list of settings changes considerably faster than the rest of the schema, and because every part of the system reads the whole settings object at once rather than querying an individual setting. The cost is that the database cannot constrain an individual setting's value. The platform compensates by defining the permitted settings in a single catalogue against which an automated check runs, so that a setting cannot exist in the data without also existing in that catalogue.

Two further points should be recorded. The schema contains no many-to-many relationship implemented without a junction table; the tables recording earned achievements and saved courses exist precisely to resolve one. In addition, one duplicated column pair was removed during the schema consolidation described in Section 4.5.2, because a lesson previously stored a copy of its chapter's title that could disagree with the chapter record itself.

Reading the tables

The tables that follow constitute the physical schema of the ACCESS database as it currently stands: 36 tables and 364 columns, or 370 including the six columns of the restricted view described in Section 4.5.2. They were generated directly from the live database catalogue rather than transcribed, so the attribute names, types, defaults and constraints in them are those the database actually holds. The Constraint column reports the key role, referential target, uniqueness and nullability of each attribute; value constraints, which apply to the content of a column rather than to its structure, are discussed with the physical design in Section 4.5.2.

4.4.2 Core Identity and Configuration

Tables 4.6 to 4.13 define the 8 entities of the core identity and configuration domain.

1. Table: **users**

Table 4.6: Users Data Dictionary

Attribute	Type	Constraint	Description
id	uuid	PRIMARY KEY	Unique identifier (UUID v4).
email	character varying	UNIQUE, NOT NULL	User email address, used for login and notifications.
role	character varying	NOT NULL	User role: learner, educator, or admin.
is_active	boolean DEFAULT true	NOT NULL	Soft-ban flag, inactive users cannot log in.
email_verified_at	timestamp	-	Timestamp when user confirmed email via verification link, NULL if unverified.
last_login_at	timestamp	-	Timestamp of last successful login.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp DEFAULT now()	NOT NULL	Timestamp of the last modification.
deleted_at	timestamp	-	Soft-delete timestamp, null if not deleted.
full_name	text	-	User's display name shown across the platform.
instructor_application_status	text	-	Status of educator application.

2. Table: **user_profiles**

Table 4.7: User Profiles Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, UNIQUE, NOT NULL	Foreign key referencing users.id.
username	text	UNIQUE	Public-facing username handle.
avatar_url	text	-	URL to the user's profile image.
phone_number	text	-	Contact phone number for account recovery and notifications.
birth_date	date	-	User's date of birth for age verification and age-group analytics.
bio	text	-	Short biographical text for educator profiles.
country	text	-	ISO 3166-1 country code for regional content localisation.
preferred_language	text DEFAULT 'en'	-	Interface language code (en, ms).
created_at	timestampz DEFAULT now()	-	Timestamp when the record was created.
updated_at	timestampz DEFAULT now()	-	Timestamp of the last modification.
disability_type	character varying	-	Disability classification for adaptive engine.
accessibility_prefs	jsonb DEFAULT ''	-	JSON object storing active accessibility settings (font size, colour scheme, read-aloud rate, dyslexia font toggle).
notification_prefs	jsonb DEFAULT ''	-	JSON object storing notification delivery preferences (email, in-app, frequency).

3. Table: referral_codes**Table 4.8: Referral Codes Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
code	text	UNIQUE, NOT NULL	Unique alphanumeric code assigned to a user for referral tracking.
user_id	uuid	FK → users	Foreign key referencing users.id.
usage_count	integer DEFAULT 0	-	Number of times used.
max_uses	integer DEFAULT 50	-	Maximum number of times this referral code can be redeemed.
is_active	boolean DEFAULT true	-	Soft-ban flag, inactive users cannot log in.
created_at	timestampz DEFAULT now()	-	Timestamp when the record was created.

4. Table: instructor_applications

Table 4.9: Instructor Applications Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users	Foreign key referencing users.id.
full_name	text	NOT NULL	User's display name shown across the platform.
email	text	NOT NULL	User email address, used for login and notifications.
experience	text	-	Free-text description of applicant's teaching or subject-matter experience.
reason	text	-	Applicant's motivation statement for wanting to become an educator on the platform.
portfolio_links	text	-	Comma-separated URLs to the applicant's teaching portfolio or work samples.
referral_code	text	-	Unique referral token string.
status	text DEFAULT 'pending'	NOT NULL	Record status (e.g., active, completed, dropped).
admin_notes	text	-	Internal reviewer notes regarding the application decision.
reviewed_by	uuid	FK → users	UUID of the administrator who reviewed the application.
reviewed_at	timestamptz	-	Timestamp of the review decision.
created_at	timestamptz DEFAULT now()	-	Timestamp when the record was created.
updated_at	timestamptz DEFAULT now()	-	Timestamp of the last modification.

5. Table: contact_messages**Table 4.10: Contact Messages Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
name	text	NOT NULL	Display name of the record.
email	text	NOT NULL	User email address, used for login and notifications.
category	text	NOT NULL	Content category classification.
subject	text	NOT NULL	Message subject line.
message	text	NOT NULL	Full body text of the contact form submission.
status	text DEFAULT 'unread'	NOT NULL	Record status (e.g., active, completed, dropped).
created_at	timestamptz DEFAULT now()	-	Timestamp when the record was created.

6. Table: notifications

Table 4.11: Notifications Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, NOT NULL	Foreign key referencing users.id.
type	text	NOT NULL	Classification type of the record.
title	text	NOT NULL	Display title of the record.
body	text	-	Message body text content.
metadata	jsonb DEFAULT ''	-	JSONB field for extensible metadata.
is_read	boolean DEFAULT false	-	Whether the message has been read.
created_at	timestampz DEFAULT now()	-	Timestamp when the record was created.

7. Table: accessibility_templates

Table 4.12: Accessibility Templates Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
name	text	NOT NULL	Display name of the record.
description	text	-	Free-text description or summary.
target_disability	text	NOT NULL	Disability category the template targets (e.g. 'dyslexia', 'adhd', 'asd').
content_structure	jsonb	NOT NULL	JSON definition of template layout rules and component arrangement.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.

8. Table: adaptive_interactions

Table 4.13: Adaptive Interactions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, NOT NULL	Foreign key referencing users.id.
lesson_id	uuid	FK → lessons	Foreign key referencing lessons.id.
course_id	uuid	FK → courses	Foreign key referencing courses.id.
adaptation_used	text	NOT NULL	Identifier of the adaptive preset applied (e.g. 'dyslexia_standard', 'adhd_focus').
session_id	text	-	Unique identifier linking interactions within a single browsing session.
duration_seconds	integer	-	Duration in seconds.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.

4.4.3 Curriculum Structure

Tables 4.14 to 4.22 define the 9 entities of the curriculum structure domain.

9. Table: courses

Table 4.14: Courses Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
created_by	uuid	FK → users, NOT NULL	Foreign key referencing the creating user.
title	character varying	NOT NULL	Display title of the record.
slug	character varying	UNIQUE, NOT NULL	URL-safe identifier string.
description	text	-	Free-text description or summary.
status	character varying DEFAULT 'draft'	NOT NULL	Record status (e.g., active, completed, dropped).
difficulty_level	character varying	-	Course difficulty classification ('beginner', 'intermediate', 'advanced') for learner filtering.
thumbnail_url	character varying	-	URL to the thumbnail image.
published_at	timestamp	-	Timestamp when published.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp DEFAULT now()	NOT NULL	Timestamp of the last modification.
deleted_at	timestamp	-	Soft-delete timestamp, null if not deleted.
category	text	-	Content category classification.
course_type	text DEFAULT 'educator'	NOT NULL	Origin of course creation ('educator' or 'system') determining edit permissions.
system_course	boolean DEFAULT false	NOT NULL	Boolean flag indicating the course was created by administrators, not educators.
built_in_course	boolean DEFAULT false	NOT NULL	Boolean flag marking pre-seeded platform courses that cannot be deleted.
created_by_role	text DEFAULT 'educator'	NOT NULL	Role of the user who created the course ('educator' or 'admin') for permission checks.
guided_learning_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling step-by-step guided lesson progression.
official_course_order	integer	-	Integer display priority for system-created courses in the catalog.
managed_by_admin	boolean DEFAULT false	NOT NULL	Boolean flag indicating administrative oversight of educator-created course.
recommended_age_group	text	-	Suggested learner age range for age-appropriate content filtering.
learning_streaks_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle activating daily learning streak tracking and rewards.
milestone_tracking_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling course milestone progress indicators.
course_layout_type	text DEFAULT 'standard'	NOT NULL	Visual layout template ('standard', 'simplified') for course pages.
chapter_organization_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling hierarchical chapter-based lesson grouping.

continued on the next page

Table 4.14 continued from the previous page

Attribute	Type	Constraint	Description
certificate_enabled	boolean DEFAULT false	-	Boolean toggle allowing certificate generation upon course completion.
certificate_settings	jsonb DEFAULT ''	-	JSON object storing certificate configuration (template, signature, expiry).
certification_locked	boolean DEFAULT false	-	Boolean flag preventing certificate issuance until all criteria are met.
supports_tts	boolean DEFAULT false	-	Boolean flag indicating whether course lessons support text-to-speech playback.
supports_transcripts	boolean DEFAULT false	-	Boolean flag indicating whether video lessons include text transcripts.
supports_focus_mode	boolean DEFAULT false	-	Boolean flag indicating whether Focus Mode is available for this course.
supports_chunked_learning	boolean DEFAULT false	-	Boolean flag enabling content to be broken into smaller learning chunks.
tags	text[] DEFAULT ''	-	JSONB array of tag strings.
accessibility_categories	text[] DEFAULT ''	-	Array of disability categories this course is designed to support.
primary_disability_focus	text	-	Main disability category the course content is optimised for.
secondary_disability_focuses	text[] DEFAULT ''	-	Array of additional disability categories the course partially supports.
target_reading_age	integer DEFAULT 13	-	Minimum reading level the course content is written at for age-appropriate filtering.
educator_custom_guide	text	-	Free-text educator instructions for how to use or navigate this course.

10. Table: course_chapters

Table 4.15: Course Chapters Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, UNIQUE, NOT NULL	Foreign key referencing courses.id.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
sequence_order	integer DEFAULT 0	UNIQUE, NOT NULL	Integer determining the display order of this chapter within the course.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.

11. Table: lessons

Table 4.16: Lessons Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, UNIQUE, NOT NULL	Foreign key referencing courses.id.
title	character varying	NOT NULL	Display title of the record.
content_html	text	-	Rich-text HTML body of the lesson rendered via Tiptap editor.
video_url	character varying	-	URL of the embedded video asset for this lesson.
transcript	text	-	Full text transcript of the lesson content for accessibility compliance.
sequence_order	integer	UNIQUE, NOT NULL	Integer determining the display order of this lesson within its course.
status	character varying DEFAULT 'draft'	NOT NULL	Record status (e.g., active, completed, dropped).
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp DEFAULT now()	NOT NULL	Timestamp of the last modification.
estimated_ duration	integer	-	Approximate lesson completion time in minutes for progress estimation.
prerequisite_ lesson_id	uuid	FK → lessons	UUID of the lesson that must be completed before this one unlocks.
simplified_ summary	text	-	Plain-text summary of lesson content for simplified reading mode.
accessibility_ notes	text	-	Accessibility improvement notes.
focus_mode_ enabled	boolean DEFAULT false	NOT NULL	Boolean flag allowing Focus Mode override for this specific lesson.
chunked_ content_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling lesson content to be split into digestible segments.
lesson_type	text DEFAULT 'standard'	NOT NULL	Lesson classification ('standard', 'video', 'interactive') determining content rendering.
chapter_id	uuid	-	Foreign key referencing course.chapters.id.
learning_ objectives	text	-	Semicolon-separated list of learning outcomes displayed before lesson content.
visibility_ status	text DEFAULT 'visible'	NOT NULL	Content visibility state ('visible', 'hidden', 'draft') controlling catalog display.
checkpoints_ enabled	boolean DEFAULT false	NOT NULL	Boolean toggle requiring milestone checkmarks before lesson progression.
adaptive_ learning_ enabled	boolean DEFAULT false	NOT NULL	Boolean flag activating adaptive content delivery for this lesson.
has_video	boolean DEFAULT true	-	Boolean flag indicating whether this lesson contains an embedded video.
has_pdf	boolean DEFAULT true	-	Boolean flag indicating whether this lesson has a downloadable PDF resource.
has_quiz	boolean DEFAULT true	-	Boolean flag indicating whether this lesson has an associated quiz.
has_transcript	boolean DEFAULT true	-	Boolean flag indicating whether this lesson includes a text transcript.
has_summary_ activity	boolean DEFAULT false	-	Boolean flag enabling the learner summary submission activity.
summary_source	text DEFAULT 'entire_lesson'	-	Content scope for summary generation ('entire_lesson', 'current_section').

continued on the next page

Table 4.16 continued from the previous page

Attribute	Type	Constraint	Description
summary_word_target	integer DEFAULT 100	-	Target word count for learner summary submissions.
summary_key_points	jsonb DEFAULT '[]'	-	JSON array of expected key points for summary evaluation.
summary_reflection_questions	jsonb DEFAULT '[]'	-	JSON array of reflection prompts shown after summary submission.
summary_ai_feedback_enabled	boolean DEFAULT false	-	Boolean toggle enabling AI-generated feedback on summaries.
lesson_layout	text DEFAULT 'standard'	-	Visual template ('standard', 'focus', 'minimal') for lesson page rendering.
has_h5p	boolean DEFAULT false	-	Flag indicating that the lesson carries at least one embedded H5P interactive object.
accessibility_score	integer DEFAULT 100	-	Accessibility compliance score.
allow_discussions	boolean DEFAULT false	-	Boolean toggle enabling threaded comments on this lesson.
allow_download	boolean DEFAULT false	-	Boolean toggle enabling resource file downloads for this lesson.

12. Table: course_accessibility_categories

Table 4.17: Course Accessibility Categories Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, NOT NULL	Foreign key referencing courses.id.
accessibility_category	text	NOT NULL	Disability category label (e.g. 'dyslexia', 'adhd', 'asd') linked to this course.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.

13. Table: course_milestones

Table 4.18: Course Milestones Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, NOT NULL	Foreign key referencing courses.id.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
required_completion_pct	integer DEFAULT 100	NOT NULL	Minimum percentage of lessons that must be completed to achieve this milestone.
icon	text	-	Emoji or icon identifier displayed alongside the milestone label.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this milestone in the progress tracker.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.

14. Table: course_achievements**Table 4.19: Course Achievements Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, NOT NULL	Foreign key referencing courses.id.
name	character varying	NOT NULL	Display name of the record.
description	text	NOT NULL	Free-text description or summary.
icon_url	character varying	-	URL of the achievement badge image displayed in the learner's profile.
requirement_type	character varying	NOT NULL	Achievement criterion type (e.g. 'quiz_score', 'course_complete', 'streak').
requirement_threshold	integer DEFAULT 1	NOT NULL	Numeric threshold value that must be met to unlock this achievement.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestampz DEFAULT now()	NOT NULL	Timestamp of the last modification.

15. Table: lesson_templates

Table 4.20: Lesson Templates Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
created_by	uuid	FK → users, NOT NULL	Foreign key referencing the creating user.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
lesson_type	text DEFAULT 'standard'	NOT NULL	Template classification ('standard', 'video', 'interactive') for the template.
content_html	text	-	Default HTML content body pre-filled in the template for educator customisation.
estimated_duration	integer	-	Default approximate completion time in minutes suggested by the template.
is_public	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this template is available to all educators.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestampz DEFAULT now()	NOT NULL	Timestamp of the last modification.

16. Table: lesson_versions**Table 4.21: Lesson Versions Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
content_html	text	NOT NULL	Snapshot of the lesson HTML content at the time this version was saved.
version_name	text	NOT NULL	Human-readable label identifying this version (e.g. 'v1', 'draft-2024-03').
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.
created_by	uuid	FK → users, NOT NULL	Foreign key referencing the creating user.

17. Table: media_assets

Table 4.22: Media Assets Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, NOT NULL	Foreign key referencing users.id.
course_id	uuid	FK → courses	Foreign key referencing courses.id.
file_name	character varying	NOT NULL	Original uploaded file name including extension (e.g. 'lecture_slides.pdf').
file_type	character varying	NOT NULL	MIME type of the file.
url	character varying	NOT NULL	External resource URL.
size_bytes	bigint	-	File size in bytes for storage quota enforcement.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.
lesson_id	uuid	FK → lessons	Foreign key referencing lessons.id.
title	character varying	-	Display title of the record.

4.4.4 Lesson Content and Interaction

Tables 4.23 to 4.28 define the 6 entities of the lesson content and interaction domain.

18. Table: lesson_interactive_content

Table 4.23: Lesson Interactive Content Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
content_type	text	NOT NULL	MIME type of the content.
title	text	NOT NULL	Display title of the record.
content_data	jsonb DEFAULT ''	NOT NULL	JSONB payload containing the H5P interactive activity configuration and state.
accessibility_settings	jsonb DEFAULT ''	-	JSON object storing accessibility overrides specific to this interactive activity.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this activity within the lesson.
created_by	uuid	FK → users	Foreign key referencing the creating user.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestampz DEFAULT now()	NOT NULL	Timestamp of the last modification.
is_draft	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this interactive content is unpublished.

19. Table: video_questions

Table 4.24: Video Questions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
title	text	NOT NULL	Display title of the record.
timestamp_ seconds	numeric	NOT NULL	Video playback position in seconds where the question is displayed.
question_text	text	NOT NULL	Display text of the question.
options	jsonb DEFAULT '[]'	NOT NULL	JSONB array of answer options.
correct_option_ index	integer DEFAULT 0	NOT NULL	Zero-based index of the correct answer in the options array.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this question within the video.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestampz DEFAULT now()	NOT NULL	Timestamp of the last modification.

20. Table: h5p_contents

Table 4.25: H5P Contents Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing the lesson the interactive object is embedded in.
title	text	NOT NULL	Display title shown above the embedded object.
embed_url	text	NOT NULL	URL of the externally hosted H5P object, rendered inside a sandboxed frame.
source_url	text	-	Canonical page for the object on its host, retained for attribution and re-editing.
description	text	-	Optional explanatory text shown to the learner before the object loads.
width	text DEFAULT '100%'	-	CSS width applied to the embed frame, defaults to full container width.
height	text DEFAULT '500px'	-	CSS height applied to the embed frame.
sequence_order	integer DEFAULT 0	NOT NULL	Position of this object among the lesson's embedded objects.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestampz DEFAULT now()	NOT NULL	Timestamp of the last modification.
thumbnail_url	text	-	Optional preview image shown before the object is loaded.

21. Table: lesson_checkpoints

Table 4.26: Lesson Checkpoints Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
checkpoint_type	text DEFAULT 'reflection'	NOT NULL	Checkpoint category ('reflection', 'quiz', 'activity') determining interaction format.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this checkpoint within the lesson.
required	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this checkpoint must be passed to proceed.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.

22. Table: lesson_ai_summaries**Table 4.27: Lesson AI Summaries Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, UNIQUE, NOT NULL	Foreign key referencing the lesson the summary describes.
summary	text	NOT NULL	Cached plain-language summary of the lesson, generated on first request and reused thereafter.
suggested_questions	text[] DEFAULT ''	-	Generated prompts offered to the learner as starting points for the lesson assistant.
source_content_hash	text	NOT NULL	Digest of the lesson content the summary was generated from, so a change to the lesson invalidates the cache rather than serving a stale summary.
model	text	NOT NULL	Identifier of the generative model that produced the summary, recorded so that output can be attributed and regenerated deliberately.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the summary was first generated.
updated_at	timestampz DEFAULT now()	NOT NULL	Timestamp of the last regeneration.

23. Table: lesson_comments

Table 4.28: Lesson Comments Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
user_id	uuid	FK → users, NOT NULL	Foreign key referencing users.id.
content	text	NOT NULL	Rich-text HTML or JSON content body.
parent_id	uuid	FK → lesson_comments	UUID of the parent comment for threaded reply nesting, NULL for top-level comments.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestampz DEFAULT now()	NOT NULL	Timestamp of the last modification.

4.4.5 Assessment

Tables 4.29 to 4.33 define the 5 entities of the assessment domain.

24. Table: quizzes

Table 4.29: Quizzes Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, UNIQUE, NOT NULL	Foreign key referencing lessons.id.
title	character varying	NOT NULL	Display title of the record.
time_limit_seconds	integer DEFAULT 0	NOT NULL	Maximum allowed quiz completion time in seconds, 0 meaning unlimited.
max_attempts	integer DEFAULT 0	NOT NULL	Maximum number of allowed attempts.
pass_threshold_pct	integer DEFAULT 60	NOT NULL	Minimum percentage required to pass.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp DEFAULT now()	NOT NULL	Timestamp of the last modification.

25. Table: quiz_questions

Table 4.30: Quiz Questions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
quiz_id	uuid	FK → quizzes, UNIQUE, NOT NULL	Foreign key referencing quizzes.id.
question_text	text	NOT NULL	Display text of the question.
question_type	character varying	NOT NULL	Question format (multiple-choice, etc.).
sequence_order	integer	UNIQUE, NOT NULL	Integer determining the display order of this question within the quiz.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
image_url	text	-	URL of an optional image asset displayed alongside the question text.

26. Table: quiz_options**Table 4.31: Quiz Options Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
question_id	uuid	FK → quiz_questions, UNIQUE, NOT NULL	Foreign key referencing quiz_questions.id.
option_text	text	NOT NULL	Display text of the quiz option.
is_correct	boolean DEFAULT false	NOT NULL	Whether this is the correct answer.
sequence_order	integer	UNIQUE, NOT NULL	Integer determining the display order of this answer option.
image_url	text	-	URL of an optional image asset displayed as this answer option.

27. Table: quiz_attempts**Table 4.32: Quiz Attempts Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, UNIQUE, NOT NULL	Foreign key referencing enrollments.id.
quiz_id	uuid	FK → quizzes, UNIQUE, NOT NULL	Foreign key referencing quizzes.id.
attempt_number	integer	UNIQUE, NOT NULL	Sequential counter for quiz attempts.
score_pct	integer	-	Quiz score as a percentage (0-100).
result	character varying DEFAULT 'in_progress'	NOT NULL	Outcome of the quiz attempt ('in_progress', 'passed', 'failed').
started_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the quiz attempt was initiated.
submitted_at	timestamp	-	Timestamp of response submission.

28. Table: quiz_answers

Table 4.33: Quiz Answers Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
attempt_id	uuid	FK → quiz_attempts, UNIQUE, NOT NULL	Foreign key referencing quiz_attempts.id.
question_id	uuid	FK → quiz_questions, UNIQUE, NOT NULL	Foreign key referencing quiz_questions.id.
selected_ option_id	uuid	FK → quiz_options	Foreign key referencing the chosen quiz option.
is_correct	boolean	-	Whether this is the correct answer.

4.4.6 Learner Progress, Recognition and Recommendation

Tables 4.34 to 4.41 define the 8 entities of the learner progress, recognition and recommendation domain.

29. Table: enrollments

Table 4.34: Enrollments Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, UNIQUE, NOT NULL	Foreign key referencing users.id.
course_id	uuid	FK → courses, UNIQUE, NOT NULL	Foreign key referencing courses.id.
enrolled_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the enrollment was created.
completed_at	timestamp	-	Timestamp when the record was completed.
status	character varying DEFAULT 'active'	NOT NULL	Record status (e.g., active, completed, dropped).

30. Table: lesson_progress

Table 4.35: Lesson Progress Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, UNIQUE, NOT NULL	Foreign key referencing enrollments.id.
lesson_id	uuid	FK → lessons, UNIQUE, NOT NULL	Foreign key referencing lessons.id.
is_viewed	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has opened this lesson.
view_count	integer DEFAULT 0	NOT NULL	Total number of times the learner has viewed this lesson.
first_viewed_at	timestamp	-	Timestamp of the learner's first access to this lesson.
last_viewed_at	timestamp	-	Timestamp of the learner's most recent access to this lesson.
time_spent_learning	integer DEFAULT 0	NOT NULL	Cumulative time in seconds the learner has spent on this lesson.
summary_completed	boolean DEFAULT false	-	Boolean flag indicating whether the learner submitted a lesson summary.
is_completed	boolean DEFAULT false	-	Boolean flag indicating whether the learner has fully completed this lesson.
progress_meta	jsonb DEFAULT ''	NOT NULL	JSONB object storing supplementary progress metrics and adaptive engine data.

31. Table: learner_checkpoints**Table 4.36: Learner Checkpoints Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, UNIQUE, NOT NULL	Foreign key referencing enrollments.id.
checkpoint_id	uuid	FK → lesson_ checkpoints, UNIQUE	Foreign key referencing lesson_checkpoints.id.
completed	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has completed this checkpoint.
completed_at	timestampz	-	Timestamp when the record was completed.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the record was created.
response_data	jsonb	-	JSONB payload of the learner's response.
lesson_id	uuid	FK → lessons	Foreign key referencing lessons.id.

32. Table: h5p_responses

Table 4.37: H5P Responses Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, NOT NULL	Foreign key referencing the learner who produced the response.
h5p_content_id	uuid	FK → h5p_contents, NOT NULL	Foreign key referencing the embedded object the response belongs to.
score	integer	-	Score reported by the object, where the object reports one.
max_score	integer	-	Maximum attainable score reported by the object.
completed	boolean DEFAULT false	NOT NULL	Whether the object reported the interaction as completed.
raw_statement	jsonb	-	The unmodified statement emitted by the object, retained so that the record can be re-interpreted without replaying the interaction.
created_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the response was recorded.

33. Table: recommendations**Table 4.38: Recommendations Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, NOT NULL	Foreign key referencing enrollments.id.
recommended_lesson_id	uuid	FK → lessons, NOT NULL	UUID of the lesson recommended to the learner by the adaptive engine.
difficulty_tier	character varying	NOT NULL	Difficulty classification of the recommended lesson relative to learner performance.
trigger_reason	text	-	Explanation of why the adaptive engine generated this recommendation.
is_acknowledged	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has viewed this recommendation.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.

34. Table: course_favorites

Table 4.39: Course Favorites Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, UNIQUE, NOT NULL	Foreign key referencing users.id.
course_id	uuid	FK → courses, UNIQUE, NOT NULL	Foreign key referencing courses.id.
created_at	timestampz DEFAULT now()	-	Timestamp when the record was created.

35. Table: user_achievements**Table 4.40: User Achievements Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, UNIQUE, NOT NULL	Foreign key referencing users.id.
achievement_id	uuid	FK → course_ achievements, UNIQUE, NOT NULL	Foreign key referencing course_achievements.id.
course_id	uuid	FK → courses, NOT NULL	Foreign key referencing courses.id.
earned_at	timestampz DEFAULT now()	NOT NULL	Timestamp when the achievement was earned.

36. Table: certificates**Table 4.41: Certificates Data Dictionary**

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, UNIQUE, NOT NULL	Foreign key referencing enrollments.id.
reference_code	character varying	UNIQUE, NOT NULL	Unique alphanumeric identifier for external certificate verification.
pdf_url	character varying	-	URL to the generated PDF certificate file stored in cloud storage.
status	character varying DEFAULT 'issued'	NOT NULL	Record status (e.g., active, completed, dropped).
issued_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the certificate was issued.
revoked_at	timestamp	-	Timestamp when the certificate was revoked, NULL if still valid.
revoke_reason	text	-	Free-text explanation for why the certificate was revoked.
course_id	uuid	FK → courses	Foreign key referencing courses.id.
user_id	uuid	FK → users	Foreign key referencing users.id.

continued on the next page

Table 4.41 continued from the previous page

Attribute	Type	Constraint	Description
learner_name	text	-	Full name of the learner as displayed on the certificate.
course_title	text	-	Title of the completed course as displayed on the certificate.
educator_name	text	-	Name of the course educator as displayed on the certificate.
institution_name	text DEFAULT 'ACCESS Platform'	-	Name of the issuing institution displayed on the certificate.
completion_date	timestampz	-	Timestamp when the learner completed all course requirements.
verification_url	text	-	Public URL for third-party verification of certificate authenticity.
skills_earned	text[] DEFAULT ''	-	Array of skill labels or competencies acquired through course completion.
course_duration_hours	numeric DEFAULT 0	-	Total duration of the course in hours as displayed on the certificate.
signed_token	text	-	Cryptographic signature token for tamper-proof certificate verification.
template_id	text DEFAULT 'default'	-	Identifier of the certificate template used for this certificate.
metadata	jsonb DEFAULT ''	-	JSONB field for extensible metadata.

4.5 Detailed Design

4.5.1 Software Design

ACCESS is a component-based web application organised by feature area. It consists of three global providers that hold state shared across the whole application, three role-scoped data access layers, a set of self-contained computation modules, and a small number of server routes reserved for operations that must not run in the browser.

Global providers

- **Authentication provider.** Wraps the application, establishes the connection to the backend, and exposes the current session together with the sign-in, sign-up and sign-out operations. It also drives the session timeout component, which watches for pointer and keyboard activity, warns thirty seconds before the fifteen-minute inactivity limit is reached, and signs the user out if the warning is not answered.
- **Accessibility provider.** Loads the learner's stored settings, normalises any

contradictory or outdated values, and applies the result to the whole interface. Continuous values such as text size and spacing are applied as style variables, and categorical values such as the active preset, background tint, layout mode and motion level are applied as attributes that the stylesheet reads. Because the stylesheet consumes these values directly, changing a setting repaints the page without rebuilding the component tree, which is what makes the response immediate.

- **Language provider.** Loads the learner’s preferred language and provides the interface text, switching between English and Bahasa Melayu at runtime.

One design point in the accessibility provider is worth recording, because getting it wrong caused a real defect. The provider distinguishes the preset a configuration is derived from, which does not change, from the preset that is currently active, which becomes “custom” as soon as the learner adjusts any single value. Two parts of the system previously disagreed about which of these represented the current preset, with the result that adjusting one unrelated setting silently discarded the preset’s content-chunking and pacing behaviour while retaining its colours. Both now read the same value.

Self-contained computation modules Four modules are deliberately free of any interaction with the database or the network, which makes them independently testable and allows the same logic to serve more than one part of the interface:

- the accessibility compliance checker described in Section 4.3.6, which takes a description of a lesson and an accessibility focus and returns a score, the rule list with pass, fail or not-applicable status, and the failures grouped by the editor tab that owns them;
- the settings resolver, which normalises a settings object and reports conflicts between settings;
- the adaptive engine, which holds the preset definitions given in Table 4.5, computes the difference between the current settings and a candidate preset (which is what

Figure 4.19 displays), and derives the lesson presentation modes from the active preset;

- the profile guidance module, which provides the educator-facing explanation of each accessibility profile, reading its numeric thresholds from the compliance checker's own definitions so that the guidance can never state a different figure from the one the checker enforces.

Data access layer Three role-scoped modules handle all database access from the browser, each following the same pattern of identifying the current user, issuing the query under that user's own session and therefore under row-level security, and reporting any error. The learner layer provides 56 operations covering the profile, catalogue, enrolment, lesson content and progress, quiz attempts, checkpoints, certificates, recommendations, saved courses and accessibility settings. The educator layer provides 73 operations covering course and lesson management with versioning, quiz and activity authoring, asset upload, learner progress, analytics and certificate issue. The administrator layer provides 34 operations covering user management, official courses, chapters, milestones and templates, application review and platform metrics.

Server routes reserved for privileged operations Most changes to data are issued directly from the browser under row-level security. Four operations cannot be, because they require privileges the calling user does not have, and are therefore implemented as server routes:

- **Role and status changes.** The route updates both the user record and the authentication account, because route protection reads the role from the session while database policies read it from the user record. Updating only one of the two was a real defect that left newly promoted educators unable to reach their own dashboard.
- **Educator application approval**, which performs the same paired update when an applicant is promoted.

- **Recommendation generation**, which must read across courses the requesting learner is not enrolled in and therefore cannot run under that learner's own access policies.
- **Certificate issue**, which checks eligibility, generates the reference code and verification address, and guarantees that the code is unique.

The recommendation engine Algorithm 1 sets out the rule-based logic of the recommendation engine. The engine produces four kinds of recommendation, each recorded with a plain-language reason, so that a learner is told why something is being suggested rather than being given an unexplained list.

Algorithm 1 Generate recommendations for a learner

Require: A learner identifier

Ensure: A replacement set of recommendations for that learner

```

1:  $E \leftarrow$  the learner's enrolments that have not been dropped
2: if  $E = \emptyset$  then
3:   return
4: end if
5:  $C \leftarrow$  the courses of  $E$  with their lessons;  $P \leftarrow$  the completed lessons of each enrolment
6:  $F \leftarrow$  the lowest failing quiz score per lesson, per enrolment
   Rules applied within each enrolment
7: for all  $e \in E$  do
8:   for all lesson  $l$  with a failing score in  $F[e]$  do
9:     record a REVISION recommendation for  $l$ , giving the score as the reason
10:  end for
11:   $n \leftarrow$  the first lesson of  $e$  in order that has not been completed
12:  if  $n$  exists and has no failing score then
13:    record a STANDARD recommendation for  $n$  as the next lesson
14:  end if
15:  if  $80 \leq \text{progress}(e) < 100$  then
16:    record an ADVANCED recommendation to reinforce the final lesson
17:  end if
18: end for
   Interest profile: completion counts more than enrolment, and saved courses also count
19:  $W \leftarrow \emptyset$   $\triangleright$  weighted tag, category and difficulty counts
20: for all  $c \in C$  do
21:   add  $c$  to  $W$  with weight 3 if completed, otherwise weight 1
22: end for
23: for all saved course  $c$  do
24:   add  $c$  to  $W$  with weight 2
25: end for
   Rule applied across courses
26: for all published course  $c$  the learner is not enrolled in do
27:    $s \leftarrow$  tag overlap + category overlap + 5 if saved + 1 if the difficulty matches
28: end for
29: record an ADVANCED recommendation for the first lesson of the two highest-scoring courses
   Replace the previous set in one step
30: delete the previous recommendations for  $E$  and insert the new set

```

Two decisions in Algorithm 1 are worth noting. Completion is weighted three times as heavily as enrolment, and a saved course five points above tag overlap, because finishing a course or deliberately saving one is a stronger statement of interest than merely starting one. The near-completion rule is also bounded strictly below 100%: without that

bound it fired on finished courses and told learners to revise the last lesson of a course they had already completed.

Server-side assessment Quiz grading is the one operation in the system where an implementation in the browser would be indefensible. The browser calls a trusted database function that verifies the caller is authenticated, verifies an active enrolment in the course that owns the quiz, determines the attempt number and refuses the attempt if the quiz's attempt limit has been reached, grades each answer against the stored correct option, records the attempt and its answers, and returns the score, the pass decision and the threshold used.

The answer key is protected on the way in as well as on the way out. The learner interface never reads the answer options table directly; it reads a restricted view that withholds which option is correct unless the reader is a member of staff or has already submitted an attempt for that quiz. The browser therefore cannot obtain the answers before submitting, even by querying the data interface directly.

Course completion is protected in the same way. A database function determines whether a course is complete and awards the achievements the learner has earned, and a guard rejects any attempt by a client to mark a course complete directly. Both grading and completion set a transaction-level marker that the guards recognise, which is how a trusted function is permitted to write records that an ordinary session may not.

4.5.2 Physical Database Design

The database is PostgreSQL hosted as a managed service, with an identical stack run locally in containers for development. The complete schema, comprising tables, the restricted view, functions, triggers, policies, indexes and permissions, is defined by version-controlled schema definitions, so a working database can be rebuilt from the project repository with a single command. That property is what makes the schema figures in this chapter verifiable rather than merely asserted.

Schema consolidation The physical schema presented here is the result of a consolidation carried out during the project. The schema definition history had reached 44 files that could not, together, rebuild the database: none of them created the core tables, and every one assumed those tables already existed. The core schema existed only in a separate database dump, so a clean rebuild produced a broken database, and the history had drifted from the live schema to the point where one definition removed a table that was still present.

The history was consolidated into a single verified baseline, and six tables that no part of the system referenced were removed. Table 4.42 records them, with the reason in each case.

Table 4.42: Tables Removed During Schema Consolidation

Table removed	Reason
User accessibility preferences	Superseded by the accessibility settings held on the learner's profile, which is what the application actually reads and writes. The table had no reader and no writer.
Password reset tokens	Password recovery is handled by the authentication service's own token generation, so no separate token table takes part.
Certificate verifications	Verification resolves a certificate by its reference code and records no event.
Learner milestones	No reader or writer. Learner-facing milestones are derived from the course milestone definitions together with live progress.
Lesson summaries	Superseded design. The lesson summary activity is now stored with the learner's other checkpoint responses.
Certificate templates	Its only query filtered on a column the table did not have and could therefore never match. Certificate layout is generated in code.

Eleven further columns were removed from the remaining tables on the same basis, including a lesson's duplicate copy of its chapter title and a reference that pointed at the wrong table. Nothing was removed merely because the interface did not use it: every object was first confirmed to be unreferenced across the application code, the server routes, the database functions, the access policies, the index definitions, the incoming foreign keys and the view.

Representative table definitions Two table definitions are reproduced below from the baseline schema, as required by the writing guide’s treatment of physical database design. Two details are deliberate and are worth explaining. The account identifier has no default value, because the record is created by a trigger on the authentication schema and inherits the identifier from it, so the two can never diverge. The role column has no default either, so that an account can never be created without an explicit role decision.

```
CREATE TABLE public.users (
  id uuid NOT NULL,
  email character varying NOT NULL,
  role character varying NOT NULL,
  is_active boolean DEFAULT true NOT NULL,
  email_verified_at timestamp,
  last_login_at timestamp,
  created_at timestamp DEFAULT now() NOT NULL,
  updated_at timestamp DEFAULT now() NOT NULL,
  deleted_at timestamp,
  full_name text,
  instructor_application_status text,
  CONSTRAINT users_role_check
    CHECK (role IN ('learner', 'educator', 'admin'))
);
```

```
CREATE TABLE public.enrollments (
  id uuid DEFAULT gen_random_uuid() NOT NULL,
  user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  course_id uuid NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  enrolled_at timestamp DEFAULT now() NOT NULL,
  completed_at timestamp,
  status character varying DEFAULT 'active' NOT NULL,
  CONSTRAINT enrollments_status_check
    CHECK (status IN ('active', 'completed', 'dropped')),
  UNIQUE (user_id, course_id)
);
```

Constraints The schema carries 33 value constraints and 20 uniqueness constraints, which fall into three groups. Enumerations restrict a text column to a permitted set of

values, such as course status, lesson layout, lesson type, checkpoint type, difficulty level, contact message category and question type. Range constraints restrict a numeric column, such as a pass threshold and a milestone percentage to between 0 and 100, an attempt number and a sequence order to above zero, and a view count to zero or more. Consistency constraints bind two columns together and are the most significant group: a progress record cannot be marked complete without also being marked viewed, and a certificate record must carry a revocation date if it is revoked and must not if it is issued, so an inconsistent certificate state cannot be written by any client.

Indexes The schema defines 115 indexes. Beyond the primary keys, they follow three patterns. Every foreign key used when reading data is indexed, so that a learner's enrolments, a lesson's progress records and a course's lessons can each be retrieved without scanning the whole table. Status and category columns used to filter lists are indexed, such as course status, certificate status and contact message status. Finally, the most frequently executed query in the platform, which lists the published courses in the public catalogue, is served by an index restricted to exactly those rows, which keeps the index small as the number of draft and archived courses grows.

Row-level security Row-level security is enabled on all 36 tables, carrying 88 policies, and the arrangement is confirmed by an executable check rather than asserted. The policies express four rules. A learner reaches their own data through the enrolment, so the policy on progress records tests whether the enrolment the record belongs to is the caller's own. An educator reaches their own courses and the progress recorded within them, but not another educator's unpublished work. Visitors who are not signed in reach published courses and nothing else. And the answer key is readable only after an attempt has been submitted. The example below shows the first and third of these.

```
-- A learner reaches progress only through their own enrolment
CREATE POLICY "Users can view their own progress"
ON public.lesson_progress FOR SELECT USING (
  EXISTS (SELECT 1 FROM enrollments e
    WHERE e.id = lesson_progress.enrollment_id
```

```

        AND e.user_id = auth.uid()));

-- The answer key is readable only after an attempt is submitted
CREATE POLICY "Staff and post-attempt learners can view quiz options"
ON public.quiz_options FOR SELECT
USING (may_see_quiz_answer_key(question_id));

```

These policies read the role from the stored user record rather than from the session, which is the point made in Chapter 3: route protection trusts the session for speed, and the database does not trust it at all.

Triggers Fifteen triggers are defined in the application schema, together with one on the authentication schema. They serve three purposes. Provisioning: a trigger creates the matching user and profile records when an account is created, fixing self-registered accounts to the learner role. Previously the role was taken from the sign-up request itself, which meant that a caller could create an administrator account simply by asking for one. Guards: the three guards described above. Derivation and notification: five triggers raise notifications on enrolment, lesson progress, quiz attempts, lesson creation and course publication, and the remainder maintain the last-modified timestamps. Placing notification generation in the database rather than in application code means a notification is raised whichever path caused the change.

Storage Three storage areas are defined: one for lesson media and course thumbnails, one for educator-supplied certificate designs, and one for profile pictures. All three are readable by address, because the application renders those addresses directly in the page. Write access is controlled by policy, and the profile picture policy restricts each user to a folder named for their own account, so that one learner cannot overwrite another's picture.

4.5.3 Other Implementation Decisions

In-video questions Video is embedded from an external host and controlled through that host's player interface (Google LLC, 2025). The lesson compares the player's current

position against the timestamps stored for the lesson’s in-video questions, and on reaching one it pauses playback and presents the question over the player. Because an embed can fail for reasons outside the platform’s control, such as a blocked host or an unavailable video, an alternative route to completing the lesson exists. Relying only on the player’s own completion signal previously made a lesson permanently uncompletable when the embed would not play, which is a total barrier rather than an inconvenience.

Notification generation Notifications are generated inside the database. When a quiz attempt is recorded, for example, a trigger identifies the educator who owns the course and creates the notification. The application never assembles these messages itself, so the behaviour cannot differ between the paths that record an attempt.

Session timeout The session timeout component tracks pointer and keyboard activity on authenticated pages. After fourteen minutes and thirty seconds of inactivity it presents a warning with a thirty-second countdown, and if the warning is not answered it signs the user out and returns to the login page. The warning is not decoration: an unannounced timeout causes work to be lost, which is the situation WCAG 2.2.6 exists to address.

Read-aloud Read-aloud uses the browser’s own speech synthesis rather than a network service (Mozilla Developer Network, 2025), so lesson text is not transmitted in order to be spoken and the feature carries no per-request cost. It never starts on its own, and it stops when the learner navigates away, which was a defect found by operating the platform rather than by reading the code. While speech is in progress, selecting a later paragraph restarts it from that paragraph.

4.6 Limitations of Design

The boundaries of this design are set out below. The accessibility-specific gaps are listed separately in Section 4.3.7 and are not repeated here.

- **No evaluation with the intended users.** The platform has not been tested by any learner with lived experience of Dyslexia, ADHD or Autism, and no assistive technology has been used with it. Everything in this chapter is verified against published guidance, against the code, and by operating the interface. This bounds what can be claimed: the design demonstrably adapts, but it has not been shown to help.
- **No quantitative baseline exists.** Because no measurement was taken before the accessibility work began, no before-and-after comparison of learner outcomes is possible even retrospectively. The comparisons in Section 4.3 demonstrate behaviour rather than effect.
- **The recommendation engine reads outcomes rather than behaviour.** It works from quiz results, completion, saved courses and course metadata. It does not use time spent on a lesson, revisit patterns or the other engagement data the platform already collects, although that data would support it. The engine is rule-based by design, because the reason for each recommendation is stated to the learner, but the rules are coarse.
- **Only one accessibility preset can be active at a time.** A learner needing Dyslexia typography together with ADHD pacing must choose one and adjust the remainder manually. Combinable presets are the clearest single extension to this design.
- **Video is not hosted by the platform.** Instructional video is embedded from an external host, so its availability, caption quality and playback behaviour are outside the platform's control, and a lesson's video can become unavailable without the platform knowing.
- **Validation is not defined in one place.** Field rules are expressed in the interface and again as database constraints. The database layer is authoritative and cannot be bypassed, so this is not a security weakness, but the duplication means a rule can be tightened in one place and not the other.
- **Route protection trusts the session.** The request check reads the role from the session rather than the database, to avoid a database query on every request. Every authorisation decision that matters is made again by row-level security against the

stored user record, so a forged session yields an interface shell and no data. This is a deliberate trade-off rather than a maximally strict design.

- **Load testing did not find a breaking point.** The load tests reported in Chapter 6 confirm that the operations a real user action performs hold their response-time targets at a realistic worst-case level of concurrency for this platform's scale, but none was pushed until it failed, so this report can state where headroom exists without stating where the ceiling is.
- **Offline use is not supported.** All features require a network connection. This is a functional limitation and also an accessibility one, since intermittent connectivity is itself a barrier for some learners.
- **Localisation is incomplete.** The interface is available in English and Bahasa Melayu, but the accessibility settings panel's own labels remain in English only, which is a poor outcome for the panel a learner reading in a second language is most likely to need. Course content is authored in one language and is not translated by the platform.

4.7 Summary

This chapter has translated the requirements of Chapter 3 into a design. The high-level design presented the behaviour of the three roles, the interactions between components for four key use cases, the user interface covering navigation, input validation and outputs, and the conceptual and logical data model as three domain entity relationship diagrams generated from the live schema.

Section 4.3 set out the accessibility design that is the substantive contribution of this project: a mapping of nineteen accessibility criteria to the feature, learner profile and module that address each; the three learner profiles and the support provided for each; the exact values applied by each preset; screenshot evidence of the same lesson rendered under each preset; and the authoring-time compliance checker that carries the content criteria a platform cannot satisfy on an educator's behalf. It also stated eight known

gaps in the same detail, the most significant being that no assistive technology and no neurodivergent learner has yet tested the platform.

The data dictionary documented 36 tables and 364 columns as they currently exist, together with the normalisation position and the two deliberate departures from it. The detailed design covered the component architecture, the self-contained computation modules, the division between operations issued by the browser and those reserved for privileged server routes, the rule-based recommendation algorithm, and the physical database implementation including constraints, indexes, access policies, triggers and storage.

The next chapter describes how this design was implemented: the development and deployment environment, the configuration and version control procedure, and the implementation status of each module.

CHAPTER 5: IMPLEMENTATION

5.1 Introduction

This chapter describes how the design presented in Chapter 4 was built. It covers the hardware and software environment in which the system was developed and deployed, the configuration management and version control procedure used to keep that environment reproducible, and the implementation status of every module.

The expected output of the implementation phase is a running system whose behaviour can be verified independently of the developer's own machine. Two decisions follow from that requirement. First, the database is defined entirely by version-controlled schema definitions and populated by a repeatable seed script, so that a complete working instance can be reconstructed from the project repository with a single command. Second, the properties that matter most and are easiest to break without noticing, namely the data isolation rules and the learner-facing data paths, are confirmed by executable checks rather than by inspection.

5.2 Development Environment Setup

5.2.1 Hardware Environment

Development was carried out on a single workstation, specified in Table 5.1. One figure in the table affected how the project was worked on rather than merely describing it: the machine has 8 GB of memory, of which 7.6 GB is usable, and the local backend stack runs eleven containers alongside the development server. The database stack and

the application server therefore compete for memory, which is why the local stack was started and stopped deliberately rather than left running.

Table 5.1: Development Workstation Specification

Component	Specification
Processor	AMD Ryzen 5 9600X, 6 cores and 12 threads
Memory	8 GB installed, 7.6 GB usable
Storage	1 TB solid-state drive
Operating system	Windows 11 Pro
Display	1920 by 1080 pixels, used for responsive and preset verification
Network	Broadband connection, required for the hosted database, the deployment platform and the external video and model services

No dedicated server hardware was provisioned. The production application runs on a serverless hosting platform and the database, authentication and file storage on managed cloud infrastructure. Development runs the same backend stack locally in containers.

5.2.2 Software Environment

Table 5.2 lists the software environment together with the versions actually used. Versions are given because several design decisions in Chapter 4 depend on them, in particular the request-protection mechanism provided by the framework version in use and the database features relied on by the schema.

Table 5.2: Software Environment and Versions

Software	Version	Role in the project
Node.js	24.15.0	Runtime for the development server, build and tooling
npm	11.12.1	Dependency and script management
TypeScript	5.9.3	Static typing across the codebase
Next.js	16.2.3	Application framework: routing, server rendering, route protection
React	18.2	User interface library
Tailwind CSS	4.x	Styling, and the layer through which accessibility settings are applied
Supabase CLI	2.116.0	Local backend stack, schema application, database rebuild and type generation

continued on the next page

Table 5.2 continued from the previous page

Software	Version	Role in the project
Docker	29.5.3	Container runtime hosting the local backend stack
PostgreSQL	17.6	Database, both locally and in the hosted project
Tiptap	3.24	Rich-text lesson editor
Recharts	3.8	Analytics charts
jsPDF	4.2	Certificate and administrator report generation
dnd kit	6.3	Drag-and-drop with keyboard support for interactive activities
DOMPurify	3.18	Sanitisation of authored lesson content
Visual Studio Code	N/A	Primary code editor
Git and GitHub	N/A	Version control and repository hosting
Vercel	N/A	Production deployment, built from the repository

The codebase comprises 335 source files totalling approximately 80,000 lines, organised by feature area as described in Chapter 4.

5.2.3 Local Backend Environment

Development does not run against the hosted database. The complete backend stack, comprising the database, the authentication service, the data interface, object storage and the administration console, runs locally in containers, and the application is pointed at it through configuration values, with the hosted configuration kept separately so that the two can be exchanged.

This arrangement exists for one specific reason. Verifying data isolation honestly requires attempting operations that should fail: signing in as one learner and trying to read another learner's progress, attempting to promote an account, or attempting to write a quiz score directly. These attempts are destructive in intent and cannot responsibly be made against a database holding real data. Running them against a disposable local instance that can be rebuilt within a minute is what made the verification reported in Section 5.4 possible.

5.3 Software Configuration Management

5.3.1 Configuration Environment Setup

Configuration in this project falls into four categories.

Runtime configuration. The database address, the public and privileged access keys, the mail credentials, the model service key and the public application address are supplied as environment values and are never committed to the repository. Two configuration files are kept, one for the active environment and one holding the hosted project's settings, so that switching between the local and hosted backend is a matter of exchanging files rather than editing values. The distinction between the two access keys is enforced structurally rather than by discipline: the public key is exposed to the browser and is subject to row-level security, while the privileged key is read only inside server routes and would bypass those policies, which is precisely why the four operations listed in Section 4.5.2 are the only ones permitted to use it.

Database configuration. The schema is never configured by hand. It is defined by four ordered schema definitions: a consolidated baseline creating every table, view, function, trigger, policy, index and permission; a definition creating the three storage areas and their policies; a definition enabling row-level security on two tables that had been left open; and a definition correcting account provisioning so that self-registration is fixed to the learner role. Applying these to an empty database produces exactly the schema documented in Chapter 4.

This replaced a history that had become unusable. The project had accumulated 44 schema files, none of which created any core table; every one assumed the tables already existed and only modified them. A clean rebuild therefore produced a broken database, and the files had drifted from the live schema. The consolidation was carried out by replaying the most recent schema dump together with the eleven definitions that post-dated it, then comparing the result column by column against the live project before

any cleanup was applied. All 44 original files are retained unmodified in an archive rather than deleted, so that the project's history remains inspectable.

Demonstration data. A seed script populates a rebuilt database with a fixed dataset: twelve accounts across the three roles, eleven courses in four lifecycle states, thirty lessons, five quizzes, ten interactive activities, twenty-one enrolments, nineteen quiz attempts, seven certificates, and the supporting progress and notification history. The script uses a fixed random seed, so repeated runs produce the same data, and it applies twenty-one referential integrity checks before reporting success.

Two properties of the seed data affect the credibility of the figures in this report. First, the chronology is generated forwards, from enrolment through first lesson view, completion, quiz attempt, course completion, achievement and certificate, and the checks reject any record that violates that order. Second, the accessibility score of each seeded lesson is not invented: every lesson is passed through the same compliance checker the educator's editor uses, and the score returned is the score stored. The weakest lessons in the dataset therefore fail for real and diagnosable reasons, which is why Figure 4.24 shows genuine findings rather than a constructed example.

Code quality configuration. Type checking runs in strict mode, and the linter uses the framework's recommended configuration together with accessibility-specific rules. A pull request template in the repository requires the author to state, for any change touching an accessibility setting, what the setting does, which standard justifies it, and whether the description matches the behaviour. This is a procedural control rather than an automated one, and its limitation is recorded in Section 5.4: nothing currently prevents a change that ignores it.

5.3.2 Version Control Procedure

The project is held in a Git repository hosted on GitHub, with 56 commits between 14 April and 26 August 2026. The deployment platform is connected to the repository

and builds the production deployment from the main branch, so a merge to that branch constitutes a release.

The working procedure was as follows.

1. **Commit granularity follows the change.** A commit corresponds to one coherent change, whether a feature, a correction or a review pass, and its message names the change rather than the date. Early in the project this was not observed, and the history contains a run of commits titled only by date. Those messages are of no use in locating when a behaviour changed, and the practice was abandoned.
2. **A destructive change is preceded by a checkpoint commit.** Before the database consolidation, a commit was made specifically to capture the state beforehand, so that the 44 replaced schema files and the schema they produced remain recoverable.
3. **Generated artefacts are committed, but their inputs are authoritative.** The entity relationship diagrams and the data dictionary in Chapter 4 are produced by scripts from the live schema. The generated files are committed so that the report can be compiled without a database, but the generator is the source of truth, and the correct response to a schema change is to regenerate rather than to edit the generated file.
4. **Credentials are never committed.** Configuration files holding keys are excluded from version control and are held in the local environment and in the deployment platform's own settings.

Two limitations of this procedure should be recorded. The project was developed by a single author, so branching was minimal and most work was committed directly to the main branch; a project with several authors would require a branch-per-change and review procedure that this one did not exercise. In addition, there is no continuous integration pipeline: the type checker, the linter and the verification scripts are run by hand and are not enforced automatically when a change is pushed. This is the most significant gap in the configuration management of the project and is recorded as such in Table 5.3.

5.4 Implementation Status

Table 5.3 reports the status of every module. Three status values are used. **Completed** means the module performs its specified function and has been exercised in a running system. **Completed with known limitations** means the module performs its specified function with a specific documented shortfall stated in the notes. **Not implemented** means the item was planned and does not exist.

Table 5.3: Implementation Status of Each Module

Module	Duration	Completed	Status	Notes
Project scaffolding and deployment pipeline	2 weeks	14/04/2026	Completed	Project structure, deployment connection, initial backend project.
Authentication and role-based access	3 weeks	08/05/2026	Completed	Account provisioning, route protection, session timeout. Self-registration fixed to the learner role after an escalation path was found and closed.
Course and lesson authoring	3 weeks	11/05/2026	Completed	Course, chapter and lesson management, rich-text editor, media upload, version snapshots.
Learner course experience	3 weeks	11/05/2026	Completed	Catalogue, enrolment, lesson viewer, roadmap, saved courses.
Assessment and grading	2 weeks	15/06/2026	Completed	Server-side grading, answer key withheld until submission, attempt limits enforced in the database.
Progress and telemetry	2 weeks	15/06/2026	Completed	View, completion, time-on-lesson and checkpoint state, consistency enforced by constraint.
Notifications	1 week	10/05/2026	Completed	Five database triggers, raised regardless of the path that caused the event.
Certificates and achievements	2 weeks	11/05/2026	Completed	Eligibility check, reference code, public verification page, PDF generation, educator issue and revocation.
Recommendation engine	2 weeks	17/06/2026	Completed	Four rule tiers with a stated reason per recommendation. Reads outcomes rather than behaviour (Section 4.6).
Educator analytics	2 weeks	17/06/2026	Completed	Course and learner views with at-risk flagging.
Administrator console and reports	3 weeks	18/06/2026	Completed	User and course management, application review, PDF reports, platform analytics including accessibility adoption.
Email delivery	1 week	25/06/2026	Completed	Password recovery and notification mail.
Interactive activities	2 weeks	26/06/2026	Completed with known limitations	Five activity types with builders and viewers, all keyboard-operable. The drag-and-drop diagram mode still has no alternative to dragging.
Embedded interactive content	1 week	26/06/2026	Completed	Embedded H5P objects with response capture.
Lesson assistant	1 week	24/08/2026	Completed	Stored plain-language summaries and lesson-scoped question answering. Optional, and the platform works without it. No assessment is graded by a model.
Accessibility personalisation engine	3 weeks	25/08/2026	Completed with known limitations	Three presets and 24 catalogued settings applied without reloading. Two settings do not fully match their labels (Section 4.3.7), and only one preset can be active at a time.

continued on the next page

Table 5.3 continued from the previous page

Module	Duration	Completed	Status	Notes
Accessibility compliance checker	2 weeks	25/08/2026	Completed	27 profile-aware rules with cited sources, running in the editor and at course level.
Localisation	1 week	25/08/2026	Completed with known limitations	Runtime switching between English and Bahasa Melayu, saved to the profile. The accessibility settings panel's own labels are not yet translated.
Security hardening	2 weeks	25/08/2026	Completed	Privilege escalation paths closed, completion derived on the server, guards added, and eight accessibility defects found by live inspection and corrected.
Database consolidation and seed data	1 week	26/08/2026	Completed	Rebuildable schema in four definitions, six unused tables removed, repeatable seed with 21 integrity checks.
Published accessibility statement	N/A	25/08/2026	Completed	Public page stating supported features and known gaps.
Continuous integration pipeline	N/A	N/A	Not implemented	Type checking, linting and the verification scripts are run manually and are not enforced automatically.
Automated accessibility scanning	N/A	N/A	Not implemented	No accessibility scanner and no assistive technology test suite exists (Section 4.3.7).
Combinable accessibility presets	N/A	N/A	Not implemented	Combining support across presets is not possible; identified as the clearest extension.
Offline support	N/A	N/A	Not implemented	No offline caching or installable application behaviour.

5.4.1 Verification Performed During Implementation

Five executable checks were built alongside the system and are run against a rebuilt local database. They are reported here as part of implementation rather than as testing, because their purpose is to confirm that what was built matches what was designed rather than to judge whether the design itself is correct. Systematic testing is the subject of Chapter 6. Table 5.4 lists them.

Table 5.4: Executable Verification Checks

Check	Assertions	What it confirms
Data isolation probe	20	That a learner cannot read another learner's progress, promote themselves, enrol someone else, forge a quiz score, or read an answer key before submitting, and that visitors who are not signed in see published courses only.
Learner scenario probe	28	That the learner-facing data paths return what the interface expects for each seeded learner scenario.
Seed data integrity check	21	Referential integrity and chronological consistency of the demonstration dataset before it is accepted.
Accessibility resolver check	18	That the settings resolver normalises contradictory and outdated accessibility settings correctly.
Settings catalogue check	234	That every accessibility setting is either catalogued with a description and a justification, or explicitly listed as excluded with a reason.

The last of these enforces a design rule rather than a behaviour: no accessibility setting may exist outside the catalogue, so adding a setting without describing it or explicitly excluding it causes the check to fail. This exists because the most damaging accessibility defects found during this project were not crashes but controls that appeared to work and did nothing. A control that cannot be added without being described is harder to leave inert.

5.5 Conclusion

This chapter has described the environment in which ACESS was built and deployed, the configuration and version control procedure that keeps that environment reproducible, and the implementation status of each module. Twenty-one modules are implemented, three of them with specific documented limitations, and four planned items are not implemented, the most consequential being the absence of a continuous integration pipeline and of automated accessibility scanning.

The substantive outcome of this phase is that the system is reconstructible rather than merely running: the schema is defined by version-controlled definitions and the demonstration data by a repeatable script, so the claims made in Chapter 4 can be

re-verified independently rather than taken on trust. Whether the system behaves correctly once running is the subject of the next chapter.

CHAPTER 6: TESTING

6.1 Introduction

This chapter describes how the ACES platform was tested after implementation. The purpose of the testing was to establish whether each function of the system behaves as specified in Chapters 3 and 4, whether the platform holds its response-time targets under realistic concurrent use, whether the accessibility claims made in Chapter 4 are actually met in the running system, and whether users working through a defined role can complete that role's tasks.

Testing was organised into four classes, kept separate because they produce different kinds of evidence.

- **Functional testing.** A set of defined test cases covering the public, learner, educator and administrator parts of the platform, each executed manually by the project author against the running application and recorded as pass or fail.
- **Performance and load testing.** Measurement of the response time and error rate of the operations that a real user action performs, under concurrent use representative of a class-sized group.
- **Manual accessibility evaluation.** A criterion-by-criterion self-assessment of the platform against the WCAG 2.2 success criteria listed in Table 4.3, carried out by the project author by operating the running system and inspecting the published lesson content.

- **User Acceptance Testing (UAT).** Sessions in which participants work through the tasks of one predefined user perspective and then complete a questionnaire about that experience.

Two points about the scope of this evaluation are stated here because they qualify everything that follows. First, the accessibility evaluation is a self-assessment. It was performed by the project author, who developed the system, and not by an independent accessibility specialist. It is therefore not an independent expert review, an accessibility audit, or any form of conformance certification, and the report does not claim conformance on the strength of it. Second, the UAT participants are not neurodivergent learners. They are proxies who were briefed to work through the system from a predefined user perspective, which is discussed in Section 6.4.5.

Security testing is not repeated as a separate class. Data isolation was verified by the executable checks described in Section 5.4.1, and this chapter refers to those results rather than re-running them.

6.2 Test Plan

6.2.1 Test Organisation

Table 6.1 identifies the people involved in testing and their responsibilities.

Table 6.1: Test Organisation

Role	Responsibility
Project author	Designed the test cases and executed them against the running system, recorded the results, diagnosed and corrected the defects found, conducted the performance and load tests, carried out the manual accessibility evaluation against WCAG 2.2, and prepared the UAT personas, task briefs and questionnaire, and is responsible for briefing the participants and collecting their responses.
UAT participants	Six volunteers, all fellow undergraduate students, one per user perspective. Each works through the tasks of the perspective assigned to them and then completes the questionnaire based on that experience.
Project supervisor	Reviewed the test scope, the test design and the evidence produced during supervision meetings.

No independent accessibility specialist and no external quality assurance tester was involved. Functional testing, performance testing and the accessibility evaluation were all carried out by the project author, which is recorded as a limitation in Section 6.5.6. The UAT is the only class in which other people took part, and their role is described in Section 6.4.5.

6.2.2 Test Environment

All testing was carried out against the local development environment described in Section 5.2 rather than against the deployed application: the application running on the development machine, connected to the complete backend stack running locally in containers, with the database rebuilt and populated by the seed script described in Section 5.3. Every test therefore exercises the actual rendered interface against a real, running database. The results reported in this chapter describe that build and should not be read as a statement about the separately hosted deployment until the same tests are repeated there.

The twelve seeded accounts listed in Table 6.2 served as the test accounts, and are the accounts named in the test cases and the UAT task briefs. All twelve share a single demonstration password, which is recorded in the project repository. Each account already carries a realistic history of part-finished courses, quiz attempts and issued certificates, so that the tests exercise varied data rather than empty accounts, and the learner accounts differ in which accessibility preset is active, which is what allows the same task to be tested under the Dyslexia, ADHD and Autism presets as well as with no preset.

Table 6.2: Seeded Test Accounts

Name	Sign-in address	Role	Active preset
Aliff Affandi	aliff.admin@access.edu.my	Administrator	None
Nurul Izzah Rahman	nurul.admin@access.edu.my	Administrator	None
Rajesh Kumar Menon	rajesh.admin@access.edu.my	Administrator	None
Dr. Siti Aminah Yusof	siti.educator@access.edu.my	Educator	None
Marcus Tan Wei Jie	marcus.educator@access.edu.my	Educator	None
Farah Nadhirah Idris	farah.educator@access.edu.my	Educator	None
Amir Hakim bin Rosli	amir.learner@access.edu.my	Learner	ADHD
Chong Mei Ling	mei.learner@access.edu.my	Learner	Dyslexia
Priya Devi a/p Ramesh	priya.learner@access.edu.my	Learner	Autism
Haziq Danial bin Zainal	haziq.learner@access.edu.my	Learner	None
Aisyah Nabila binti Kamal	aisyah.learner@access.edu.my	Learner	None
Daniel Lim Jun Hao	daniel.learner@access.edu.my	Learner	None

Every change a test case makes to the shared seed data is reversed at the end of the test, so that the test cases can be repeated without side effects accumulating. An enrolment created by a test is removed, a progress or accessibility record is restored to its previous value, an unsubmitted quiz attempt is discarded, a test course is deleted, and an approved educator application and the account it creates are both reverted.

One documentation problem was identified and corrected while preparing this environment. The reference document listing the test accounts still described thirty generic accounts from an earlier version of the seed data, none of which exist in the current database. Only the twelve accounts in Table 6.2 are real, which was confirmed directly against the running database, and the document was rewritten to describe those twelve accounts only.

6.2.3 Test Schedule

Table 6.3 states the cycles of testing, when each was carried out, the method used, and its status at the time of writing.

Table 6.3: Test Schedule

Cycle	Period	Method	Status
Verification during development	14 April to 26 August 2026	The five executable checks in Table 5.4, run after changes to the affected area.	Completed
Functional test case execution and defect correction	21 July to 1 August 2026	The nineteen test cases in Table 6.4, executed against the local environment, with the defects found corrected and the affected cases repeated.	Completed
Performance and load testing	6 to 8 August 2026	Three load scenarios and one concurrency test, described in Section 6.4.3.	Completed
Manual accessibility evaluation	12 to 15 August 2026	Criterion-by-criterion self-assessment against WCAG 2.2, recorded in Table 6.6.	Completed
UAT preparation	19 to 22 August 2026	Definition of the six user perspectives, their task briefs and the questionnaire (Section 6.4.5).	Completed
UAT execution and response collection	25 August to 2 September 2026	Six persona-based sessions, one participant per perspective, followed by the questionnaire.	In progress at the time of writing
Final testing review	2 September 2026	Consolidation of the results reported in Section 6.5.	Completed

6.3 Test Strategy

Functional testing followed a black-box strategy, working from the specified requirements and the rendered interface rather than from the internal structure of the code, and a top-down order of execution that began with complete user journeys and then narrowed to individual conditions such as invalid input, empty states and boundary values. Knowledge of the internal implementation was used only during diagnosis, after a test case had already failed.

6.3.1 Classes of Tests

- **Output correctness and functionality.** Whether a real account performing a real task through the rendered interface produces the correct result on screen and in the

stored data. This is the primary evidence of functional correctness.

- **Security and data isolation.** Whether one account can read or write another account's data. This was verified by the executable checks reported in Section 5.4.1 and is not repeated here.
- **Stress and load.** Whether the operations a user action performs hold their response-time targets under concurrent use.
- **Accessibility.** Whether each WCAG 2.2 criterion the platform targets is actually met in the running system, assessed criterion by criterion against the claims made in Table 4.3.
- **User acceptance.** Whether a participant working to a defined user perspective can complete that perspective's tasks and judges the interface suitable for it.

6.4 Test Design

6.4.1 Test Data

Real seeded data was used in preference to invented data. The demonstration dataset described in Section 5.3 supplies the twelve accounts of Table 6.2, eleven courses across four lifecycle states, thirty lessons, five quizzes, and a history of enrolments, lesson progress, quiz attempts and certificates, all generated in a chronologically consistent order and validated by twenty-one integrity checks before it is accepted. Every test case in Table 6.4 names the account, course, lesson or quiz it uses, and every one of those exists in the seeded database, so a result can be reproduced by rebuilding the database and repeating the steps.

The published courses used by the test cases are “Foundations of Accessible Learning”, “Reading Fluency with Dyslexia Support”, “Focus and Study Skills for ADHD Learners”, “Digital Tools for Independent Learning” and “Advanced Comprehension Strategies”. The two courses owned by Farah Nadhirah Idris, “Introduction to Assistive Technology” (draft) and “Inclusive Classroom Basics”

(awaiting review), are used for the cases that require an unpublished course or a course with no enrolments. The three quizzes exercised are “Foundations Review Quiz”, “Fluency Check” (no attempt limit) and “ADHD Study Skills Assessment” (limit of two attempts), which is what makes the exhausted-attempts case in LEARNER-10 reachable with a seeded account.

6.4.2 Functional Test Cases

Nineteen test cases were defined, covering the public pages, the learner experience under each of the four accessibility settings, the educator authoring and analytics functions, and the administrator moderation functions. Each test case signs in as a seeded account through the login form and then works through the same screens, controls and form fields a real user would. Where the result on screen alone cannot confirm that a change was actually saved, for example that approving an application changed its stored status rather than only its on-screen label, the test case also confirms the change in the stored data.

Test case selection covers three categories. Ten cases exercise the normal path of a feature. Six exercise an alternative or invalid path, such as attempting to enrol twice in the same course, attempting to continue past a form step with required fields left blank, or opening a quiz for which all permitted attempts have been used. Three exercise an empty or boundary state, such as a learner with no certificates and a course with no enrolments.

The test steps were repeated consistently across runs with the assistance of a browser automation tool, which drove the same screens and controls that a person would use. The tool was used only to replay the recorded steps reliably; the test cases themselves were designed, executed and judged by the project author, and every result in Table 6.4 was confirmed by observing the running interface. No conclusion about usability or accessibility is drawn from the tool.

Table 6.4 states each test case together with its test data, the steps followed, the expected result, the actual result and the final status.

Table 6.4: Functional Test Cases and Results

ID	Role / Preset	Description	Test Data	Steps	Expected Result	Actual Result	Status
PUBLIC-01	Visitor, not signed in	Confirm the landing page is reachable without signing in, and establish what happens when a visitor opens the course catalogue.	No account signed in.	1) Open the home page without signing in. 2) Confirm the page loads. 3) Open the course catalogue page.	The home page loads, and the catalogue either shows a public preview or redirects to the login page.	The home page loaded correctly. The catalogue redirected to the login page, confirming that no public catalogue page exists.	Pass
PUBLIC-02	Visitor, not signed in	Confirm an administrator page cannot be opened by a visitor who is not signed in.	No account signed in; the administrator dashboard address.	1) Enter the administrator dashboard address directly in the browser. 2) Observe which page loads.	The visitor is redirected to the login page and the administrator page is not displayed.	The visitor was redirected to the login page with a note that they would be returned to the dashboard after signing in. The administrator page was never displayed.	Pass
LEARNER-01	Learner, no preset	Enrol in a published course the learner is not already taking.	Account: haziq.learner. Course: “Focus and Study Skills for ADHD Learners” (published, owned by Dr. Siti Aminah Yusof). Haziq’s seeded enrolments cover only two other courses.	1) Sign in as Haziq. 2) Open the course detail page. 3) Select Enrol in Course. 4) Check the learner’s enrolments.	The enrol control is replaced once used, and a new active enrolment is recorded.	The enrolment was recorded correctly and was removed afterwards to restore the account to its seeded state.	Pass

continued on the next page

Table 6.4 continued from the previous page

ID	Role / Preset	Description	Test Data	Steps	Expected Result	Actual Result	Status
LEARNER-02	Learner, ADHD preset	Apply the ADHD preset, then complete a lesson that has not been finished.	Account: amir.learner. Course: “Digital Tools for Independent Learning”. Lesson: “Capturing Notes That You Will Actually Reread”, not completed in the seeded data.	1) Sign in as Amir. 2) Apply the ADHD preset from the accessibility settings panel and save. 3) Open the lesson. 4) Work through it to the final step and select Complete Lesson.	A learner using the ADHD preset can reach and use the Complete Lesson control, as a learner with no preset can.	The step navigation bar rendered correctly under the ADHD preset, the final step was reached, and the lesson was marked complete in the stored progress record.	Pass
LEARNER-03	Learner, Dyslexia preset	Take a quiz attempt through the normal quiz screen.	Account: mei.learner, whose profile has the Dyslexia preset and simplified reading level active. Quiz: “Fluency Check” in “Reading Fluency with Dyslexia Support”, which has no attempt limit.	1) Sign in as Mei. 2) Start a quiz attempt. 3) Answer the first question.	The quiz continues to the next question or shows feedback rather than failing to render.	The simplified reading banner appeared, questions were presented one at a time, and answering the first question advanced the attempt correctly.	Pass

continued on the next page

Table 6.4 continued from the previous page

ID	Role / Preset	Description	Test Data	Steps	Expected Result	Actual Result	Status
LEARNER-04	Learner, ADHD preset	View certificates already earned.	Account: amir.learner, who holds one certificate issued automatically on completing “Foundations of Accessible Learning” and one issued by Dr. Siti Aminah Yusof for “Focus and Study Skills for ADHD Learners”.	1) Sign in as Amir. 2) Open the Achievements and Certificates page.	Both certificates are shown, each with a reference code, the educator’s name and a completion date.	Both certificates were shown with the correct reference codes, issuer names and dates. Nothing was changed by this test.	Pass
LEARNER-05	Learner, no preset	Complete a lesson that has not been finished, with no preset active. This is the control case for LEARNER-02, 06 and 07.	Account: aisyah.learner. Course: “Advanced Comprehension Strategies”. Lesson: “Mapping the Structure of an Argument”.	1) Sign in as Aisyah with no preset active. 2) Open the lesson and read to the end. 3) Select Complete Lesson. 4) Check the progress page.	The control is available, and the lesson is marked complete afterwards.	The lesson was marked complete as expected, and the progress page reflected the change.	Pass

continued on the next page

Table 6.4 continued from the previous page

ID	Role / Preset	Description	Test Data	Steps	Expected Result	Actual Result	Status
LEARNER-06	Learner, Dyslexia preset	Complete an unfinished lesson under the Dyslexia preset, including its video requirement.	Account: mei.learner. Course: “Digital Tools for Independent Learning”. Lesson: “Dictation and Voice Typing”, which contains an embedded video.	1) Sign in as Mei with the Dyslexia preset active. 2) Confirm the video has been watched. 3) Read to the end of the lesson. 4) Select Complete Lesson.	The lesson is marked complete, as it would be with no preset active.	The lesson was completed correctly once the video was marked as watched. Selecting Complete Lesson beforehand correctly refused and explained what was outstanding. The lesson text was rendered in the dyslexia-friendly typeface.	Pass
LEARNER-07	Learner, Autism preset	Confirm the video acknowledgement control can be selected under the Autism preset’s step-by-step layout.	Account: priya.learner. Course: “Focus and Study Skills for ADHD Learners”. Lesson: “Beating the Starting Problem”.	1) Sign in as Priya with the Autism preset active. 2) Locate the “I have watched this video” control. 3) Select it.	Selecting the control registers the action and marks the video as watched.	The control registered correctly and the video was marked as watched.	Pass
LEARNER-08	Learner, no preset	Confirm a learner cannot enrol twice in the same course.	Account: haziq.learner. Course: “Foundations of Accessible Learning”, in which he already holds an active enrolment.	1) Sign in as Haziq. 2) Reopen the detail page of that course.	No enrol control is offered; the page offers to continue the course instead.	The page offered Continue Learning in place of an enrol control, so a second enrolment could not be created.	Pass

continued on the next page

Table 6.4 continued from the previous page

ID	Role / Preset	Description	Test Data	Steps	Expected Result	Actual Result	Status
LEARNER-09	Learner with no certificates	Open the certificates page for a learner who has earned none.	Account: haziq.learner, who has completed no course and holds no certificate or achievement in the seeded data.	1) Sign in as Haziq. 2) Open the Achievements and Certificates page.	The page states that no certificates have been earned rather than showing a blank or broken page.	The page displayed a clear message that no certificates had been earned.	Pass
LEARNER-10	Learner, ADHD preset, attempts exhausted	Open a quiz for which all permitted attempts have been used, and confirm the interface explains why no further attempt is possible.	Account: amir.learner. Quiz: “ADHD Study Skills Assessment”, which permits two attempts; Amir’s seeded history records two attempts, scoring 67% and 100%.	1) Sign in as Amir. 2) Open the lesson containing that quiz. 3) Navigate to the quiz section.	No start control is offered, and a message explains that all permitted attempts have been used.	The quiz section displayed a heading, a warning icon and a message stating that the maximum number of attempts had been reached.	Pass
EDUCATOR-01 and 02	Educator	Create a course through the course creation wizard, then set its accessibility focus.	Account: farah.educator. A new course created for this test and deleted afterwards.	1) Sign in as Farah. 2) Complete all four steps of the wizard and create the course. 3) Confirm it appears in her course list. 4) Open Settings, set the accessibility focus to ADHD and save.	The course appears in the list and its accessibility focus is saved as ADHD.	Both results were as expected. The test course was deleted afterwards.	Pass

continued on the next page

Table 6.4 continued from the previous page

ID	Role / Preset	Description	Test Data	Steps	Expected Result	Actual Result	Status
EDUCATOR-03	Educator	View the course analytics dashboard.	Account: siti.educator, who owns four published courses in the seeded data.	1) Sign in as Siti. 2) Open the Analytics page.	Per-course figures are shown for enrolments, active learners, learners at risk, average progress, completion rate and average quiz score.	All figures rendered correctly for all four courses. Nothing was changed by this test.	Pass
EDUCATOR-04	Educator	Confirm the course creation wizard does not allow the first step to be completed with required fields blank.	Account: farah.educator; the wizard with title and description left blank.	1) Sign in as Farah. 2) Open the wizard. 3) Select Continue without entering a title or description.	Continue remains unavailable until both fields are supplied.	Continue remained unavailable while the fields were empty and became available once both were supplied.	Pass
EDUCATOR-05	Educator	View analytics for a course with no enrolments.	Account: farah.educator. Course: "Introduction to Assistive Technology", which is unpublished and has no enrolments in the seeded data.	1) Sign in as Farah. 2) Open the Analytics page and locate that course.	The course shows zero values rather than a broken or missing calculation.	Every figure for the course showed zero, with no missing or broken values anywhere on the page.	Pass
ADMIN-01	Administrator	Approve a pending application to become an educator.	Account: aliff.admin. The seeded pending application from Tan Chee Meng.	1) Sign in as Aliff. 2) Open the pending application. 3) Select Approve and confirm. 4) Check the stored status.	The status changes to approved and an educator account is created for the applicant.	The status changed to approved and the educator account was created. The change was reverted afterwards.	Pass

continued on the next page

Table 6.4 continued from the previous page

ID	Role / Preset	Description	Test Data	Steps	Expected Result	Actual Result	Status
ADMIN-02	Administrator	Reject a pending educator application.	Account: aliff.admin. A temporary application created for this test and removed afterwards.	1) Sign in as Aliff. 2) Open the pending application. 3) Select Reject and confirm. 4) Check the stored status.	The status changes to rejected.	The status changed to rejected, and the temporary application was removed afterwards.	Pass
A11Y-01	Learner, Dyslexia preset	Confirm that closing the incomplete-tasks dialog leaves the rest of the page usable.	Account: mei.learner; the dialog closed by its close control, by the Escape key and by selecting outside it.	1) Attempt to complete a lesson before its requirements are met, which opens the dialog. 2) Close the dialog by each of the three methods in turn. 3) After each, select another element on the page.	The dialog closes cleanly each time and the page remains fully usable.	All three methods closed the dialog and left the page fully usable.	Pass

6.4.3 Load Testing

Load testing measured the three data operations that a real user action performs most often. Each scenario calls the same operation, under the same authentication and access-policy conditions, that the application itself uses for that action, rather than a substitute. The scenarios were executed with the k6 load testing tool against the local environment described in Section 6.2.2.

Concurrent catalogue reads. This scenario retrieves the published course catalogue using the same query the learner catalogue page issues, as a visitor who is not signed in. ACCESS is intended as a small institutional deployment, so fifty concurrent users approximates an entire class together with staff opening the catalogue at the same moment, which is a realistic worst case for this operation rather than steady background traffic. The load was increased from zero to fifty users over thirty seconds, held for thirty seconds and then reduced over ten seconds, so that the database connection pool reaches a steady state before the response times are read.

Concurrent enrolments. This scenario creates and then removes an enrolment using each learner's own session, so that the enrolment access policy is evaluated on every request rather than bypassed. A single repeated insertion cannot sustain a duration test, because the uniqueness rule on the learner and course pair prevents a second identical enrolment, so each virtual user alternates between enrolling and withdrawing. Forty concurrent users, one per temporary account created for the test, models a group enrolling in a newly announced course within the same half minute.

Concurrent quiz submissions. This scenario calls the grading function, which is the only path by which a quiz result can be written, using the same forty accounts and the same thirty-second window. The quiz used is "Fluency Check", which has no attempt limit and is therefore the only seeded quiz able to absorb a sustained load without later attempts being refused. The first execution of this scenario identified a genuine concurrency defect, recorded as Defect 3 in Section 6.5.2, which was corrected before the result reported here was recorded.

Table 6.5 summarises the results. None of the three scenarios was taken to a breaking point; all held their response-time targets with substantial margin at the level of concurrency tested. This is evidence that the operations can absorb a realistic burst for a deployment of this size, not evidence of where the limit lies.

Table 6.5: Load Test Results

Scenario	Virtual users	Duration	95th percentile response time	Error rate
Catalogue reads	50	30 s hold, 70 s total	6.99 ms (target below 800 ms)	0.00% (0 of 2,018)
Concurrent enrolments	40	30 s	65.86 ms (target below 1,000 ms)	0.00% (0 of 2,444)
Concurrent quiz submissions	40	30 s	86.06 ms (target below 1,200 ms)	0.00% (0 of 1,302)
Same-enrolment concurrency test	8 simultaneous requests	One batch	334 ms average	0.00% (0 of 8); all eight attempt numbers distinct

6.4.4 Manual Accessibility Evaluation

The accessibility evaluation is a self-assessment of the platform against the WCAG 2.2 success criteria listed in Table 4.3, carried out by the project author. Its purpose is to check the design claims made in Chapter 4 against the behaviour of the running system rather than accept them, and it is described as a self-assessment throughout because it was performed by the developer of the system rather than by an independent accessibility specialist.

The method was as follows. Each criterion in Table 4.3 was taken in turn. The feature claimed to satisfy it was located in the running platform and operated directly, under each of the four accessibility settings where the criterion depends on the preset. Where the criterion concerns authored content rather than the platform, the twenty-nine published lessons in the seeded dataset were inspected against it. A criterion was recorded as Pass only where the behaviour was observed to match the claim, Partial where part of the claim held and a specific shortfall was identified, Fail where the observed behaviour contradicted the claim, and Pending where the judgement requires a

screen reader, other assistive technology, or a sensory judgement that this evaluation could not supply. Table 6.6 records the finding for each criterion together with the observation supporting it.

Table 6.6: WCAG 2.2 Self-Assessment Findings

Criterion	Finding	Observation
1.1.1 Non-text Content (A)	Partial	The compliance checker enforces the rule. A review of all 29 published lessons found three images, none missing alternative text, and one carrying empty alternative text. Whether that image is genuinely decorative requires a judgement on the content itself.
1.2.1 Video-only (A)	Fail	The compliance checker requires a transcript on any lesson containing video, but two of the seven published lessons with video have no transcript. The check is applied when a lesson is saved, so lessons authored before the rule was introduced were not re-evaluated.
1.2.2 Captions (A)	Pending	Requires a captioned video to be played and judged, which was not possible for the externally hosted video used in the seeded lessons.
1.3.1 Info and Relationships (A)	Pass	The compliance checker enforces the rule, and no published lesson over 200 words lacks a heading.
1.4.4 Resize Text (AA)	Pending	The range of 12 to 24 px is implemented as documented and was observed to take effect. Confirming that the control is operable without a pointing device requires assistive technology testing.
1.4.5 Images of Text (AA)	Pending	Distinguishing real text from an image of text across all authored content could not be established reliably by inspection alone.
1.4.8 Visual Presentation (AAA)	Pass	The reading widths documented in Table 4.5 were confirmed on a rendered lesson. A conflict was found in which an older simplified-reading setting overrode the Dyslexia preset's own background and typeface; this was corrected as Defect 7 and the preset's presentation was confirmed to apply afterwards.
1.4.12 Text Spacing (AA)	Pending	Spacing was observed to change as set. Confirming that no content is clipped at the extremes of the range across every screen requires a fuller inspection than this evaluation covered.
2.2.1 Timing Adjustable (A)	Fail	The quiz time limit is displayed to the learner as documented, but when the timer expires and no accessibility preset is active, the attempt is submitted automatically with no warning and no option to extend. An extend-or-submit prompt is presented only when a preset is active, so a learner using no preset receives no adjustment mechanism at all.
2.2.4 Interruptions (AAA)	Partial	Distraction-free mode removes navigation during a lesson as intended, but under the ADHD preset it also removes the sidebar throughout the rest of the platform rather than only inside a lesson, which leaves no route back to the accessibility settings.
2.2.6 Timeouts (AAA)	Pass	The fifteen-minute inactivity timeout and the thirty-second warning were confirmed to match the documented behaviour.
2.3.3 Animation (AAA)	Partial	Setting motion to none suppresses transitions and animation defined in the stylesheet, confirmed on the lesson completion effect. It does not affect animation produced by the animation library, which does not use the properties the rule targets, so one completion animation still plays at its fixed duration regardless of the setting.
2.4.4 Link Purpose (A)	Partial	The rule is enforced for authored lesson content. One control in the platform's own interface, a bare "Read more" in the timeline activity, is not descriptive and is outside anything the checker evaluates.
2.4.6 Headings and Labels (AA)	Pass	As for 1.3.1, no published lesson over 200 words lacks a heading.
3.1.5 Reading Level (AAA)	Partial	The Flesch–Kincaid calculation is implemented as documented. However, the quiz view presented to a learner whose reading level is set to simplified displays three untranslated interface labels instead of plain-language text when an answer is wrong, which is the opposite of what that setting is intended to achieve.
3.2.3 Consistent Navigation (AA)	Pass	The navigation shell and the order of its items were compared across the learner, educator and administrator areas and were found to be consistent within each role.
3.2.5 Change on Request (AAA)	Partial	The preset preview dialog was confirmed to show the before-and-after values prior to applying a preset. Several embedded media frames declare an automatic-playback permission, although none of the addresses checked requests automatic playback, so nothing was observed to start on its own.

continued on the next page

Table 6.6 continued from the previous page

Criterion	Finding	Observation
4.1.2 Name, Role, Value (A)	Pending	Navigation is built from real links carrying current-page state, and icon-only controls carry accessible names and states, both confirmed by inspection. Whether these are announced correctly cannot be established without a screen reader.
W3C cognitive guidance, objectives 3–5	Pending	The visual schedule and step-by-step guidance were confirmed to render and to list the phases of a lesson before it begins. Whether the result is genuinely clear to a learner is a judgement carried into the user acceptance testing described in Section 6.4.5.

6.4.5 User Acceptance Testing

Purpose User Acceptance Testing was carried out to obtain feedback from people other than the developer on whether the system is usable and whether it supports the tasks belonging to each of its user roles. Functional testing establishes that a function produces the correct result; it does not establish whether a person can find that function, understand what the screen is telling them, or complete a realistic task without help. That is what this class of testing contributes.

Why persona-based proxy participants were used The intended users of ACCESS are learners with Dyslexia, ADHD and Autism Spectrum Disorder. Recruiting participants with these conditions was not possible within the scope and timeframe of this project, and doing so would also raise consent and ethical approval requirements beyond what a final-year project of this size can undertake. On the supervisor’s advice, the UAT was therefore conducted with proxy participants: fellow undergraduate students who were each briefed to work through the system from one predefined user perspective and to evaluate it from that point of view.

This limitation must be stated plainly. The participants are not neurodivergent learners, and they are not accessibility specialists. Their responses describe how a person working to a defined brief experienced the interface. They do not describe the lived experience of a learner with Dyslexia, ADHD or Autism, and they are not evidence that the accessibility features meet the needs of those learners. The appropriate reading of this class of evidence is that it provides preliminary user-perspective feedback on the usability and apparent suitability of the system when a person interacts with it according

to a defined persona. It does not replace evaluation with the actual target users, which is recorded as future work in Chapter 7.

Participants and procedure Six participants take part, one per perspective. All are undergraduate students at the same faculty, none has used ACESS before, and none was involved in developing it. Each participant receives the following in writing: a short description of the persona they are asked to represent, a statement of what that user needs from the system, the accessibility expectations attached to that perspective, and the list of tasks to perform. Each is also given the address of the running system and the sign-in details of the seeded account belonging to that perspective, taken from Table 6.2.

Participants work through the tasks on their own, without being guided, so that difficulty in finding a feature shows up rather than being smoothed over. The project author is present only to supply the account details and to answer questions about the brief itself, not about how to use the system. After completing the tasks, each participant completes the questionnaire described below, answering it from the point of view of the persona they were asked to represent. The status of this cycle at the time of writing is recorded in Table 6.3, and the results are reported in Section 6.5.5.

The six user perspectives Table 6.7 states each perspective, what that user needs from ACESS, and the accessibility expectations attached to it. The four learner perspectives correspond to the four accessibility settings the platform supports, so that the default experience and each of the three presets are all covered.

Table 6.7: User Acceptance Testing Perspectives

Perspective	What this user needs from ACESS	Accessibility expectations
Default learner	A learner who has not set any accessibility preferences. Needs to find a course, enrol, work through lessons in order, be assessed, and see how far they have progressed.	The standard interface should be usable and readable without any adjustment, and no accessibility feature should be required in order to complete a course.

continued on the next page

Table 6.7 continued from the previous page

Perspective	What this user needs from ACESS	Accessibility expectations
Dyslexia learner	A learner who finds long passages of text tiring to decode and loses their place in dense paragraphs. Needs to change how text is presented and to have an alternative to reading.	A dyslexia-friendly typeface, larger text, wider line and word spacing, a tinted background, a narrower reading column, a reading spotlight, and read-aloud, all reachable without leaving the lesson.
Autism learner	A learner who works best when the structure of a task is known in advance and stays the same. Needs to know what a lesson contains before starting it and to meet no unexpected changes.	A visual schedule shown before the lesson, step-by-step guidance that explains why a control is unavailable, a consistent page layout between lessons, reduced colour and no motion, and no media that starts on its own.
ADHD learner	A learner for whom sustained attention is costly and who loses their place after an interruption. Needs a reduced interface, clear next steps, and short units of work.	A distraction-free lesson view, a bar showing the current task, a task checklist, a progress timeline, content divided into short sections, and a quiz that does not impose hidden time pressure.
Educator	An instructor who authors and maintains courses and monitors the learners taking them. Needs to create structured content, check it before publishing, and see how learners are progressing.	Accessibility problems in authored content should be reported while the lesson is being written, with the reason stated and a route to the correction.
Administrator	A platform operator responsible for who may publish and what is published. Needs to moderate courses, review educator applications, manage accounts and see platform-level activity.	Platform-level reporting should show which accessibility features are actually being used, so that accessibility can be monitored rather than assumed.

Tasks performed for each perspective Table 6.8 lists the tasks given to each perspective. Every task uses a function that is implemented in the current build, and each was selected so that the participant encounters the parts of the system that matter most to that perspective.

Table 6.8: User Acceptance Testing Task Briefs

Perspective	Tasks
Default learner	(1) Sign in with the default learner account. (2) Browse the course catalogue and open a course you are not taking. (3) Enrol in that course. (4) Open its first lesson and work through it to the end. (5) Mark the lesson complete. (6) Attempt the lesson quiz and review the result. (7) Open the Progress page and find how far through the course you are. (8) Open Achievements and Certificates.

continued on the next page

Table 6.8 continued from the previous page

Perspective	Tasks
Dyslexia learner	(1) Sign in with the Dyslexia learner account. (2) Open Accessibility from the sidebar and select the Dyslexia preset, reading the preview of what will change before confirming it. (3) Open a lesson and read one full section. (4) Use the Listen control to hear part of the lesson read aloud. (5) Use the reading toolbar inside the lesson to change the text size and line spacing. (6) Turn the reading spotlight on and off. (7) Mark the lesson complete.
Autism learner	(1) Sign in with the Autism learner account. (2) Apply the Autism preset. (3) Before opening a lesson, look at its visual schedule and note what the lesson will involve. (4) Work through the lesson using the step-by-step guidance. (5) Open a second lesson in the same course and compare its layout with the first. (6) Mark a lesson complete. (7) Open the Progress page.
ADHD learner	(1) Sign in with the ADHD learner account. (2) Apply the ADHD preset. (3) Open a lesson and note that the sidebar and notifications are hidden. (4) Work through the lesson section by section, using the task checklist and the current-task bar. (5) Leave the lesson and return to it, using the progress information to find your place. (6) Attempt the lesson quiz and note the timer shown.
Educator	(1) Sign in with the educator account. (2) Create a new course through the course creation wizard, giving it a title, description and difficulty level. (3) Add a lesson containing at least one heading and one image. (4) Run the accessibility compliance check on that lesson and correct at least one issue it reports. (5) Add a quiz question to the lesson. (6) Open the Analytics page and read the figures for one of your courses. (7) Open Learner Progress and find one learner's progress. (8) Open the Certificates page for one of your courses.
Administrator	(1) Sign in with the administrator account. (2) Open Course Management, find a course awaiting review, and approve or reject it. (3) Open Educator Applications and review one pending application. (4) Open User Management and find a specific user account. (5) Open Analytics, including the Accessibility tab, and read the platform figures. (6) Open Reports and generate one report. (7) Open Feedback and mark one enquiry as read.

Questionnaire Responses were collected through a single online questionnaire built with Google Forms. The form opens with an explanation of the purpose of the evaluation and a statement that the participant is evaluating the system from an assigned user perspective rather than as themselves. A single question then asks which perspective the participant was assigned, and the form branches to the section for that perspective. Every section presents the same eight agreement statements, so that results are comparable across perspectives, followed by two statements specific to that perspective. All statements are rated on a five-point scale from Strongly Disagree to Strongly Agree. Every path then closes with the same four open-ended questions. Table 6.9 lists the common statements, and Table 6.10 the perspective-specific ones.

Table 6.9: Common Agreement Statements Asked of Every Perspective

No.	Aspect	Statement
Q1	Ease of use	The system was easy to use.
Q2	Navigation	Moving between pages and finding the features I needed was straightforward.
Q3	Clarity of information	The information shown on screen was clear and easy to understand.
Q4	Readability	The text and content were comfortable to read.
Q5	Instructions	The on-screen instructions and labels were easy to follow.
Q6	Support for the perspective	The interface suited the needs of the user perspective I was asked to represent.
Q7	Task completion	The system made the tasks in my brief easy to complete.
Q8	Overall satisfaction	Overall, I am satisfied with the system for this user perspective.

Table 6.10: Perspective-Specific Agreement Statements

Perspective	Statements
Default learner	Finding a course, enrolling and working through a lesson followed a logical order. The quiz result and progress information were easy to interpret.
Dyslexia learner	The Dyslexia preset made the lesson text more comfortable to read than the default settings. The read-aloud and reading toolbar controls were easy to find and use.
Autism learner	The visual schedule made it clear what the lesson would involve before I started it. The layout and structure stayed consistent and predictable between lessons.
ADHD learner	The distraction-free lesson view helped me stay on the current task. The task checklist and the current-task bar made it clear what to do next.
Educator	Creating a course and writing a lesson was straightforward. The accessibility compliance check gave feedback I could understand and act on.
Administrator	Reviewing courses and educator applications was straightforward. The platform analytics gave a clear overview of activity on the system.

Each path of the form closes with four open-ended questions: what the participant needed from the system that was not available; what they would improve; whether they encountered anything confusing or difficult; and any other comments. These are included because a five-point scale records that something was rated poorly but not why, and the reason is what a change to the system has to be based on.

How the responses are interpreted With one participant per perspective, the results are read as indicative feedback rather than as a statistical result. Three things are

reported: the distribution of agreement ratings across the eight common statements, which indicates whether any aspect of the interface was consistently rated poorly across perspectives; the perspective-specific ratings, which indicate whether each accessibility preset was perceived to serve the perspective it was designed for; and the themes appearing in the open-ended answers, which are the most useful part of a sample this size. No statistical significance is claimed and no average is treated as a measurement of accessibility. The results are used to identify candidate improvements, which are reported alongside the other future work in Chapter 7.

6.5 Test Results and Analysis

6.5.1 Functional Test Results

All nineteen test cases in Table 6.4 recorded a final status of Pass. Table 6.11 summarises their distribution across the platform.

Table 6.11: Summary of Functional Test Results

Area tested	Test cases	Pass	Fail
Public access and route protection	2	2	0
Learner experience	10	10	0
Educator authoring and analytics	4	4	0
Administrator moderation	2	2	0
Accessibility interaction	1	1	0
Total	19	19	0

Two observations follow from the distribution. First, the learner experience carries the largest share of the test cases because it is the part of the platform where the accessibility presets change what is actually rendered, and a function that works with no preset active cannot be assumed to work under each of the three presets. Test cases LEARNER-02, 05, 06 and 07 test the same underlying function, completing a lesson, under four different presets for exactly this reason, and two of the defects recorded below were found only because that comparison was made.

Second, seven of the nineteen cases test an invalid, repeated, empty or boundary condition rather than the normal path. Two of the defects recorded below were found by those cases and not by the normal-path cases, which supports the value of testing the alternative paths of a function rather than only the path a feature is designed around.

6.5.2 Defects Identified and Resolved

Seven defects were identified during testing: four through functional test case execution, one through load testing and two through the manual accessibility evaluation. All seven were corrected in the codebase and the affected test cases were repeated to confirm the correction before the results in Table 6.4 were recorded. Table 6.12 summarises them.

Six of the seven were failures of what the interface actually renders or of the behaviour of concurrent requests, rather than failures of stored data. None of them would have been detected by inspecting the database alone, and all of them occurred in modules already marked complete in Table 5.3. This is the clearest single finding of the testing phase and is discussed further in Section 6.5.6.

Table 6.12: Defects Identified During Testing and Their Resolution

No.	Found by	Description	Resolution
1	Test case A11Y-01	After the incomplete-tasks dialog was closed, the whole lesson page became permanently unresponsive to the mouse, with no visible sign that anything was wrong. The cause was that the dialog component library tracks open dialogs in a single shared counter, and the lesson page opens five dialogs at once, so the counter never returned to zero and the page was never released.	A small guard component was added that releases the page as soon as no dialog is open. Confirmed by opening and closing the dialog by all three available methods and checking that a subsequent selection succeeds each time.

continued on the next page

Table 6.12 continued from the previous page

No.	Found by	Description	Resolution
2	Test case LEARNER-07	Under the Autism preset, the floating step-by-step checklist panel overlapped the lesson's action bar on a common screen width and absorbed selections intended for the control beneath it, so the video acknowledgement control could not be used.	The panel is now displayed only at wider screen sizes, and the panel background no longer receives pointer input, so only its own controls do. Confirmed at a screen width where the panel is still displayed, checking that the control beneath now receives the selection.
3	Load testing	Two of 1,292 concurrent quiz submissions failed because the grading function read the highest existing attempt number and then inserted the new attempt without preventing another request from doing the same in between, so two simultaneous submissions for the same enrolment could be assigned the same attempt number.	A lock keyed on the enrolment and quiz was added so that concurrent submissions for the same pair are handled one at a time. Confirmed by a dedicated concurrency test issuing eight simultaneous submissions, all of which succeeded with distinct attempt numbers.
4	Test case ADMIN-01	Approving an educator application intermittently produced no stored change, although the interface reported no error. Two causes were found: the application details dialog and the confirmation dialog were separate components, so one closed at the moment the other opened and the resulting selection was lost. The second cause is a characteristic of the development server, which compiles a page on first use and can discard a request that arrives at that moment.	The three dialog views were combined into one so that only a single open or close transition occurs. The second cause belongs to the development environment rather than to the application; it is mitigated by issuing one request before testing begins. Confirmed by repeating the test case three times consecutively with no failures.
5	Test case LEARNER-02	Under the ADHD preset, no lesson completion control could be reached at all. The step navigation bar that advances the lesson through its phases was shown only when distraction-free mode had been switched on manually, whereas the ADHD preset enables that mode by a different route, so the lesson could never advance to the final phase where the completion control appears.	The navigation bar is now shown whenever distraction-free mode is active, by whichever route it was activated. Confirmed by signing in with the ADHD account, advancing through the lesson phases, completing the lesson and checking the stored progress record.

continued on the next page

Table 6.12 continued from the previous page

No.	Found by	Description	Resolution
6	Test case LEARNER-10	A quiz whose permitted attempts had all been used rendered its heading and question count and then nothing else, giving no explanation for the absence of a start control. The cause was that the reason for refusal was stored as a full explanatory sentence rather than as the category value the display logic tests, so no case matched and no message was shown.	The category value and the explanatory sentence are now stored separately, and the sentence is displayed beneath a “Maximum attempts reached” heading. Confirmed with the same account and quiz.
7	Accessibility evaluation, criterion 1.4.8	A learner using the Dyslexia preset received a high-contrast theme at the same time as the preset’s own cream background and dyslexia-friendly typeface, because an older simplified-reading setting reapplied its own theme on every profile load and overrode the preset.	The theme override was removed from the older setting, leaving its other behaviour unchanged. Confirmed by signing in again with the Dyslexia account and checking that the preset’s own background and typeface apply without conflict.

6.5.3 Load Test Results

The load test results are given in Table 6.5. After Defect 3 was corrected, all three scenarios and the concurrency test recorded a 0.00% error rate, and the 95th percentile response time in every case was at least an order of magnitude below its target: 6.99 ms against 800 ms for catalogue reads, 65.86 ms against 1,000 ms for enrolments and 86.06 ms against 1,200 ms for quiz submissions. The non-functional performance requirements stated in Section 3.3.3 are therefore met at the level of concurrency tested. The qualification recorded in Section 6.4.3 stands: these results establish that the platform has headroom at a realistic load for a deployment of this size, not where its limit lies.

6.5.4 Accessibility Evaluation Results

The self-assessment in Table 6.6 covered nineteen criteria. Five were assessed as Pass, six as Partial, two as Fail and six as Pending, the last group being those that require a screen reader, other assistive technology, or a judgement about how the interface is experienced. Table 6.13 summarises the distribution.

Table 6.13: Summary of the WCAG 2.2 Self-Assessment

Finding	Count	Criteria
Pass	5	1.3.1, 1.4.8, 2.2.6, 2.4.6, 3.2.3
Partial	6	1.1.1, 2.2.4, 2.3.3, 2.4.4, 3.1.5, 3.2.5
Fail	2	1.2.1, 2.2.1
Pending	6	1.2.2, 1.4.4, 1.4.5, 1.4.12, 4.1.2, and the W3C cognitive guidance objectives

The two failures are significant and are stated rather than qualified. Criterion 1.2.1 fails because two of the seven published lessons containing video have no transcript, which the platform's own compliance rule prohibits; the rule is applied when a lesson is saved, so lessons authored before the rule was introduced were never re-evaluated. Criterion 2.2.1 fails because a quiz timer expiring with no accessibility preset active submits the attempt automatically, without warning and without offering an extension, so the learners least likely to have configured a preset are the ones given no adjustment mechanism. Correcting both is recorded as future work in Chapter 7.

The six Pending findings define the boundary of what this evaluation can establish. Four of them require a screen reader or another assistive technology, which was not available; the remaining two require a judgement about how the interface is experienced rather than how it behaves. These are the criteria on which no conclusion should be drawn from this chapter, and they are the strongest single argument for the independent accessibility audit recommended in Chapter 7.

6.5.5 User Acceptance Testing Results

At the time of writing, the UAT questionnaire had been distributed to the six participants and the response collection window recorded in Table 6.3 had not closed, so no results are reported here. No responses are estimated, summarised or otherwise presented in this chapter, because doing so would misrepresent evidence that does not yet exist.

When the responses are collected, this section will report the number of participants and the number of responses received for each perspective; the distribution of the agreement ratings for the eight common statements in Table 6.9 and for the perspective-specific statements in Table 6.10; the aspects of the interface rated most and least favourably across the six perspectives; and the themes appearing in the four open-ended questions, with the specific problems identified separated from general comments. The interpretation will follow the terms set out in Section 6.4.5: with one participant per perspective, the results indicate where the interface caused difficulty and where it did not, and are not treated as a measurement of accessibility or of learning outcomes.

6.5.6 Limitations of the Testing Performed

Five limitations of this testing should be recorded.

- **Functional, performance and accessibility testing were carried out by the project author.** Every test case was designed, executed and judged by the same person who implemented the feature being tested, with no independent reviewer.
- **The accessibility evaluation is a self-assessment.** It was not carried out by an accessibility specialist, it was not an independent audit, and it confers no conformance status. Its findings should be read as the developer's own assessment against published criteria.
- **No assistive technology was used.** Six criteria in Table 6.6 remain Pending for this

reason, and the screen-reader behaviour of the platform is therefore inferred from its markup rather than observed.

- **The UAT participants are proxies, not the intended users.** They are undergraduate students working to a persona brief. Their feedback describes how a briefed user experienced the interface and is not evidence about the experience of learners with Dyslexia, ADHD or Autism.
- **Testing was carried out against the local environment.** The deployed application was not tested with the same test cases, so the results describe the local build.

6.6 Conclusion

This chapter has reported the testing carried out on the ACESS platform. Nineteen functional test cases covering the public, learner, educator and administrator functions were executed against the running system, and all nineteen recorded a final result of Pass. Three load scenarios and one concurrency test confirmed that the operations performed by a real user action hold their response-time targets with substantial margin at a level of concurrency representative of a class-sized group. A manual self-assessment against the nineteen accessibility criteria the platform targets returned five passes, six partial results, two failures and six criteria that could not be judged without assistive technology.

Seven defects were identified and corrected during this phase. Six of them were failures of what the interface renders or of concurrent behaviour rather than of stored data, and all of them occurred in modules previously recorded as complete. This is the most useful finding of the testing phase, and it supports the position taken in Chapter 4 that accessibility claims verified only by inspection are insufficient.

The user acceptance testing was designed and distributed as described in Section 6.4.5, with six user perspectives covering the default learner experience, the three accessibility presets, the educator and the administrator. The responses had not been collected at the time of writing, and none are reported. The two constraints on this chapter's evidence, that the accessibility evaluation is the developer's own

self-assessment and that the UAT participants are proxies rather than neurodivergent learners, are stated in Section 6.5.6 and carried forward into the conclusion in the next chapter.

CHAPTER 7: CONCLUSION

7.1 Introduction

This chapter closes the report. It restates the four objectives set out in Chapter 1 and reports how the design, implementation and testing work addressed each one, states what the project contributes, sets out the limitations that remain after implementation and testing, and identifies the work that should follow.

7.2 Project Summarisation

The ACCESS platform was designed, built and evaluated as a web-based learning management system in which accessibility settings are part of the platform rather than an addition to it. The four objectives stated in Chapter 1 were addressed as follows.

Objective 1: to analyse the accessibility requirements of learners with Dyslexia, ADHD and Autism Spectrum Disorder. This was achieved in Chapter 2 through a review of the Universal Design for Learning framework, WCAG 2.2, the W3C guidance on making content usable for people with cognitive and learning disabilities, and the British Dyslexia Association Style Guide, together with a comparison of four established learning management systems. The comparison identified a specific gap: all four platforms can enlarge text and three can read text aloud, but none offers a single learner-controlled setting that changes typography, reading width, content structure and pacing together. That gap defined the requirements taken forward into Chapter 3, and the standards reviewed became the rule set used by the accessibility compliance checker described in Chapter 4.

Objective 2: to develop a functional accessible platform with adaptive content delivery, text-to-speech, adjustable accessibility settings, lesson and quiz management, and role-based access. This was achieved and forms the bulk of Chapters 4 and 5. The accessibility personalisation engine applies three presets and twenty-four individual settings without reloading the page; the rule-based recommendation engine produces revision, next-lesson and cross-course recommendations with a stated reason for each; quizzes are graded on the server with the answer key withheld until submission; and the learner, educator and administrator roles are separated both in the interface and in the database. Table 5.3 records twenty-one modules as implemented, three of them with a specific documented limitation, and four planned items as not implemented, namely a continuous integration pipeline, automated accessibility scanning, combinable accessibility presets and offline support.

Objective 3: to implement certificate generation and educator analytics. This was achieved. Completion certificates are generated as PDF documents carrying a reference code and a public verification address, so that a certificate can be checked by a third party without an account, and a revoked certificate keeps its record so that verification reports the revocation rather than reporting the certificate as unknown. Educator analytics aggregates lesson progress and quiz results into per-course and per-learner views with at-risk flagging, and the administrator analytics view additionally reports which accessibility features are actually being used.

Objective 4: to evaluate the functional correctness, usability and accessibility of the platform. This was addressed through the four classes of testing reported in Chapter 6, and the form the evaluation actually took should be stated plainly rather than overstated. Nineteen functional test cases covering the public, learner, educator and administrator functions were executed against the running system and all recorded a final result of Pass. Three load scenarios and one concurrency test confirmed that the operations a user action performs hold their response-time targets at a level of concurrency representative of a class-sized group. Accessibility was evaluated by a WCAG 2.2 self-assessment carried out by the project author, which returned five passes, six partial results, two

failures and six criteria that could not be judged without assistive technology. Usability was addressed through User Acceptance Testing, designed as six sessions in which a participant works through the tasks of one predefined user perspective and then completes a questionnaire covering ease of use, navigation, clarity, readability, support for that perspective and overall satisfaction. The responses had not been collected at the time of writing and are therefore not reported in this report.

Two qualifications apply to this objective and are not softened here. The accessibility evaluation is a self-assessment by the developer of the system, not an independent expert review or an audit, and it confers no conformance status. The UAT participants are undergraduate students working to a persona brief, not learners with Dyslexia, ADHD or Autism, so their feedback describes how a briefed user experienced the interface and not the experience of the learners the platform is designed for.

The most useful outcome of the evaluation was not the pass count. It was that seven defects were found in modules that had already been recorded as complete, six of which were failures of what the interface actually renders or of behaviour under concurrent requests rather than of stored data. Two of those defects made a lesson impossible to complete for a learner using a particular accessibility preset. This supports the position taken in Chapter 4 that an accessibility feature verified only by reading the code that implements it cannot be assumed to work.

7.3 Project Contribution

For the faculty, the project delivers a working final-year project artefact together with a method that other data-heavy or accessibility-focused student projects could reuse: the database schema is rebuildable from version-controlled definitions, the demonstration dataset is produced by a repeatable script, and the entity relationship diagrams and data dictionary in Chapter 4 are generated from the live database rather than drawn by hand, so the design documentation cannot fall out of step with the system it describes.

For practice in accessible software development, the most transferable result is the empirical one described above. Several of the corrected defects were problems of

rendering, reachability or timing that no amount of code review would have exposed, and they occurred in features already marked complete. Two patterns used in this project respond directly to that: an accessibility compliance checker that scores authored content against a fixed, cited rule set while the content is being written, and a settings catalogue that refuses to let an accessibility setting exist without a description and a justification. Both are applicable to any content-authoring platform that wants to prevent accessibility controls from silently doing nothing.

For users of the platform itself, a learner receives three accessibility presets that change typography, layout, pacing and motion together and remain adjustable afterwards; an educator receives authoring tools with an accessibility check built into the writing workflow; and an administrator receives the moderation and oversight tools needed to run the platform. The platform also publishes an accessibility statement setting out what it supports and what it does not, so that a prospective user can read the limitations before investing time in it.

7.4 Project Limitation

The limitations visible at the design stage are set out in Section 4.6 and the limitations of the testing in Section 6.5.6. The following are the limitations of the project as a whole and are the ones that most constrain what can be claimed from it.

- **No neurodivergent learner took part in the project.** Nobody with lived experience of Dyslexia, ADHD or Autism took part in designing, building or evaluating ACCESS. The design follows published guidance rather than the people it is intended for, and the UAT used proxy participants working to a persona brief. This is the most significant limitation of the work.
- **No independent accessibility specialist evaluated the system.** The WCAG 2.2 evaluation reported in Chapter 6 is the project author's own self-assessment. The platform has not been audited, and no conformance claim is made anywhere in this report.

- **No assistive technology was used.** Six of the nineteen criteria assessed remain undetermined for this reason, and the screen-reader behaviour of the platform is inferred from its markup rather than observed.
- **The project cannot show that the platform improves learning outcomes.** No measurement of learner performance was taken before the accessibility work began, so no before-and-after comparison is possible even in principle. The evidence in this report shows that the interface adapts as designed; it does not show that the adaptation helps a neurodivergent learner learn better.
- **Verification depends entirely on the developer.** Every test case, every executable check and every defect correction was written, run and judged by the same person who built the feature, and no automated pipeline prevents a regression from reaching the deployed system.
- **Two accessibility failures remain open.** A lesson containing video without a transcript can still be published, and a quiz timer expiring with no accessibility preset active submits the attempt without warning. Both were identified during the accessibility evaluation and are carried forward as future work rather than presented as resolved.

7.5 Future Works

- **Evaluation with the intended users.** The single most valuable next step is a usability study with participants who actually have Dyslexia, ADHD or Autism, using the existing presets as the intervention and a validated instrument such as the System Usability Scale (Brooke, 1996). This would produce the first evidence about the platform's effect that this project does not have.
- **An independent accessibility audit.** An evaluation against WCAG 2.2 Level AA by someone other than the developer, including screen-reader testing, would replace the self-assessment in Chapter 6 with a verified position and would settle the six criteria this project could not judge.

- **Correcting the two open accessibility failures.** The compliance check should be re-applied to lessons published before a rule was introduced, and the quiz timer should offer an extension or a warning to every learner rather than only to those with a preset active.
- **Combinable accessibility presets.** A learner who needs Dyslexia typography together with ADHD pacing must currently choose one preset and adjust the rest by hand. Allowing settings to be combined across presets is the clearest single extension to the design.
- **A continuous integration pipeline.** The five executable checks described in Chapter 5 already catch real regressions but are run by hand. Running them automatically on every change would close the gap identified in Section 5.4.
- **Completing localisation.** The accessibility settings panel's own labels remain in English only, which is a poor outcome for the panel a learner reading in a second language is most likely to need.
- **Repeating the test cases against the deployed application.** All testing reported here was carried out against the local development environment, so the deployed system remains unverified by the same evidence.

7.6 Conclusion

ACCESS set out to show that a learning platform can adapt to a learner's stated accessibility needs rather than requiring the learner to adapt to the platform, and to make that adaptation something that can be checked rather than asserted. The four objectives set in Chapter 1 were met to the extent a single-developer final-year project can meet them. The system is built and working, its database and demonstration data can be rebuilt from the project repository so that the claims in this report can be re-verified, and seven defects that would have undermined its accessibility or its correctness were found and corrected before submission.

What the project does not have is evidence from the people it was built for. The accessibility evaluation is the developer's own, the user acceptance participants were proxies working to a brief, and no measurement of learning outcomes exists. This report states that limitation in each chapter where it applies rather than only here, because it defines what the work has established and what still needs to be done: ACCESS demonstrates that the adaptation is feasible and that it behaves as designed, and the next step is to find out whether it helps.

References

- Anthology Inc. (2023). Blackboard learn. Accessed 2 September 2026, <<https://www.blackboard.com/>>.
- Anthology Inc. (2024a). Anthology ally: Accessibility reporting and alternative formats. Accessed 2 September 2026, <<https://help.blackboard.com/Ally>>.
- Anthology Inc. (2024b). Blackboard learn: Achievements. Accessed 2 September 2026, <<https://help.blackboard.com/Learn/Instructor/Ultra/Performance/Achievements>>.
- Barkley, R. A. (1997). Implications of executive function deficits for treatment of adhd children: Empirical and practical issues. *Journal of Clinical Child Psychology*, 26(2):129–140.
- Batanero, C., Boticario, J. G., et al. (2014). Accessibility of learning management systems: A review. *Journal of Universal Computer Science*, 20(9):1253–1272.
- Bernier, Claudéric (2025). dnd kit — a modern drag and drop toolkit for react. Accessed 2 September 2026, <<https://dndkit.com/>>.
- Braille Institute of America (2021). Atkinson hyperlegible font. Accessed 2 September 2026, <<https://brailleinstitute.org/freefont>>.
- Brainhub (2024). Tiptap: The headless rich text editor framework. Accessed 2 September 2026, <<https://tiptap.dev/>>.
- Brickfield Education Labs (2024). Accessibility toolkit for moodle (brickfield) — plugin directory entry. Accessed 2 September 2026, <https://moodle.org/plugins/tool_brickfield>.
- British Dyslexia Association (2023). Dyslexia style guide 2023: Creating dyslexia friendly content. Accessed 2 September 2026, <<https://www.bdadyslexia.org.uk/advice/employers/creating-a-dyslexia-friendly-workplace/dyslexia-friendly-style-guide>>.

Brooke, J. (1996). Sus: A “quick and dirty” usability scale. In Jordan, P. W., Thomas, B., Weerdmeester, B. A., and McClelland, I. L., editors, *Usability evaluation in industry*, pages 189–194. Taylor & Francis.

Burl, M. and O’Brien, K. (2024). Dropout rates and accessibility barriers in e-learning for neurodivergent students. *Computers & Education*, 198:104–118.

Docker, Inc. (2025). Docker desktop documentation. Accessed 2 September 2026, <<https://docs.docker.com/desktop/>>.

Figma (2023). Figma: The collaborative interface design tool. Accessed 2 September 2026, <<https://www.figma.com/>>.

Framer (2024). Framer motion: Production-ready motion library for react. Accessed 2 September 2026, <<https://www.framer.com/motion/>>.

Franceschini, S., Gori, S., Ruffino, M., Viola, S., Molteni, M., and Facoetti, A. (2013). Action video games make dyslexic children read better. *Current Biology*, 23(6):462–466.

Gonzalez, Abelardo (2024). Opendyslexic: A typeface for dyslexia. Accessed 2 September 2026, <<https://opendyslexic.org/>>.

Google DeepMind (2025). Gemini api documentation. Accessed 2 September 2026, <<https://ai.google.dev/gemini-api/docs>>.

Google LLC (2023). Google classroom. Accessed 2 September 2026, <<https://classroom.google.com/>>.

Google LLC (2024). Use google classroom with a screen reader and accessibility tools. Accessed 2 September 2026, <<https://support.google.com/edu/classroom/answer/6272747>>.

Google LLC (2025). Youtube iframe player api reference. Accessed 2 September 2026, <https://developers.google.com/youtube/iframe_api_reference>.

Heiderich, Mario and Cure53 (2025). Dompurify: A dom-only xss sanitizer for html. Accessed 2 September 2026, <<https://github.com/cure53/DOMPurify>>.

Instructure, Inc. (2023). Canvas learning management system. Accessed 2 September 2026, <<https://www.instructure.com/canvas>>.

Instructure, Inc. (2024a). Canvas accessibility standards and voluntary product accessibility template. Accessed 2 September 2026, <<https://www.instructure.com/canvas/accessibility>>.

Instructure, Inc. (2024b). How do i use microsoft immersive reader in a canvas course? Accessed 2 September 2026, <<https://community.canvaslms.com/t5/Instructor-Guide/tkb-p/Instructor>>.

Instructure, Inc. (2024c). How do i use the accessibility checker in the rich content editor? Accessed 2 September 2026, <<https://community.canvaslms.com/t5/Instructor-Guide/tkb-p/Instructor>>.

Kincaid, J. P., Fishburne, R. P., Rogers, R. L., and Chissom, B. S. (1975). Derivation of new readability formulas (automated readability index, fog count and flesch reading ease formula) for navy enlisted personnel. Technical Report Research Branch Report 8-75, Naval Technical Training Command, Millington TN Research Branch.

Lawson, W. (2011). *The Passionate Mind: How People with Autism Learn*. London: Jessica Kingsley Publishers.

Meta Platforms, Inc. (2023). React: The library for web and native user interfaces. Accessed 2 September 2026, <<https://react.dev/>>.

Meyer, A., Rose, D. H., and Gordon, D. (2014). *Universal Design for Learning: Theory and Practice*. Wakefield, MA: CAST Professional Publishing.

Microsoft (2023). Typescript: Javascript with syntax for types. Accessed 2 September 2026, <<https://www.typescriptlang.org/docs/>>.

MoodleHQ (2023a). Moodle architecture: Technical overview. Accessed 2 September 2026, <<https://docs.moodle.org/dev/Architecture>>.

MoodleHQ (2023b). Moodle: Open-source learning platform. Accessed 2 September 2026, <<https://moodle.org/>>.

MoodleHQ (2024a). Moodle accessibility checker (atto and tinymce editor plugins). Accessed 2 September 2026, <https://docs.moodle.org/en/Accessibility_checker>.

MoodleHQ (2024b). Moodle accessibility documentation. Accessed 2 September 2026, <<https://docs.moodle.org/en/Accessibility>>.

Mozilla Developer Network (2025). Web speech api — speech synthesis. Accessed 2 September 2026, <https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API>.

Munoz, D. P., Armstrong, I. T., Hampton, K. A., and Moore, K. D. (2003). Saccadic eye movements in attention deficit hyperactivity disorder. *Clinical Neurophysiology*, 114(9):1667–1674.

Parallax Agency Ltd. (2025). jspdf: Client-side javascript pdf generation. Accessed 2 September 2026, <<https://github.com/parallax/jspdf>>.

PostgreSQL Global Development Group (2025). Postgresql 17 documentation. Accessed 2 September 2026, <<https://www.postgresql.org/docs/17/>>.

PostgREST Contributors (2025). Postgrest: Rest api for any postgresql database. Accessed 2 September 2026, <<https://postgrest.org/>>.

Prensky, M. (2001). Digital natives, digital immigrants part 1. *On the Horizon*, 9(5):1–6.

Recharts Team (2024). Recharts: A composable charting library built on react components. Accessed 2 September 2026, <<https://recharts.org/>>.

Rello, L. and Baeza-Yates, R. (2013). Good fonts for dyslexia. In *Proceedings of the 15th International ACM SIGACCESS Conference on Computers and Accessibility*, pages 1–8.

Schwaber, K. (1997). Scrum development process. In *Business Object Design and Implementation: OOPSLA’95 Workshop Proceedings*, pages 117–134. Springer.

Spencer, M. and Podesta, F. (2022). Neurodiversity in higher education: A comprehensive review of progress and challenges. *Journal of Applied Higher Education*, 13(2):45–62.

Supabase (2023). Supabase documentation: The open source firebase alternative. Accessed 2 September 2026, <<https://supabase.com/docs>>.

Supabase (2024). Supabase row level security documentation. Accessed 2 September 2026, <<https://supabase.com/docs/guides/auth/row-level-security>>.

Supabase Inc. (2025a). Supabase auth documentation. Accessed 2 September 2026, <<https://supabase.com/docs/guides/auth>>.

Supabase Inc. (2025b). Supabase cli and local development. Accessed 2 September 2026, <<https://supabase.com/docs/guides/local-development>>.

Tailwind Labs (2023). Tailwind css documentation. Accessed 2 September 2026, <<https://tailwindcss.com/docs>>.

Vercel Inc. (2023a). Next.js documentation: Server-side rendering (ssr). Accessed 2 September 2026, <<https://nextjs.org/docs>>.

Vercel Inc. (2023b). Vercel documentation: The platform for frontend developers. Accessed 2 September 2026, <<https://vercel.com/docs>>.

Vercel Inc. (2024). Next.js middleware documentation. Accessed 2 September 2026, <<https://nextjs.org/docs/app/building-your-application/routing/middleware>>.

W3C Web Accessibility Initiative (2021). Making content usable for people with cognitive and learning disabilities. Accessed 2 September 2026, <<https://www.w3.org/TR/coga-usable/>>.

W3C Web Accessibility Initiative (2023). Web content accessibility guidelines (wcag) 2.2. Accessed 2 September 2026, <<https://www.w3.org/TR/WCAG22/>>.

Zelazo, P. D., Muller, U., Frye, D., and Marcovitch, S. (2003). The development of executive function in early childhood. *Monographs of the Society for Research in Child Development*, 68(3):vii–137.